

1 Appendix A: Barcode Metric and Threshold-Gap Repair

1.1 Scope and relation to Chapter 2

Appendix A is the detailed persistence-theoretic Foundation Appendix for Chapter 2 in the AK-HPDST v19.0.0 *Threshold-Gap Metric Repair Release*. The legacy transposed acronym AK-HDPST may occur in historical artifacts, but it has no normative use in this appendix.

This appendix supplies the definitions, counterexamples, proofs, exact hypotheses, and audit-facing records behind the Chapter 2 contract. It does not enlarge the Core and does not create an external-domain theorem.

The two central repairs are the following.

For a positive threshold $\tau > 0$, hard barcode threshold deletion T_τ^{bar} is neither a global exact Serre localization nor a globally non-expansive operation under the standard bottleneck metric. Its valid metric replacement is the sharp two-sided threshold-gap-safe theorem.

Window restriction and hard threshold deletion commute exactly when every globally retained bar, including every infinite bar, is either disjoint from the window or remains retained after clipping. A criterion that checks only finite globally retained bars is false on bounded windows.

Accordingly, this appendix establishes and records:

- (i) constructible one-parameter persistence and barcode normal forms over a field;
- (ii) the standard finite/infinite bottleneck convention and interleaving isometry;
- (iii) failure of the positive-threshold Serre property;
- (iv) failure of global non-expansiveness for hard threshold deletion;
- (v) sharp two-sided threshold-gap-safe stability with closed band $[\tau - \varepsilon, \tau + \varepsilon]$;
- (vi) the identity $\text{gap}_\tau = \text{clear}_\tau$, with no factor $1/2$;
- (vii) typed scalarization of a quantale-valued metric ledger, together with evidence that the scalarized value bounds the actual bottleneck discrepancy consumed by the proof;
- (viii) the fixed order window $\rightarrow$ pre-threshold profile $\rightarrow$ hard deletion;
- (ix) the distinction between exact comparison and metric threshold-gap-certified comparison;
- (x) the infinite-bar obstruction to the former finite-only commutation condition;
- (xi) an exact necessary-and-sufficient crop-collapse commutation criterion;
- (xii) conditional functoriality and exactness only in declared τ -split or exact-admissible regimes;
- (xiii) actual-morphism, terminal-tameness, and monitored-scope requirements for Type IV diagnostics;
- (xiv) the distinction between manifest completeness and mathematical certification.

Remark 1.1 (Metric repair relative to v18). Earlier v18 formulations correctly rejected the overstrong Serre-localization interpretation of positive-threshold deletion, but still retained language suggesting global barcode non-expansiveness. The v19 repair removes that remaining overclaim. The replacement theorem is Theorem 1.33.

Remark 1.2 (What this appendix does not do). This appendix is a Foundation Appendix for the persistence-theoretic Core. It does not prove an arithmetic, PDE, geometric, categorical, or cryptographic theorem. Any external-domain conclusion requires a separately proved Interface or Bridge Theorem.

1.2 Constructible one-parameter persistence

Fix a field k . Let

$$\text{Pers}_k := [(\mathbb{R}, \leq), \text{Vect}_k]$$

be the category of one-parameter persistence modules over k , indexed by the ordered real line.

Definition 1.3 (Constructible persistence module). A persistence module $M \in \text{Pers}_k$ is called *constructible* if:

- (i) $M(t)$ is finite-dimensional over k for every $t \in \mathbb{R}$;
- (ii) on every bounded interval $J \subset \mathbb{R}$, there is a finite critical set outside which all structure maps are isomorphisms;
- (iii) there is no accumulation of critical parameters inside a bounded interval.

The full subcategory is denoted by

$$\text{Pers}_k^{\text{cons}} \subset \text{Pers}_k.$$

Remark 1.4 (Working regime). All Core statements in this appendix are made in the pointwise finite-dimensional constructible one-parameter regime over k , or for finite barcode profiles obtained from declared windowed Core evaluations. Wild, nonconstructible, infinite-rank, or uncontrolled multiparameter data are outside scope unless an Interface theorem reduces them to declared constructible one-parameter profiles.

Definition 1.5 (Finite-type filtered chain complex). Let $\text{Ch}^b(k)$ be the category of bounded chain complexes of finite-dimensional k -vector spaces. A *finite-type filtered chain complex* is a pair

$$F = (C_\bullet, \{F^t C_\bullet\}_{t \in \mathbb{R}})$$

such that:

- (i) $F^t C_\bullet \subseteq F^{t'} C_\bullet$ whenever $t \leq t'$;
- (ii) the filtration is exhaustive and left-bounded;
- (iii) for every degree i , the persistence module

$$\mathbf{P}_i(F) : \mathbb{R} \longrightarrow \text{Vect}_k, \quad t \longmapsto H_i(F^t C_\bullet)$$

belongs to $\text{Pers}_k^{\text{cons}}$.

The category of such objects and filtration-preserving chain maps is denoted by

$$\text{FiltCh}(k).$$

Remark 1.6 (Persistence layer first). The Core reads F through monitored persistence profiles

$$\mathbf{P}_i(F).$$

Any filtered-level or derived-level use requires an additional representative policy. Barcode-level deletion does not by itself supply a global strict filtered lift.

1.3 Barcode normal forms and bar lengths

Definition 1.7 (Interval module). For an interval $I \subseteq \mathbb{R}$, the interval module k_I is defined by

$$k_I(t) = \begin{cases} k, & t \in I, \\ 0, & t \notin I, \end{cases}$$

with identity structure maps when both parameters lie in I , and zero otherwise.

Definition 1.8 (Bar and barcode). An interval summand k_I is a *bar*. A barcode of $M \in \text{Pers}_k^{\text{cons}}$ is an interval decomposition

$$M \cong \bigoplus_{\lambda \in \Lambda} k_{I_\lambda}$$

with locally finite support on bounded intervals. The multiset of bars is

$$\bar{(M)} = \{I_\lambda\}_{\lambda \in \Lambda}.$$

Theorem 1.9 (Interval decomposition). Every $M \in \text{Pers}_k^{\text{cons}}$ admits a locally finite interval decomposition

$$M \cong \bigoplus_{\lambda \in \Lambda} k_{I_\lambda}.$$

The barcode is unique up to permutation of interval summands.

Proof sketch. This is the standard interval-decomposition theorem for pointwise finite-dimensional one-parameter persistence modules over a field. Constructibility gives local finite type, and uniqueness follows from the Krull–Schmidt property in the interval-decomposable regime. $\square$

Definition 1.10 (Length of a bar). If I is finite with endpoint values $a < b$, define

$$\ell(I) := b - a.$$

If $I = [a, \infty)$, up to harmless endpoint decoration at a , define

$$\ell(I) := +\infty$$

for thresholding purposes.

Remark 1.11 (Endpoint independence). The value $\ell(I)$ depends only on endpoint values, not on finite-endpoint open/closed decoration. Every threshold rule below is therefore independent of that bookkeeping.

1.4 Standard bottleneck metric and infinite-bar convention

Definition 1.12 (Standard bottleneck distance with infinite bars). Let B, C be finite barcode profiles, or locally finite constructible profiles restricted to a declared bounded audit range. The bottleneck distance $d_B(B, C)$ is computed by partial matchings of bars and diagonal copies.

The infinite-bar convention is:

- (i) a finite bar I may be matched to the diagonal with cost $\ell(I)/2$;
- (ii) an infinite bar is never matched to the diagonal at finite cost;
- (iii) a finite bar and an infinite bar never match at finite cost;

(iv) $[a, \infty)$ and $[a', \infty)$ may match with cost

$$|a - a'|.$$

Lemma 1.13 (Infinite-bar threshold invariance). *For every finite $\tau > 0$,*

$$T_\tau^{\text{bar}}([a, \infty)) = [a, \infty).$$

Moreover, an infinite bar never lies in a finite threshold band

$$[\tau - \varepsilon, \tau + \varepsilon].$$

Proof. An infinite bar has threshold length $+\infty$, whereas hard deletion removes only finite bars of length at most τ . $\square$

Remark 1.14 (Bottleneck–interleaving isometry). For q -tame or constructible one-parameter persistence modules over k , the isometry theorem gives

$$d_{\text{int}}(M, N) = d_B(\bar{(M)}, \bar{(N)}).$$

The equality is interpreted under Definition 1.12. Consequently, a d_B -statement may be transferred to d_{int} only when the persistence objects and barcode profiles compared are exactly those appearing in the certified record.

1.5 Positive-threshold short bars are not Serre in general

Proposition 1.15 (Positive-threshold short-bar objects are not Serre in general). *Let $\tau > 0$. The class of constructible persistence modules all of whose barcode summands have finite length at most τ is not extension-closed in general. Hence it is not a Serre subcategory of $\text{Pers}_k^{\text{cons}}$ in general.*

Proof. For $\tau = 1$, consider

$$0 \longrightarrow k_{[1,2)} \longrightarrow k_{[0,2)} \longrightarrow k_{[0,1)} \longrightarrow 0.$$

The subobject and quotient have length 1, while the middle interval has length 2. $\square$

Corollary 1.16 (No global exact reflector claim). *Hard positive-threshold deletion is not asserted to arise globally from a Serre localization, an abelian exact reflector, or a global exact endofunctor on all of $\text{Pers}_k^{\text{cons}}$.*

Remark 1.17 (Why this guard-rail matters). Without this restriction, exactness, functoriality, filtered lifts, and tower comparison maps would inherit unsupported categorical strength. Stronger calculus must be proved in a declared τ -split or exact-admissible regime.

1.6 Hard barcode threshold deletion

Definition 1.18 (Hard barcode threshold deletion). For $\tau > 0$, define

$$T_\tau^{\text{bar}}(B) := \{ I \in B \mid \ell(I) > \tau \text{ or } \ell(I) = +\infty \}.$$

Thus finite bars of length $\leq \tau$ are deleted, finite bars of length $> \tau$ are retained, and infinite bars are retained. For a module M , the notation $T_\tau^{\text{bar}}(M)$ means that the operation is applied to $\bar{(M)}$ at barcode-profile level.

Remark 1.19 (Status of T_τ^{bar}). In the general Core, T_τ^{bar} is an operation on barcode normal forms, hence on isomorphism classes. It is not asserted to be a strict exact endofunctor on all morphisms.

Proposition 1.20 (Basic properties of hard threshold deletion). *For every $\tau > 0$, T_τ^{bar} is:*

- (i) *endpoint-independent;*
- (ii) *idempotent;*
- (iii) *infinite-bar preserving;*
- (iv) *threshold-monotone.*

Explicitly,

$$T_\tau^{\text{bar}} T_\tau^{\text{bar}}(B) = T_\tau^{\text{bar}}(B).$$

Proof. Each assertion follows directly from deletion by the predicate $\ell(I) \leq \tau$ on finite bars. $\square$

Proposition 1.21 (Threshold composition). *For $\tau, \sigma > 0$,*

$$T_\sigma^{\text{bar}} T_\tau^{\text{bar}}(B) = T_{\max\{\tau, \sigma\}}^{\text{bar}}(B).$$

Proof. The composite deletes exactly the finite bars whose length is at most one of the two thresholds, equivalently at most their maximum. $\square$

1.7 Counterexample to global non-expansiveness

Example 1.22 (Failure of global non-expansiveness). Fix $\tau > 0$ and $0 < \eta < \tau/4$. Let

$$B = \{[0, \tau + \eta)\}, \quad C = \{[0, \tau - \eta)\}.$$

Matching the two bars gives cost 2η , while matching both to the diagonal costs $(\tau + \eta)/2 > 2\eta$. Hence

$$d_B(B, C) = 2\eta.$$

After deletion,

$$T_\tau^{\text{bar}}(B) = \{[0, \tau + \eta)\}, \quad T_\tau^{\text{bar}}(C) = \emptyset,$$

and therefore

$$d_B(T_\tau^{\text{bar}}(B), T_\tau^{\text{bar}}(C)) = \frac{\tau + \eta}{2} > 2\eta.$$

Thus global non-expansiveness fails.

Remark 1.23 (Meaning of the counterexample). A small endpoint perturbation can move a finite bar across the hard classification boundary $\ell = \tau$. The correct stability theorem must therefore exclude both input barcodes from a neighborhood of the threshold.

Corollary 1.24 (Rejected global non-expansiveness claim). *Global non-expansiveness of hard positive-threshold deletion under the standard bottleneck metric is rejected in v19 and may not be used in a v19 proof.*

1.8 Threshold clearance and gap

Definition 1.25 (Threshold clearance). For a barcode profile B , define

$$\text{clear}_\tau(B) := \inf_{I \in B_{\text{fin}}} |\ell(I) - \tau|.$$

If B has no finite bars, set $\text{clear}_\tau(B) = +\infty$. If B contains a finite bar of length exactly τ , then $\text{clear}_\tau(B) = 0$.

Definition 1.26 (Threshold gap as ledger alias). For one profile,

$$\text{gap}_\tau(B) := \text{clear}_\tau(B).$$

For a finite family,

$$\text{gap}_\tau(B_1, \dots, B_m) := \min_j \text{clear}_\tau(B_j).$$

Thus gap_τ is a ledger-facing alias, not a second invariant, and there is no factor $1/2$.

Definition 1.27 (Typed bottleneck scalarization). Let

$$(\mathbb{V}, \oplus, 0_{\mathbb{V}}, \preceq)$$

be the declared metric-ledger quantale. A bottleneck scalarization is a monotone, subadditive map

$$\text{val}_{\mathbb{B}} : \mathbb{V} \longrightarrow [0, +\infty]$$

with

$$\text{val}_{\mathbb{B}}(0_{\mathbb{V}}) = 0, \quad \text{val}_{\mathbb{B}}(q \oplus q') \leq \text{val}_{\mathbb{B}}(q) + \text{val}_{\mathbb{B}}(q').$$

For $\mathfrak{d}_{\text{met}} \in \mathbb{V}$, define

$$\varepsilon_{\mathbb{B}}(\mathfrak{d}_{\text{met}}) := \text{val}_{\mathbb{B}}(\mathfrak{d}_{\text{met}}).$$

A certified use additionally contains a theorem, verified computation, or artifact establishing that the relevant actual discrepancy satisfies

$$d_B(B, C) \leq \varepsilon_{\mathbb{B}}(\mathfrak{d}_{\text{met}}).$$

The shorthand

$$\Sigma\delta < \text{gap}_\tau$$

means the typed real inequality

$$\varepsilon_{\mathbb{B}}(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau(\mathfrak{B})$$

for the declared protected family $\mathfrak{B}$.

Only metric defects belong to $\mathfrak{d}_{\text{met}}$. Representative existence, actual morphisms, eligibility, terminal tameness, higher coherence, claim status, manifest consistency, and artifact identity are non-scalar obligations.

Remark 1.28 (Sharp threshold-safety margin versus scalarization tightness). Under an ε -bottleneck matching, a finite bar length may change by at most 2ε . Nevertheless, for a two-sided certificate the sharp forbidden band is

$$[\tau - \varepsilon, \tau + \varepsilon],$$

not

$$[\tau - 2\varepsilon, \tau + 2\varepsilon].$$

The word *sharp* refers to the optimal half-width for a true bottleneck bound; it does not assert that a chosen scalarization is numerically tight.

Definition 1.29 (Windowed pre-threshold profile). For $F \in \text{FiltCh}(k)$, monitored degree i , and window W , define

$$B_{i,W}(F) := W_W \mathbf{P}_i(F).$$

Threshold clearance and threshold gap are measured on this profile. The repaired evaluation is

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(B_{i,W}(F)).$$

Remark 1.30 (Clearance is measured before deletion, after windowing). Clearance is measured on

$$B_{i,W}(F) = W_W \mathbf{P}_i(F),$$

not on the already deleted profile. Once a bar has been deleted, its threshold proximity can no longer be audited.

Definition 1.31 (Threshold-gap record). A threshold-gap record for a metric comparison of two repaired evaluations contains:

- (i) immutable identifiers for the two windowed pre-threshold profiles B and C ;
- (ii) the monitored degree, common window, and threshold;
- (iii) the finite-bar length data or certified summaries for both profiles;
- (iv) $\text{clear}_\tau(B)$, $\text{clear}_\tau(C)$, and $\text{gap}_\tau(B, C)$;
- (v) the metric-ledger element $\mathbf{d}_{\text{met}}$;
- (vi) the scalarization val_B and

$$\varepsilon_B := \text{val}_B(\mathbf{d}_{\text{met}});$$

- (vii) evidence that

$$d_B(B, C) \leq \varepsilon_B;$$

- (viii) confirmation that neither finite-bar family meets

$$[\tau - \varepsilon_B, \tau + \varepsilon_B];$$

- (ix) every crop or window-edge component used to derive the windowed bound;
- (x) artifact references sufficient to reconstruct the comparison;
- (xi) the specialized status

$$\text{gap_certified}, \quad \text{gap_failed}, \quad \text{gap_pending}.$$

The record is two-sided by definition. The specialized statuses map respectively to the atomic statuses `pass`, `reject`, and `undefined`.

For a single profile, a clearance record may be stored as a readiness record, but it does not by itself certify a comparison.

Remark 1.32 (Length exactly τ is not gap-safe). A finite bar of length exactly τ is deleted by the rule $\ell(I) \leq \tau$, but it gives $\text{clear}_\tau = 0$. The evaluation is defined, whereas a positive robustness certificate at that threshold fails.

1.9 Threshold-gap-safe stability

Theorem 1.33 (Metric-repair theorem: sharp threshold-gap-safe stability). *Let B, C be finite barcodes, possibly with infinite bars, under Definition 1.12. Let $\tau > 0$, $\varepsilon \geq 0$, and suppose*

$$d_B(B, C) \leq \varepsilon.$$

Assume no finite bar of either barcode has length in

$$[\tau - \varepsilon, \tau + \varepsilon].$$

Then

$$d_B(T_\tau^{\text{bar}}(B), T_\tau^{\text{bar}}(C)) \leq \varepsilon.$$

Proof. If $\varepsilon = 0$, the barcodes agree and deterministic deletion gives the result.

Assume $\varepsilon > 0$ and choose a bottleneck matching of cost at most ε , which exists for finite barcodes.

Infinite bars match only infinite bars and are retained on both sides.

Let I be a retained finite bar of B . The closed-band hypothesis gives

$$\ell(I) > \tau + \varepsilon.$$

If I is matched to the diagonal, retain that diagonal match. If it is matched to a finite bar $J \in C$, then

$$|\ell(I) - \ell(J)| \leq 2\varepsilon.$$

If J were deleted, then the closed-band hypothesis on C would force

$$\ell(J) < \tau - \varepsilon,$$

hence

$$\ell(I) - \ell(J) > 2\varepsilon,$$

a contradiction. Thus J is retained.

The symmetric argument applies to retained bars of C . After removing pairs deleted on both sides, the original matching therefore restricts, together with retained-to-diagonal matches, to a matching of the retained barcodes of cost at most ε . $\square$

Example 1.34 (Sharpness of the ε -band). Let $0 < c < \varepsilon$, choose $0 < \eta < \varepsilon - c$, and choose τ so large that the diagonal costs below exceed ε . Set

$$L_+ = \tau + c + \eta, \quad L_- = \tau - c - \eta$$

and take centered bars I, J of lengths L_+, L_- . Both lengths avoid the smaller band $[\tau - c, \tau + c]$, and

$$d_B(\{I\}, \{J\}) = c + \eta < \varepsilon.$$

But I is retained, J is deleted, and the post-deletion distance is $L_+/2 > \varepsilon$. Thus no smaller two-sided half-width $c < \varepsilon$ suffices in general.

Remark 1.35 (Two-sided certificate). It is insufficient for only one barcode to avoid the threshold band. A one-sided 2ε -type margin may be used as a separately declared conservative policy, but it is not the sharp two-sided v19 theorem.

Corollary 1.36 (Scalarized-ledger-safe threshold classification). *Let*

$$\varepsilon_B := \text{val}_B(\mathfrak{d}_{\text{met}}).$$

Suppose the record proves

$$d_B(B, C) \leq \varepsilon_B$$

and

$$\varepsilon_B < \text{gap}_\tau(B, C).$$

Then

$$d_B(T_\tau^{\text{bar}}(B), T_\tau^{\text{bar}}(C)) \leq \varepsilon_B.$$

Proof. The strict inequality implies two-sided avoidance of the closed band with half-width ε_B . Apply Theorem 1.33. $\square$

1.10 Window restriction and window-edge budget

Definition 1.37 (Right-open window). A domain window is

$$W = [u, u'), \quad u' \in \mathbb{R} \cup \{+\infty\}.$$

A finite u' gives a bounded audit window; $u' = +\infty$ gives a terminal audit window.

Definition 1.38 (Window restriction). Define $W_W(M)$ by restriction to W , with extension by zero outside W whenever a globally indexed module is required. At barcode level, bars are clipped to $I \cap W$, and empty intersections are discarded.

Remark 1.39 (Infinite bars under window restriction). On a bounded window, an infinite bar $[a, \infty)$ is clipped to

$$[\max\{a, u\}, u')$$

and therefore becomes finite. On a terminal window it remains infinite.

Proposition 1.40 (No unconditional crop–collapse commutation). *In general,*

$$W_W T_\tau^{\text{bar}}(B) \neq T_\tau^{\text{bar}} W_W(B).$$

Proof. Take $B = \{[0, 10)\}$, $W = [0, 1)$, and $\tau = 2$. Global deletion retains the length-10 bar, which clips to $[0, 1)$. Windowing first produces a length-1 bar, which is then deleted. $\square$

Definition 1.41 (Window-edge clearance). A window-edge clearance record supplies a lower bound on the distance of relevant endpoints or critical parameters from the active window boundaries. Its role is to justify that cropping does not silently gain or lose relevant bars under a source perturbation.

Definition 1.42 (Crop budget). A crop budget

$$\delta_{\text{crop}}(i, W)$$

is a metric-ledger component bounding discrepancy introduced by restriction, clipping, endpoint uncertainty, or moving window boundaries. A source-level perturbation may be transported to a windowed pre-threshold bound only through a theorem proving zero crop error or through a ledger that includes the crop contribution.

Remark 1.43 (Gap-safe stability consumes windowed profile distance). Theorem 1.33 consumes

$$d_B(B, C) \leq \varepsilon$$

for the actual profiles to which hard deletion is applied. In AK applications these are the windowed pre-threshold profiles. It does not itself prove source-to-window metric stability.

Example 1.44 (Infinite-bar obstruction to the former finite-only condition). Let

$$B = \{[0, +\infty)\}, \quad W = [0, 1), \quad \tau = 2.$$

Global threshold deletion retains the infinite bar, after which cropping gives $[0, 1)$. Windowing first turns the bar into a finite length-1 bar, which is deleted. Thus infinite globally retained bars must be checked.

Definition 1.45 (τ -window-adapted condition). The pair (B, W) is τ -window-adapted if every bar retained globally by T_τ^{bar} , including every infinite bar, is either:

- (i) disjoint from W ; or
- (ii) clipped to a bar retained by threshold deletion on W .

Proposition 1.46 (Exact crop–collapse commutation criterion). For a barcode B , right-open window W , and $\tau > 0$, the following are equivalent:

- (i) (B, W) is τ -window-adapted;
- (ii)

$$W_W T_\tau^{\text{bar}}(B) = T_\tau^{\text{bar}} W_W(B)$$

as barcode multisets, including multiplicity.

Proof. A globally deleted finite bar has length at most τ , and clipping cannot increase its length, so it is deleted in both orders. A globally retained bar contributes equally in both orders exactly when it is disjoint from the window or its nonempty crop remains retained. Multiplicity makes this criterion necessary as well as sufficient. $\square$

Remark 1.47 (Default policy and audit use). The Core does not assume τ -window-adaptedness. The default order is window first, threshold second. Commutation may be used only when Definition 1.45 is verified for every globally retained bar and recorded as an artifact-backed non-scalar obligation.

1.11 Core windowed evaluation

Definition 1.48 (Windowed repaired evaluation). For $F \in \text{FiltCh}(k)$, monitored degree i , window W , and $\tau > 0$, define

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Remark 1.49 (Fixed order). The notation means the ordered pipeline

$$F \mapsto \mathbf{P}_i(F) \mapsto B_{i,W}(F) \mapsto T_\tau^{\text{bar}}(B_{i,W}(F)).$$

Definition 1.50 (Repaired evaluation record). A repaired evaluation record contains:

- (i) immutable source-object and profile identifiers;

- (ii) degree i , window W , and threshold τ ;
- (iii) the pre-threshold profile $B_{i,W}(F)$;
- (iv) the repaired profile $\text{Eval}_{i,W,\tau}(F)$;
- (v) the comparison mode

`exact,      metric_gap_certified,      not_compared;`

- (vi) reconstructible artifact references.

Definition 1.51 (Threshold-gap-certified repaired evaluation record). A repaired evaluation record is threshold-gap-certified only when every metric comparison transporting through hard deletion is supplemented by the two-sided record of Definition 1.31.

An exact equality or proved profile isomorphism may instead be stored with comparison mode `exact`. Such a record does not automatically certify robustness under a positive perturbation.

Remark 1.52 (Evaluation is definable without certification). The repaired profile is definitionally available once the pre-threshold profile is available. Threshold-gap certification is required only for metric transport or a claim of stable threshold classification. It is not fabricated for a purely exact identity.

1.12 Overlap Gate: post-threshold comparison and pre-threshold gap

Definition 1.53 (Overlap pre-threshold profiles). For local inputs F_α, F_β and

$$W_{\alpha\beta} = W_\alpha \cap W_\beta,$$

define

$$\begin{aligned} B_{i,W_{\alpha\beta}}^\alpha &:= \mathbb{W}_{W_{\alpha\beta}} \mathbf{P}_i(F_\alpha), \\ B_{i,W_{\alpha\beta}}^\beta &:= \mathbb{W}_{W_{\alpha\beta}} \mathbf{P}_i(F_\beta). \end{aligned}$$

The repaired overlap profiles are obtained by applying T_τ^{bar} to these two inputs.

Remark 1.54 (Two-level structure of the Overlap Gate). The threshold-gap theorem requires a discrepancy bound on the *pre-threshold* profiles. The theorem then transports that bound to the *post-threshold* repaired profiles. A post-threshold comparison alone is not a substitute for the pre-threshold hypothesis.

Proposition 1.55 (Overlap transfer under a scalarized threshold-gap certificate). *Let*

$$\varepsilon_{\alpha\beta,i} := \text{val}_B(\mathfrak{d}_{\alpha\beta,i}).$$

Suppose the record proves

$$d_B(B_{i,W_{\alpha\beta}}^\alpha, B_{i,W_{\alpha\beta}}^\beta) \leq \varepsilon_{\alpha\beta,i}$$

and

$$\varepsilon_{\alpha\beta,i} < \text{gap}_\tau(B_{i,W_{\alpha\beta}}^\alpha, B_{i,W_{\alpha\beta}}^\beta).$$

Then

$$d_B(\text{Eval}_{i,W_{\alpha\beta},\tau}(F_\alpha), \text{Eval}_{i,W_{\alpha\beta},\tau}(F_\beta)) \leq \varepsilon_{\alpha\beta,i}.$$

In the declared constructible regime, the same bound may be read using d_{int} .

Proof. Apply Corollary 1.36 to the two pre-threshold overlap profiles. □

1.13 Exact-admissible regimes

Definition 1.56 (τ -split object). An object $M \in \text{Pers}_k^{\text{cons}}$ is τ -split if it is equipped with declared data

$$M \cong M_{\leq \tau} \oplus M_{> \tau}$$

such that finite bars in $M_{\leq \tau}$ have length $\leq \tau$, finite bars in $M_{> \tau}$ have length $> \tau$, and all infinite bars lie in $M_{> \tau}$.

Definition 1.57 (τ -split morphism). A morphism $f : M \rightarrow N$ between τ -split objects is τ -split if

$$f(M_{\leq \tau}) \subseteq N_{\leq \tau}, \quad f(M_{> \tau}) \subseteq N_{> \tau}.$$

Definition 1.58 (τ -split regime). A τ -split regime is a declared subcategory or workflow domain in which every object and morphism under consideration is τ -split with the chosen decompositions as part of the data.

Proposition 1.59 (Functorial deletion in a τ -split regime). *Inside a declared τ -split regime, the assignment*

$$M \mapsto M_{> \tau}$$

is functorial and agrees with $T_\tau^{\text{bar}}(M)$ on barcode profiles.

Proof. Every declared morphism restricts to the retained summands. □

Definition 1.60 (Exact-admissible collapse regime). An exact-admissible collapse regime is a τ -split regime in which the retained-summand assignment is additionally proved to preserve every exact sequence used by the relevant argument.

Remark 1.61 (Exactness is conditional). Exactness is not a global Core theorem. A proof object invoking it records the decomposition policy, morphism compatibility, exact sequences tested, and the proof or artifact establishing exact preservation.

1.14 Filtered representatives of windowed collapse

Definition 1.62 (Windowed collapsed persistence profile). For $F \in \text{FiltCh}(k)$, define

$$\mathcal{P}_{i, W, \tau}(F) := \text{Eval}_{i, W, \tau}(F).$$

Definition 1.63 (Admissible filtered representative). A filtered object $C_{\tau, W}F$ is an admissible representative if, for every monitored degree i ,

$$\mathbf{P}_i(C_{\tau, W}F) \cong \mathcal{P}_{i, W, \tau}(F)$$

at the declared profile-equivalence strength. Existence is an independent non-scalar obligation.

Remark 1.64 (No global strict lift by default). The Core does not assert a global strict endofunctor

$$C_\tau : \text{FiltCh}(k) \longrightarrow \text{FiltCh}(k).$$

The default notation $C_{\tau, W}F$ denotes a declared window-relative representative.

Definition 1.65 (Representative equivalence). Two representatives are equivalent for a declared gate if their monitored persistence profiles agree in every monitored degree under the declared barcode or persistence-profile equivalence policy. This does not automatically imply chain isomorphism, filtered quasi-isomorphism, or derived equivalence.

Remark 1.66 (Filtered quasi-isomorphism). Filtered quasi-isomorphism may be required when the argument uses filtered-chain or derived constructions. It is not required merely to define a barcode-level repaired profile.

1.15 Consequences for $B\text{-Gate}^+$

Definition 1.67 (Windowed B -side input). The B -side input is

$$\text{Eval}_{i,W,\tau}(F),$$

or a verified representative $C_{\tau,W}F$ realizing that profile.

Definition 1.68 (Windowed topological clause). The topological clause is

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

If representative compatibility is verified, it may equivalently be recorded as

$$\text{PH}_1(C_{\tau,W}F) = 0.$$

Definition 1.69 (Windowed categorical clause). The categorical clause may be invoked only when an admissible, realization-compatible representative exists and the eligible regime, including

$$R(C_{\tau,W}F) \in D^{[-1,0]}(k\text{-mod}),$$

is verified. Only then is

$$\text{Ext}^1(R(C_{\tau,W}F), k) = 0$$

a well-typed Core clause.

Definition 1.70 (Threshold-gap-certified $B\text{-Gate}^+$ input). A $B\text{-Gate}^+$ input is threshold-gap-certified only when the proof object records:

- (i) the post-threshold repaired profile;
- (ii) every pre-threshold profile participating in a metric comparison;
- (iii) every two-sided threshold-gap record;
- (iv) every actual discrepancy upper-bound artifact;
- (v) every crop or window-edge component;
- (vi) every required non-scalar obligation.

A purely exact clause is instead recorded as exact and does not fabricate a positive robustness budget.

Remark 1.71 (PH before Ext). The repaired persistence profile is primary. The Ext clause is read only through an eligible representative and the one-way bridge. No reverse implication is supplied here.

1.16 Tower comparison and typed Type IV input

Definition 1.72 (Windowed collapsed tower objects). For a tower

$$F_0 \rightarrow F_1 \rightarrow F_2 \rightarrow \cdots,$$

write

$$M_{n;i,W,\tau} := \text{Eval}_{i,W,\tau}(F_n).$$

The sequence of barcode profiles is not by itself a directed system: actual profile morphisms and composition laws must be declared, unless a registered τ -split or exact-admissible theorem supplies them.

Definition 1.73 (Tower comparison artifact). A tower comparison artifact contains:

- (i) an actual repaired profile system and its transition morphisms;
- (ii) a source profile $\text{ColimProfile}_{i,W,\tau}(F_\bullet)$;
- (iii) an apex profile $\text{ApexProfile}_{i,W,\tau}(F_\infty)$;
- (iv) an actual morphism

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty);$$

- (v) kernel/cokernel typing, terminal-tameness evidence, and immutable artifact references.

Barcode multisets or bottleneck matchings without actual morphisms are not a tower comparison artifact.

Remark 1.74 (No automatic tower comparison). The comparison morphism is not supplied automatically by hard threshold deletion, a barcode matching, or the mere coexistence of a source tower and an apex.

Definition 1.75 (Typed Type IV scope status). Every run records exactly one Type IV status:

$$\text{not_invoked}, \quad \text{undefined}, \quad \text{certified_zero}, \quad \text{obstructed}.$$

They mean respectively:

- (i) tower semantics are outside the claim;
- (ii) tower semantics are in scope but required data are absent or ill-typed;
- (iii) a valid terminal-generic diagnostic is defined and zero;
- (iv) a valid terminal-generic diagnostic is defined and nonzero.

Definition 1.76 (Collapse diagnostics notation contract). For a valid, terminal-tame comparison artifact, define

$$\mu_{i,W,\tau} := \text{tgdim}_W(\ker \phi_{i,W,\tau}), \quad \nu_{i,W,\tau} := \text{tgdim}_W(\text{coker } \phi_{i,W,\tau}).$$

On a right-unbounded window, tgdim_W is the eventual generic dimension and counts infinite bars. On a bounded window, it counts defect bars cofinal to the right endpoint and is not replaced by dimension after zero extension to $+\infty$.

For a declared finite monitored scope

$$\mathcal{M} = \{(i, W, \tau)\},$$

define

$$\mu_{\text{Collapse}} := \sum_{(i,W,\tau) \in \mathcal{M}} \mu_{i,W,\tau}, \quad \nu_{\text{Collapse}} := \sum_{(i,W,\tau) \in \mathcal{M}} \nu_{i,W,\tau}.$$

For an infinite monitored scope, aggregate diagnostics require a separately proved summability or aggregation theorem.

The legacy notation μ_{Collapse} is deprecated.

Remark 1.77 (Type IV remains Core, but zero is typed). The equality

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

may be used only under status `certified_zero`. It means absence of monitored terminal-generic defect relative to the declared artifact. It does not imply full isomorphism on finite interior bars.

1.17 Manifest consequences

Definition 1.78 (Collapse policy manifest entry). A collapse-policy entry contains:

- (i) immutable run, object, profile, and artifact identifiers;
- (ii) τ , monitored degrees, window, and window type;
- (iii) evaluation order;
- (iv) collapse status

`barcode_level,      tau_split,      exact_admissible;`
- (v) comparison mode;
- (vi) repaired evaluation records;
- (vii) every required threshold-gap record;
- (viii) the metric ledger, scalarization, and actual discrepancy upper-bound evidence;
- (ix) crop and window-edge records;
- (x) any τ -window-adapted commutation certificate;
- (xi) representative and eligibility records;
- (xii) Type IV status, monitored scope, and comparison artifact when invoked;
- (xiii) all non-scalar obligations;
- (xiv) repair ancestry and Claim Register references.

Definition 1.79 (Metric and crop repair record). An Appendix A repair record contains:

- (i) the rejected earlier claim;
- (ii) the replacement theorem;
- (iii) the counterexample;
- (iv) the protected pre-threshold profiles;
- (v) the two-sided threshold-gap record;
- (vi) the metric ledger, scalarization, and discrepancy upper-bound evidence;
- (vii) every crop component;
- (viii) any exact crop–collapse criterion used;
- (ix) confirmation that retained infinite bars were included;
- (x) Type IV scope and artifact references when tower semantics are invoked.

Remark 1.80 (Audit consequence). A proof object that merely says “apply T_τ ” is incomplete. It must identify the collapse regime, comparison mode, metric and non-scalar obligations, and supporting artifacts.

Manifest completeness is necessary for certification but is not sufficient for mathematical truth. A complete manifest may faithfully record a failed comparison or an obstructed tower. Certification additionally requires every invoked theorem, computation, and non-scalar obligation to be valid.

1.18 Operating summary

For each monitored (F, i, W, τ) , the Foundation pipeline is

$$F \mapsto \mathbf{P}_i(F) \mapsto B_{i,W}(F) \mapsto T_\tau^{\text{bar}}(B_{i,W}(F)) = \text{Eval}_{i,W,\tau}(F).$$

Threshold clearance is measured after windowing and before deletion. For a metric comparison, the certified chain is

$$\mathfrak{d}_{\text{met}} \xrightarrow{\text{val}_B} \varepsilon_B, \quad d_B(B, C) \leq \varepsilon_B < \text{gap}_\tau(B, C).$$

The middle inequality requires its own proof or artifact.

The default order is window first, threshold second. Filtered arguments require declared representatives. Tower diagnostics require actual morphisms, terminal tameness, a declared monitored scope, and a typed status. Exactness requires a declared exact-admissible regime.

1.19 Foundation package

Theorem 1.81 (Appendix A v19 final metric-repair foundation package). *Within the constructible one-parameter regime and the standard infinite-bar convention:*

- (i) *constructible modules admit interval decompositions;*
- (ii) *hard deletion removes exactly finite bars of length $\leq \tau$, preserves infinite bars, is idempotent, threshold-compositional, and threshold-monotone;*
- (iii) *positive-threshold short-bar objects are not extension-closed in general;*
- (iv) *hard deletion is not globally non-expansive;*
- (v) *the sharp two-sided threshold-gap theorem holds;*
- (vi) $\text{gap}_\tau = \text{clear}_\tau$;
- (vii) *scalarized budgets are usable only with actual discrepancy upper-bound evidence;*
- (viii) *clearance is measured after windowing and before deletion;*
- (ix) *crop and hard deletion do not commute in general;*
- (x) *they commute exactly under the all-retained-bars criterion;*
- (xi) *functoriality and exactness are conditional on declared split or exact-admissible regimes;*
- (xii) *representatives are window-relative declared representatives, not a global collapse functor;*
- (xiii) *Type IV requires actual morphisms, terminal tameness, finite or otherwise justified aggregation scope, and typed status;*
- (xiv) *manifest completeness does not replace mathematical validity.*

Proof. Items (i)–(ii) follow from Theorem 1.9, Lemma 1.13, and Propositions 1.20–1.21. Item (iii) is Proposition 1.15. Item (iv) is Example 1.22. Item (v) is Theorem 1.33 and Example 1.34. Items (vi)–(vii) are Definitions 1.26 and 1.27. Item (viii) follows from Definitions 1.29, 1.41, and 1.42. Items (ix)–(x) are Proposition 1.40, Example 1.44, and Proposition 1.46. Item (xi) is Proposition 1.59 and Definition 1.60. Item (xii) is Definition 1.63. Item (xiii) is Definitions 1.73, 1.75, and 1.76. Item (xiv) is Remark 1.80. $\square$

1.20 Explicit non-claims

The following are not asserted.

- (i) global non-expansiveness of hard positive-threshold deletion;
- (ii) a positive-threshold Serre subcategory in general;
- (iii) a global exact endofunctor, abelian reflector, or Serre localization;
- (iv) unconditional crop–collapse commutation;
- (v) a finite-only commutation criterion;
- (vi) source-to-window metric transport without a crop theorem or budget;
- (vii) direct comparison of an unscalarized quantale value with real clearance;
- (viii) scalarization without evidence that it bounds the consumed bottleneck discrepancy;
- (ix) numerical repair of a missing non-scalar obligation;
- (x) one-sided threshold-gap certification;
- (xi) gap safety for a finite bar of length exactly τ ;
- (xii) a sharp 2ε -band for the two-sided theorem;
- (xiii) $\text{gap}_\tau = \frac{1}{2} \text{clear}_\tau$;
- (xiv) a global strict filtered lift;
- (xv) an automatic tower comparison map;
- (xvi) a kernel or cokernel defined merely by a barcode matching;
- (xvii) identifying undefined Type IV input with zero;
- (xviii) fabricating numerical zero for a no-tower claim;
- (xix) full comparison isomorphism from terminal-generic zero alone;
- (xx) an aggregate Type IV value over an undeclared infinite scope;
- (xxi) identifying a bounded-window cofinal bar with a global infinite bar;
- (xxii) a well-typed Ext clause without an eligible representative;
- (xxiii) identifying manifest completeness with proof;
- (xxiv) replacing a Core obligation by a Search artifact or AI ranking.

1.21 Provenance and status

Interval decomposition, barcode uniqueness, and the bottleneck/interleaving isometry are standard one-parameter persistence results. The v19-specific content is the metric repair, crop repair, typed ledger discipline, representative boundary, and Type IV input contract.

Status labels for Appendix A. *Standard persistence theorem:* Theorem 1.9.

Metric-repair theorem: Theorem 1.33, Corollary 1.36.

Foundation closure theorem: Theorem 1.81.

Core correction results: Proposition 1.15, Corollary 1.16, Example 1.22, Corollary 1.24, Proposition 1.40, Example 1.44, Proposition 1.46.

Core definitions: Definitions 1.3, 1.5, 1.7, 1.8, 1.10, 1.12, 1.18, 1.25, 1.26, 1.27, 1.29, 1.37, 1.38, 1.45, 1.48, 1.62, 1.63, 1.65, 1.67, 1.68, 1.69, 1.72, 1.73, 1.75, 1.76.

Operational policy and records: Definitions 1.31, 1.41, 1.42, 1.50, 1.51, 1.53, 1.70, 1.78, 1.79; Remarks 1.49, 1.47, 1.74, 1.77, 1.80.

Exact-admissible optional regime: Definitions 1.56, 1.57, 1.58, 1.60; Proposition 1.59.

Explicit exclusions: Subsection 1.20.

2 Appendix B: Filtered Representatives of Windowed Barcode Collapse

2.1 Scope and dependence on the repaired Core

Appendix B supplies the filtered-level foundation used by Chapters 2–4 after the v19 metric repair. It is subordinate to Chapter 1, Chapter 2, and Appendix A.

The canonical project acronym is AK-HPDST. The legacy transposed acronym AK-HDPST may occur in historical artifacts, but it has no normative use in this appendix.

The Core does not treat positive-threshold deletion as a global exact Serre reflector or as a globally non-expansive endofunctor. Its primary object is the windowed barcode-level evaluation

$$\text{Eval}_{i,W,\tau}(F) := T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

The purpose of this appendix is to specify when and how a family of such repaired persistence profiles may be represented by a filtered chain complex for filtered, derived, Ext, or tower-level arguments.

The central rule is:

A filtered representative does not define collapse. It realizes a declared windowed repaired persistence profile, at a declared strength, for a declared monitored scope.

Accordingly, Appendix B separates the following notions:

- (i) the primary barcode-profile evaluation;
- (ii) object-level realization of a finite barcode family;
- (iii) monitored profile equivalence;
- (iv) levelwise filtered quasi-isomorphism;
- (v) realization-compatible equivalence;

- (vi) actual representative morphism data;
- (vii) optional global or exact-admissible representative constructions;
- (viii) tower comparison artifacts, which remain additional data.

Remark 2.1 (Appendix B does not enlarge the Core). This appendix does not restore a global exact collapse functor, a Serre localization, global non-expansiveness, automatic tower comparison, or a reverse Ext^1 -to-persistence bridge.

Its object-level realization theorem concerns finite barcode profile families only. It does not provide functoriality on arbitrary morphisms, preserve additional geometric structure, or establish eligibility for a derived or Ext -level bridge.

Remark 2.2 (Dependency direction). Appendix B consumes the barcode, window, threshold, and metric conventions of Appendix A. Appendix C consumes the realization-compatible representative contract of Appendix B. Appendix D may consume representative tower data, but an actual tower comparison morphism is never generated by Appendix B alone.

2.2 Filtered chain complexes and monitored persistence

Definition 2.3 (Finite-type filtered chain complex). A *finite-type filtered chain complex* over a field k is a bounded chain complex $C_\bullet$ of finite-dimensional k -vector spaces, together with an increasing filtration by subcomplexes

$$F^t C_\bullet \subseteq F^{t'} C_\bullet \quad (t \leq t'),$$

such that:

- (i) the filtration is exhaustive and left-bounded;
- (ii) only finitely many filtration-critical values occur;
- (iii) for every homological degree i , the persistence module

$$\mathbf{P}_i(F) : t \longmapsto H_i(F^t C_\bullet)$$

belongs to $\text{Pers}_k^{\text{cons}}$.

The category of such objects and filtration-preserving chain maps is denoted by

$$\text{FiltCh}(k).$$

Definition 2.4 (Persistence functor). For every homological degree i , define

$$\mathbf{P}_i : \text{FiltCh}(k) \longrightarrow \text{Pers}_k^{\text{cons}}$$

by

$$\mathbf{P}_i(F)(t) := H_i(F^t C_\bullet).$$

Definition 2.5 (Monitored scope). A *monitored scope* is a finite set of homological degrees

$$I_{\text{mon}} \subset \mathbb{Z}.$$

All representative assertions in this appendix are relative to a declared monitored scope unless a full-degree statement is explicitly proved.

Remark 2.6 (Monitored-degree convention). Agreement in I_{mon} does not imply agreement in unmonitored degrees. A proof object must not silently upgrade a monitored representative to a full filtered-chain equivalence.

2.3 Three levels of representative equivalence

Definition 2.7 (Monitored persistence equivalence). Two filtered objects $F, G \in \text{FiltCh}(k)$ are *monitored-persistence equivalent*, written

$$F \simeq_{\text{PH}, I_{\text{mon}}} G,$$

if for every $i \in I_{\text{mon}}$ there is an isomorphism

$$\mathbf{P}_i(F) \cong \mathbf{P}_i(G)$$

in $\text{Pers}_k^{\text{cons}}$.

Definition 2.8 (Levelwise filtered quasi-isomorphism). A filtration-preserving chain map

$$f : F \longrightarrow G$$

is a *levelwise filtered quasi-isomorphism* if, for every parameter t , the map of chain complexes

$$F^t C_{\bullet} \longrightarrow G^t D_{\bullet}$$

is a quasi-isomorphism.

Such a map induces

$$\mathbf{P}_i(F) \cong \mathbf{P}_i(G)$$

for every degree i , and hence monitored-persistence equivalence for every monitored scope.

Definition 2.9 (Realization-compatible equivalence). Fix a realization functor or realization procedure

$$\mathcal{R} : \text{FiltCh}(k) \dashrightarrow D^b(k\text{-mod})$$

on a declared domain.

Two representatives F, G are $\mathcal{R}$ -*equivalent* if:

- (i) both $\mathcal{R}(F)$ and $\mathcal{R}(G)$ are defined;
- (ii) an actual isomorphism

$$\mathcal{R}(F) \cong \mathcal{R}(G)$$

is supplied in $D^b(k\text{-mod})$;

- (iii) the isomorphism is compatible with every structure consumed by the relevant Ext or bridge argument.

Proposition 2.10 (Implications between equivalence levels). *A levelwise filtered quasi-isomorphism implies monitored-persistence equivalence. Monitored-persistence equivalence alone does not imply a levelwise filtered quasi-isomorphism, chain-homotopy equivalence, or $\mathcal{R}$ -equivalence.*

Proof. The first statement follows by taking homology at every filtration level and using naturality of the structure maps.

For the second statement, monitored persistence records only the declared homology persistence modules. It forgets chain-level extension data, unmonitored degrees, and any extra structure consumed by $\mathcal{R}$. Hence no stronger equivalence follows without an additional theorem or artifact. $\square$

Remark 2.11 (Why the equivalence hierarchy matters). Levelwise filtered quasi-isomorphism is sufficient for every monitored persistence clause, but it is stronger than mere profile agreement. The weaker profile relation is therefore used only where the proof consumes no additional chain-level or derived structure.

Remark 2.12 (No automatic localization). The v19 Core does not require a global localization of $\text{FiltCh}(k)$ at monitored-persistence equivalences. If a later construction uses a localized category, it must declare the class of weak equivalences, prove that the localization is suitable for the intended argument, and record the actual morphisms used there.

Definition 2.13 (Localized representative category). When a declared workflow has fixed a class $\mathcal{W}$ of filtered quasi-isomorphisms for which the required localization exists, write

$$\text{Ho}_{\mathcal{W}}(\text{FiltCh}(k))$$

for that declared localization.

This notation is optional and has no default global meaning in Appendix B.

2.4 Windowed repaired persistence profiles

Recall the fixed evaluation order from Chapter 1 and Appendix A:

$$F \mapsto \mathbf{P}_i(F) \mapsto \mathbf{W}_W \mathbf{P}_i(F) \mapsto T_{\tau}^{\text{bar}}(\mathbf{W}_W \mathbf{P}_i(F)).$$

Definition 2.14 (Windowed repaired persistence profile). For $F \in \text{FiltCh}(k)$, $i \in I_{\text{mon}}$, a right-open window W , and $\tau > 0$, define

$$\mathcal{P}_{i,W,\tau}(F) := \text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(\mathbf{W}_W \mathbf{P}_i(F)).$$

Definition 2.15 (Profile family over the monitored scope). For a finite monitored scope I_{mon} , define

$$\mathcal{P}_{W,\tau}^{I_{\text{mon}}}(F) := \{\mathcal{P}_{i,W,\tau}(F)\}_{i \in I_{\text{mon}}}.$$

Remark 2.16 (The profile is primary). The family $\mathcal{P}_{W,\tau}^{I_{\text{mon}}}(F)$ is the primary Core object. A filtered representative is secondary evidence used only when a chain-level, derived, Ext, or tower-level construction requires it.

Definition 2.17 (Comparison mode for profile claims). Every comparison between representative profiles is declared as exactly one of the following:

$$\text{exact}, \quad \text{metric_gap_certified}, \quad \text{not_compared}.$$

An exact comparison is supported by an actual isomorphism or equality. A metric comparison is supported by a two-sided threshold-gap record and an artifact-backed bottleneck bound on the relevant windowed pre-threshold profiles.

2.5 Explicit realization of finite barcode profile families

The following construction turns the object-level existence of a bare filtered representative into a theorem in the finite barcode regime. It does not assert functoriality or preservation of additional structures.

Definition 2.18 (Elementary interval complex). Fix a degree i .

For a finite right-open interval $I = [b, d)$ with $b < d$, define the filtered chain complex $E_i(I)$ by generators

$$x_I \in C_i, \quad y_I \in C_{i+1},$$

with

$$\partial y_I = x_I, \quad \partial x_I = 0,$$

and filtration values

$$\text{fil}(x_I) = b, \quad \text{fil}(y_I) = d.$$

For an infinite interval $I = [b, \infty)$, define $E_i(I)$ to have one generator $x_I \in C_i$, with $\partial x_I = 0$ and $\text{fil}(x_I) = b$.

Lemma 2.19 (Persistence of an elementary interval complex). *For every interval I in Definition 2.18,*

$$\mathbf{P}_i(E_i(I)) \cong k_I,$$

and

$$\mathbf{P}_j(E_i(I)) = 0 \quad (j \neq i).$$

Proof. For $I = [b, d)$, no generator is present before b . For $b \leq t < d$, only x_I is present, so it represents one nonzero class in degree i . At $t \geq d$, the generator y_I appears and $\partial y_I = x_I$, so the pair is acyclic. This gives the interval module $k_{[b, d)}$ in degree i and no other homology.

For $I = [b, \infty)$, the class x_I appears at b and is never killed, producing $k_{[b, \infty)}$. $\square$

Definition 2.20 (Bare barcode realization). Let I_{mon} be finite, and let

$$\mathcal{B} = \{B_i\}_{i \in I_{\text{mon}}}$$

be a family of finite barcodes written in right-open interval normal form. Define

$$\text{Rep}_{\text{bar}}(\mathcal{B}) := \bigoplus_{i \in I_{\text{mon}}} \bigoplus_{I \in B_i} E_i(I).$$

Theorem 2.21 (Finite barcode family realization). *Let I_{mon} be finite and let each B_i be a finite barcode. Then $\text{Rep}_{\text{bar}}(\{B_i\})$ is a finite-type filtered chain complex and*

$$\mathbf{P}_i(\text{Rep}_{\text{bar}}(\{B_j\})) \cong B_i$$

for every $i \in I_{\text{mon}}$.

Consequently, every finite windowed repaired profile family

$$\mathcal{P}_{W, \tau}^{I_{\text{mon}}}(F)$$

admits at least one bare filtered representative.

Proof. The direct sum is finite because both the monitored scope and every barcode are finite. It is bounded and finite-dimensional. The persistence homology of a finite direct sum is the direct sum of the persistence homologies of its summands. The result follows from Lemma 2.19. $\square$

Remark 2.22 (Strength and limitation of the realization theorem). Theorem 2.21 proves object-level existence of a filtered complex realizing the declared finite profiles. It does not supply:

- a canonical choice independent of barcode decomposition data;
- a functor on arbitrary persistence morphisms;
- compatibility with the original filtered object F ;
- geometric, arithmetic, sheaf-theoretic, or categorical structure;
- $\mathcal{R}$ -equivalence between different representatives;
- the amplitude or edge-realization hypotheses needed for Appendix C.

Those are independent obligations.

2.6 Admissible filtered representatives and invocation status

Definition 2.23 (Admissible filtered representative). Let $F \in \text{FiltCh}(k)$, let W be a right-open window, let $\tau > 0$, and let I_{mon} be finite.

A filtered object

$$C_{\tau, W}F \in \text{FiltCh}(k)$$

is an *admissible filtered representative* if it is equipped with actual profile isomorphisms

$$\rho_i : \mathbf{P}_i(C_{\tau, W}F) \xrightarrow{\cong} \mathcal{P}_{i, W, \tau}(F)$$

for every $i \in I_{\text{mon}}$.

The tuple

$$(C_{\tau, W}F, I_{\text{mon}}, \{\rho_i\}_{i \in I_{\text{mon}}})$$

is part of the representative artifact.

Definition 2.24 (Representative compatibility). An admissible representative is *profile-compatible* if its declared isomorphisms ρ_i identify its monitored persistence exactly with the windowed repaired profile family.

Proposition 2.25 (Compatibility is exactly the representative condition). *A filtered object is an admissible representative in the sense of Definition 2.23 if and only if it is supplied with profile-compatibility isomorphisms for every monitored degree.*

Proof. This is the defining condition, with the requirement that the isomorphisms and monitored scope are explicit artifact data. $\square$

Definition 2.26 (Representative lifecycle status). For each tuple $(F, W, \tau, I_{\text{mon}})$, the representative lifecycle status is one of

`not_required`, `constructed`, `verified`, `failed`, `missing`.

Their meanings are:

- (i) `not_required`: no filtered, derived, Ext, or tower clause uses a representative;
- (ii) `constructed`: a candidate exists, but profile compatibility has not yet been verified;
- (iii) `verified`: the candidate and all required profile isomorphisms are verified;
- (iv) `failed`: a supplied candidate fails a required compatibility check;
- (v) `missing`: a representative is required by the claim but no candidate has been supplied.

Remark 2.27 (Lifecycle status versus canonical audit status). The lifecycle vocabulary is not a replacement for the canonical atomic audit statuses of Chapter 7.

When a representative is required:

`verified` $\mapsto$ `pass`,

`failed` $\mapsto$ `reject`,

`constructed` or `missing` $\mapsto$ `undefined`.

The status `not_required` maps to `not_invoked` only under an explicit scope policy.

Remark 2.28 (Representative, not functor). The notation $C_{\tau, W}F$ denotes a chosen representative artifact. It does not assert that $C_{\tau, W}$ is globally defined, canonical, or functorial.

2.7 Implementable range, non-uniqueness, and invariance

Definition 2.29 (Implementable representative range). The *implementable representative range* is the declared class of tuples

$$(F, W, \tau, I_{\text{mon}})$$

for which the proof object supplies:

- (i) the finite repaired profile family $\mathcal{P}_{W, \tau}^{I_{\text{mon}}}(F)$;
- (ii) a constructed representative;
- (iii) verified profile-compatibility isomorphisms;
- (iv) immutable artifact references;
- (v) every stronger compatibility required by the invoked downstream clause.

Remark 2.30 (Why an implementable range is needed). The bare realization theorem proves that finite barcode profiles can be realized by filtered complexes. It does not prove that a representative preserves the extra structure required by a particular derived, geometric, or tower-level argument. The implementable range records exactly where those stronger obligations have also been discharged.

Axiom 2.1 (Structure-compatible representative availability). *A run may claim membership in the implementable representative range only after an actual representative artifact has been constructed and verified.*

The finite barcode realization theorem discharges bare object-level existence. Any stronger requirement—for example realization compatibility, geometric compatibility, functoriality, or tower coherence—must be discharged separately.

Remark 2.31 (Status of the availability axiom). Axiom 2.1 is an audit admissibility rule, not an assertion that every repaired profile has a unique canonical or structure-compatible representative.

Proposition 2.32 (Non-uniqueness of filtered representatives). *An admissible filtered representative is generally not unique as a strict filtered chain complex.*

Proof. A monitored persistence profile does not determine chain-level contractible summands, unmonitored degrees, bases, or additional structure. Different filtered complexes can therefore realize the same monitored profile family. $\square$

Definition 2.33 (Admissible representative choice). An admissible representative choice consists of:

- (i) the representative $C_{\tau, W}F$;
- (ii) the finite monitored scope;
- (iii) the profile isomorphisms ρ_i ;
- (iv) the comparison mode;
- (v) the declared equivalence strength;
- (vi) any realization-compatibility data;
- (vii) artifact identifiers and hashes.

Proposition 2.34 (Representative invariance of persistence-level clauses). *If $C_{\tau,W}F$ and $C'_{\tau,W}F$ are admissible representatives of the same profile family, then every Core clause depending only on the monitored persistence modules has the same value for both representatives.*

Proof. For each monitored degree,

$$\mathbf{P}_i(C_{\tau,W}F) \cong \mathcal{P}_{i,W,\tau}(F) \cong \mathbf{P}_i(C'_{\tau,W}F).$$

Hence every predicate invariant under persistence-module isomorphism has the same value. $\square$

Remark 2.35 (No derived invariance from profile invariance). Proposition 2.34 applies only to persistence-level clauses. It does not imply invariance of $\mathcal{R}$, Ext, products, higher operations, or tower comparison maps. Those require stronger compatibility artifacts.

2.8 Global representative notation and its restricted use

Definition 2.36 (Global representative construction). A *global representative construction* at threshold τ is a declared assignment

$$F \longmapsto C_{\tau}F$$

with actual profile isomorphisms

$$\mathbf{P}_i(C_{\tau}F) \cong T_{\tau}^{\text{bar}}(\mathbf{P}_i(F))$$

for every declared monitored degree.

Remark 2.37 (Global representative construction is not default). Definition 2.36 is optional. It does not assert functoriality on morphisms or compatibility with window restriction.

Definition 2.38 (Window compatibility of a global representative). A global representative construction is *window-compatible on W* if for every monitored degree there is an actual isomorphism

$$W_W \mathbf{P}_i(C_{\tau}F) \cong T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Proposition 2.39 (Restriction under window compatibility). *If a global representative construction is window-compatible on W , then the restricted object $(C_{\tau}F)|_W$ is an admissible representative of the windowed repaired profile.*

Proof. By the definition of restriction to the declared window,

$$\mathbf{P}_i((C_{\tau}F)|_W) \cong W_W \mathbf{P}_i(C_{\tau}F).$$

The defining window-compatibility isomorphism therefore gives

$$\mathbf{P}_i((C_{\tau}F)|_W) \cong T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)),$$

which is precisely the monitored profile isomorphism required by Definition 2.23. $\square$

Remark 2.40 (Why restriction is not the default). Appendix A, Proposition 1.40, proves that

$$W_W T_{\tau}^{\text{bar}} \not\cong T_{\tau}^{\text{bar}} W_W$$

in general. The infinite-bar obstruction is exhibited in Example 1.44, and the exact window-adapted criterion is Definition 1.45 together with Proposition 1.46. Therefore

$$C_{\tau,W}F = (C_{\tau}F)|_W$$

may be used only under a verified window-compatibility or τ -window-adapted criterion.

2.9 Profile algebra at the representative level

Definition 2.41 (Window-normalized representative). An admissible representative $C_{\tau,W}F$ is *window-normalized* if its monitored persistence profiles are supported on W with the same extension-by-zero convention used in Appendix A.

Proposition 2.42 (Windowed representative idempotence). *Let $C_{\tau,W}F$ be a verified window-normalized representative. Suppose a second representative is constructed using the same window, monitored scope, and profile-equivalence policy. Then*

$$C_{\tau,W}(C_{\tau,W}F) \simeq_{\text{PH}, I_{\text{mon}}} C_{\tau,W}F.$$

Proof. Window normalization ensures that reapplying W_W does not clip the monitored profile further. Every finite bar in the first repaired profile has length strictly greater than τ , and every infinite bar is retained. Barcode-level idempotence therefore gives identical monitored repaired profiles. $\square$

Remark 2.43 (Idempotence is profile-level). The proposition does not assert strict equality of filtered complexes or the existence of a globally idempotent endofunctor.

Proposition 2.44 (Windowed threshold composition). *Let $\tau, \sigma > 0$. Suppose all relevant representatives exist, are window-normalized, and use the same monitored scope and window. Then*

$$C_{\sigma,W}(C_{\tau,W}F) \simeq_{\text{PH}, I_{\text{mon}}} C_{\max\{\tau,\sigma\},W}F.$$

Proof. On every monitored profile,

$$T_{\sigma}^{\text{bar}} T_{\tau}^{\text{bar}} = T_{\max\{\tau,\sigma\}}^{\text{bar}}.$$

Window normalization prevents a second crop from creating new short bars. Thus both representatives realize the same monitored profile family. $\square$

Remark 2.45 (Threshold sweeps). Threshold composition is a statement about repaired profiles and verified representatives. It does not create a strict nested family of filtered complexes or actual transition morphisms between threshold levels.

2.10 Representative morphisms and conditional functoriality

Definition 2.46 (Repaired profile morphism artifact). Let $f : F \rightarrow G$ be filtration-preserving. A *repaired profile morphism artifact* for (i, W, τ) is an actual persistence-module morphism

$$\bar{f}_{i,W,\tau} : \mathcal{P}_{i,W,\tau}(F) \longrightarrow \mathcal{P}_{i,W,\tau}(G)$$

with declared provenance and compatibility with the intended source map.

A bottleneck matching or a small bottleneck distance is not such a morphism.

Definition 2.47 (Representative morphism datum). Let $C_{\tau,W}F$ and $C_{\tau,W}G$ be admissible representatives. A *representative morphism datum* consists of an actual filtration-preserving chain map, or a declared morphism in a justified localized category,

$$C_{\tau,W}f : C_{\tau,W}F \longrightarrow C_{\tau,W}G,$$

such that the induced monitored persistence morphisms commute with the profile isomorphisms and the repaired profile morphisms $\bar{f}_{i,W,\tau}$.

Definition 2.48 (Weak representative functoriality). A representative construction is *weakly functorial* on a declared subcategory if:

- (i) every object has a verified representative;
- (ii) every morphism has a repaired profile morphism artifact and a representative morphism datum;
- (iii) identity morphisms are represented by identity data up to the declared equivalence;
- (iv) composition is compatible up to a specified coherent representative equivalence.

Proposition 2.49 (Weak functoriality is conditional). *Weak representative functoriality is available only relative to the declared subcategory, actual morphism artifacts, and composition-coherence data. It is not automatic in the general Core.*

Proof. Barcode-level hard deletion is not a globally defined exact functor on all persistence morphisms, and arbitrary representative choices do not determine chain maps. The listed artifacts are therefore additional data, not consequences of object-level profile realization. $\square$

Remark 2.50 (No strict global functor). Appendix B does not assert a strict global endofunctor

$$C_\tau : \text{FiltCh}(k) \longrightarrow \text{FiltCh}(k)$$

or a strict bifunctorial construction $(F, W) \mapsto C_{\tau, W}F$.

2.11 Compatibility with realization and Ext-level use

Definition 2.51 (Realization-compatible representative). An admissible representative $C_{\tau, W}F$ is *realization-compatible* for a declared realization $\mathcal{R}$ if:

- (i) $\mathcal{R}(C_{\tau, W}F)$ is defined in $D^b(k\text{-mod})$;
- (ii) the representative artifact records the exact realization used;
- (iii) every alternative representative used interchangeably is supplied with an actual $\mathcal{R}$ -equivalence;
- (iv) the amplitude, edge-realization, and eligibility predicates required by Appendix C can be evaluated;
- (v) all additional structure consumed by the Ext argument is preserved.

Proposition 2.52 (Realization invariance under stronger equivalence). *If two admissible representatives are $\mathcal{R}$ -equivalent, then their realized objects are isomorphic in $D^b(k\text{-mod})$, and every isomorphism-invariant derived predicate has the same value on both.*

Proof. This is immediate from the supplied isomorphism in the derived category. $\square$

Remark 2.53 (Profile equivalence is insufficient for Ext). Monitored-persistence equivalence alone does not justify replacing one representative by another inside Ext. The one-way persistence-to-Ext bridge consumes a realization-compatible representative and the eligible regime of Appendix C.

2.12 Relation to $B\text{-Gate}^+$

Definition 2.54 (Representative input to $B\text{-Gate}^+$). A representative input consists of:

- (i) F , W , τ , and the finite monitored scope;
- (ii) the pre-threshold profiles $W_W \mathbf{P}_i(F)$;
- (iii) the repaired profiles $\mathcal{P}_{i,W,\tau}(F)$;
- (iv) the comparison mode and every required threshold-gap record;
- (v) a verified admissible representative if a filtered, derived, Ext, or tower clause is invoked;
- (vi) realization-compatible data if the Ext clause is invoked;
- (vii) the representative lifecycle status and canonical atomic status;
- (viii) immutable artifact references.

Definition 2.55 (Topological clause via representative). The primary topological clause is

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

If a verified admissible representative is supplied, the same clause may be read as

$$\text{PH}_1(C_{\tau,W}F) = 0$$

through the declared profile isomorphism.

Definition 2.56 (Categorical clause via representative). The categorical clause

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0$$

may be invoked only when the representative is verified, realization-compatible, and eligible under Appendix C.

Remark 2.57 (No representative, no Ext clause). Barcode-level evaluation can proceed without a representative. A missing or unverified representative makes a required Ext clause undefined; it is not a pass and is not repaired by a numerical budget.

2.13 Representative towers and Type IV input

Definition 2.58 (Representative tower). Let

$$F_0 \longrightarrow F_1 \longrightarrow F_2 \longrightarrow \cdots$$

be a directed tower.

A *representative tower* for (i, W, τ) consists of:

- (i) verified representatives $C_{\tau,W}F_n$;
- (ii) actual representative morphism data between consecutive levels;
- (iii) identity and composition coherence;

(iv) compatibility with the repaired profile system.

Definition 2.59 (Representative tower comparison artifact). A representative tower comparison artifact is declared data specifying:

- (i) a repaired directed profile system;
- (ii) a source profile $\text{ColimProfile}_{i,W,\tau}(F_\bullet)$;
- (iii) an apex profile $\text{ApexProfile}_{i,W,\tau}(F_\infty)$;
- (iv) an actual comparison morphism

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty);$$

(v) kernel, cokernel, terminal-tameness, monitored-scope, and provenance data required by Chapter 5 and Appendix D.

Remark 2.60 (Type IV input). Representative existence, representative tower existence, and comparison morphism existence are three distinct obligations. The first two do not supply the third.

A barcode matching, small bottleneck distance, expected colimit map, or sequence of barcode multisets is not an actual Type IV comparison artifact.

2.14 Certificate, obligation, and manifest requirements

Definition 2.61 (Representative declaration in a certificate record). A certificate record using an admissible representative declares at least:

- (i) the source object, window, threshold, and monitored scope;
- (ii) the fixed evaluation order;
- (iii) the pre-threshold and repaired profiles;
- (iv) the comparison mode;
- (v) every threshold-gap record used for a metric comparison;
- (vi) the representative identifier and lifecycle status;
- (vii) the profile isomorphisms ρ_i ;
- (viii) the declared equivalence strength;
- (ix) the window-normalization status;
- (x) realization compatibility and eligibility, if invoked;
- (xi) actual morphism and coherence artifacts, if invoked;
- (xii) tower comparison artifacts, if Type IV is invoked;
- (xiii) claim, proof, and artifact references;
- (xiv) the canonical atomic status of every required obligation.

Definition 2.62 (Representative obligation bundle). The representative obligation bundle is the typed family consisting of:

- (i) existence;
- (ii) profile compatibility;
- (iii) monitored-scope correctness;
- (iv) equivalence strength;
- (v) window normalization when repeated evaluation is used;
- (vi) realization compatibility when derived data are used;
- (vii) actual morphism existence when functoriality is used;
- (viii) composition coherence when a diagram or tower is used;
- (ix) artifact identity and provenance.

Remark 2.63 (Appendix F and G handoff). Appendix F elaborates the status algebra, dependency closure, failure routing, and non-compensation rules for the obligation bundle of Definition 2.62.

Appendix G records the corresponding fields, identifiers, and artifact references in the canonical manifest schema.

Neither an obligation list nor a complete manifest proves that the representative is mathematically correct; the proof or verified artifact remains necessary.

2.15 Filtered-representative foundation package

Theorem 2.64 (Appendix B filtered-representative package). *Within the finite monitored barcode regime and the declared v19 representative discipline, the following package is available.*

- (i) *The primary object is the repaired profile family $\mathcal{P}_{W,\tau}^{I_{\text{mon}}}(F)$.*
- (ii) *Every finite repaired barcode profile family admits a bare finite-type filtered-chain realization.*
- (iii) *An admissible representative is a filtered object together with explicit monitored profile isomorphisms.*
- (iv) *Representatives are generally non-unique.*
- (v) *Monitored-persistence equivalence preserves persistence-level Core clauses, but does not imply derived or realization-level equivalence.*
- (vi) *A global representative construction is optional and need not be window-compatible.*
- (vii) *Representative idempotence and threshold composition hold at profile level under the stated window-normalization hypotheses.*
- (viii) *A bottleneck matching is not a persistence morphism.*
- (ix) *Weak functoriality requires actual repaired profile morphisms, representative morphisms, and composition coherence.*

- (x) *Ext-level use requires a realization-compatible representative and the eligible regime of Appendix C.*
- (xi) *A representative tower does not automatically supply a Type IV comparison morphism.*
- (xii) *Representative existence and compatibility are non-scalar obligations and must be recorded in the proof object.*

Proof. Item (i) is Definitions 2.14 and 2.15. Item (ii) is Theorem 2.21. Item (iii) is Definition 2.23. Item (iv) is Proposition 2.32. Item (v) is Proposition 2.10 and Proposition 2.34. Item (vi) is Definitions 2.36 and 2.38. Item (vii) is Propositions 2.42 and 2.44. Item (viii) is Definition 2.46. Item (ix) is Proposition 2.49. Item (x) is Definition 2.51. Item (xi) is Definitions 2.58 and 2.59. Item (xii) is Definitions 2.26 and 2.62. $\square$

2.16 Explicit exclusions

The following are not asserted in Appendix B.

- (i) No strict global endofunctor

$$C_\tau : \text{FiltCh}(k) \longrightarrow \text{FiltCh}(k)$$

is asserted by default.

- (ii) No strict global functor

$$(F, W) \longmapsto C_{\tau, W}F$$

is asserted by default.

- (iii) No uniqueness or hidden canonicity of a representative is asserted.
- (iv) No monitored-persistence equivalence is automatically promoted to a filtered quasi-isomorphism, chain-homotopy equivalence, or derived equivalence.
- (v) No bare barcode realization is automatically compatible with the original geometry, arithmetic, sheaf, category, or realization functor.
- (vi) No unconditional identity

$$C_{\tau, W}F = (C_\tau F)|_W$$
 is asserted.
- (vii) No global exactness, Serre localization, or reflector claim is restored.
- (viii) No profile idempotence is upgraded to strict filtered-complex idempotence.
- (ix) No threshold-composition identity produces actual transition morphisms without additional data.
- (x) No bottleneck matching is treated as a persistence morphism.
- (xi) No representative morphism is inferred merely from object-level representative existence.
- (xii) No Ext clause is invoked without realization compatibility and eligibility.
- (xiii) No representative tower automatically supplies a colimit, apex, or comparison morphism.
- (xiv) No missing representative or morphism is repaired by a metric budget.

- (xv) No lifecycle status replaces the canonical audit statuses of Chapter 7.
- (xvi) No complete manifest replaces mathematical verification.
- (xvii) No AWFS, soft-commuting calculus, Mirror/Transfer 2-cell, Search mechanism, or external-domain realization is introduced here.

2.17 Provenance and status

The barcode-level operation T_{τ}^{bar} , the fixed evaluation order, and the threshold-gap metric repair are supplied by Chapter 2 and Appendix A. Crop–collapse noncommutation is Proposition 1.40; the infinite-bar obstruction is Example 1.44; and the exact all-retained-bars commutation criterion is Definition 1.45 together with Proposition 1.46.

The elementary interval-complex construction is the standard filtered-chain realization of a finite persistence barcode. Its use here proves bare object-level existence for finite repaired profile families.

The v19-specific contribution of Appendix B is the typed separation of:

- profile realization;
- filtered quasi-isomorphism;
- realization-compatible equivalence;
- actual morphism data;
- tower comparison artifacts;
- lifecycle and canonical audit statuses.

Status labels for Appendix B. *Foundation realization theorem:* Theorem 2.21.

Foundation closure theorem: Theorem 2.64.

Core definitions: Definitions 2.3, 2.4, 2.5, 2.7, 2.8, 2.9, 2.13, 2.14, 2.15, 2.17, 2.18, 2.20, 2.23, 2.24, 2.26, 2.29, 2.33, 2.36, 2.38, 2.41, 2.46, 2.47, 2.48, 2.51, 2.54, 2.55, 2.56, 2.58, 2.59, 2.61, 2.62.

Foundation propositions and lemmas: Lemma 2.19, Propositions 2.10, 2.25, 2.32, 2.34, 2.39, 2.42, 2.44, 2.49, 2.52.

Operational axiom and cautions: Axiom 2.1; Remarks 2.1, 2.2, 2.6, 2.12, 2.16, 2.22, 2.27, 2.28, 2.31, 2.35, 2.37, 2.40, 2.43, 2.45, 2.50, 2.53, 2.57, 2.60, 2.63.

Explicit exclusions: Subsection 2.16.

3 Appendix C: Eligible Realization and Canonical Ext-edge Details

3.1 Scope and dependence on the repaired Core

Appendix C supplies the detailed derived-category and eligibility foundation for Chapter 4. It consumes the repaired evaluation discipline of Chapters 2–3 and Appendix A, together with the representative discipline of Appendix B.

The canonical project acronym is AK-HPDST. The legacy transposed acronym AK-HDPST may occur in historical artifacts, but it has no normative use in this appendix.

The only Core bridge proved here is the one-way implication

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0,$$

under a complete eligible-realization record.

The proof has three logically separate inputs:

- (i) a repaired degree-one persistence profile and an admissible representative;
- (ii) a vector-space-valued residual edge extractor and an edge-realization certificate;
- (iii) the canonical two-term Ext^1 -edge obtained from the standard truncation triangle over a field.

The bridge does not use global non-expansiveness, a global collapse functor, Serre localization, a reverse faithfulness principle, Search-layer output, or a numerical budget to manufacture missing categorical data.

Remark 3.1 (Appendix C does not enlarge the Core). This appendix expands the proof and audit contract already fixed by Chapter 4. It does not introduce a stronger bridge.

In particular, it proves only

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0$$

inside the eligible regime. No converse and no global equivalence

$$\text{PH}_1 \iff \text{Ext}^1$$

is asserted.

Remark 3.2 (Metric repair is consumed, not reproved). Appendix C does not reprove threshold-gap-safe stability. If the repaired vanishing input is transported under perturbation, the proof object must cite the relevant two-sided threshold-gap certificate and the scalarized metric budget from Chapters 1–3 and Appendix A.

Exact vanishing of a fixed repaired profile may be used directly. Numerical clearance cannot create a representative, a realization, an amplitude certificate, or an edge-realization certificate.

3.2 Derived category setting

Fix a field k . Let

$$D^b(k\text{-mod})$$

denote the bounded derived category of finite-dimensional k -vector spaces, equipped with its standard t -structure.

Definition 3.3 (Bounded derived object). An object

$$A \in D^b(k\text{-mod})$$

is represented by a bounded cochain complex of finite-dimensional k -vector spaces. Its cohomology objects are denoted by $H^j(A)$.

Definition 3.4 (Amplitude). An object $A \in D^b(k\text{-mod})$ has amplitude contained in $[a, b]$ if

$$H^j(A) = 0 \quad \text{for all } j \notin [a, b].$$

We write

$$A \in D^{[a,b]}(k\text{-mod}).$$

Definition 3.5 (Two-term eligible amplitude). An object $A \in D^b(k\text{-mod})$ has eligible two-term amplitude if

$$A \in D^{[-1,0]}(k\text{-mod}).$$

Equivalently,

$$H^j(A) = 0 \quad (j \neq -1, 0).$$

Remark 3.6 (Why amplitude $[-1, 0]$ is used). The two-term amplitude is the declared Core typing contract for the Ext^1 -edge. It yields a canonical truncation triangle with only H^{-1} and H^0 , makes the edge proof transparent, and prevents a broader derived realization from being silently treated as Core-eligible.

No extension beyond this amplitude is claimed in v19.0.0, even if a stronger statement may hold in a separately analysed semisimple regime.

3.3 Repaired degree-one input and admissible representatives

Definition 3.7 (Repaired windowed collapsed profile). For $F \in \text{FiltCh}(k)$, monitored degree i , right-open window $W = [u, u')$, and threshold $\tau > 0$, define

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

This is the post-threshold repaired persistence profile. Threshold clearance, when required, is measured on the pre-threshold profile

$$W_W \mathbf{P}_i(F).$$

Definition 3.8 (Admissible representative). An object

$$C_{\tau,W}F \in \text{FiltCh}(k)$$

is an admissible representative for the monitored scope if

$$\mathbf{P}_i(C_{\tau,W}F) \cong \text{Eval}_{i,W,\tau}(F)$$

for every declared monitored degree i , under the representative policy of Appendix B, Definition 2.23.

Remark 3.9 (Representative, not global collapse). The notation $C_{\tau,W}F$ denotes a declared representative of the windowed repaired profile. It does not denote a globally strict collapse functor and does not imply

$$C_{\tau,W}F = (C_\tau F)|_W.$$

Remark 3.10 (Dependence on Appendix B). Appendix B distinguishes monitored profile equivalence, filtered quasi-isomorphism, and realization-compatible equivalence. Appendix C uses the strongest of these only when required by the Ext clause.

A profile-level representative is not automatically realization-compatible.

Definition 3.11 (Degree-one compatible representative). An admissible representative is degree-one compatible if

$$\mathbf{P}_1(C_{\tau,W}F) \cong \text{Eval}_{1,W,\tau}(F).$$

When this condition is part of the record, the notation

$$\text{PH}_1(C_{\tau,W}F) = 0$$

is used only as shorthand for the vanishing of the realized repaired persistence profile.

3.4 Realization data and the residual edge extractor

Definition 3.12 (Realization functor). A realization functor or realization procedure is a declared functor

$$\mathcal{R} : C_{\text{rep}} \longrightarrow D^b(k\text{-mod}),$$

where C_{rep} is a declared representative subcategory of $\text{FiltCh}(k)$.

Its domain, version, treatment of morphisms, and relation to the declared representative-equivalence policy are part of the realization record.

Definition 3.13 (Realization-compatible representative). A degree-one compatible representative $C_{\tau, W}F$ is *realization-compatible* if:

- (i) it belongs to the domain of $\mathcal{R}$;
- (ii) the realized object $\mathcal{R}(C_{\tau, W}F)$ is well-defined;
- (iii) every alternative representative used interchangeably with it is related by a declared realization equivalence in the sense of Appendix B, Definition 2.9;
- (iv) the amplitude and edge-realization conditions below can be audited.

Remark 3.14 (Why realization compatibility is required). The persistence profile and the realized derived object live in different categories. Profile equivalence alone does not imply that two representatives have isomorphic realizations.

Without realization compatibility, the value of

$$\text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k)$$

may depend on an untracked representative choice.

Definition 3.15 (Eligible realized object). For a realization-compatible representative define

$$A_{F, W, \tau} := \mathcal{R}(C_{\tau, W}F).$$

Definition 3.16 (Residual degree-one persistence edge). A residual degree-one edge extractor is a declared vector-space-valued assignment

$$\text{Edge}_1 : \mathfrak{P}_1^{\text{rep}} \longrightarrow \text{Vect}_k$$

on a declared class $\mathfrak{P}_1^{\text{rep}}$ of repaired degree-one persistence profiles, satisfying at least the zero-compatibility condition

$$\text{Edge}_1(0) = 0.$$

The extractor need not be faithful on all profiles unless a separate theorem asserts that stronger property.

Remark 3.17 (Why an edge extractor is mandatory). The object

$$H^{-1}(A_{F, W, \tau})$$

is a finite-dimensional k -vector space, whereas

$$\text{Eval}_{1, W, \tau}(F)$$

is a persistence profile. They cannot be directly identified without a type-changing construction.

The extractor Edge_1 supplies the required typed interface.

Definition 3.18 (Edge-realization condition). An edge-realization certificate for the declared object, window, threshold, representative, realization, and edge extractor is a declared isomorphism

$$\epsilon_{F,W,\tau} : H^{-1}(A_{F,W,\tau}) \xrightarrow{\sim} \text{Edge}_1(\text{Eval}_{1,W,\tau}(F)).$$

The only extractor property used by the Core bridge is zero compatibility:

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies \text{Edge}_1(\text{Eval}_{1,W,\tau}(F)) = 0.$$

Remark 3.19 (Meaning of the edge-realization condition). The amplitude condition alone does not connect repaired persistence to H^{-1} . The edge-realization certificate is an independent non-scalar obligation specifying exactly which vector-space edge of the repaired profile is realized by H^{-1} .

Remark 3.20 (No raw persistence-to-derived shortcut). The edge extractor is applied to the repaired profile

$$\text{Eval}_{1,W,\tau}(F),$$

not to the raw persistence module $\mathbf{P}_1(F)$. The bridge is therefore windowed and post-threshold.

Definition 3.21 (Eligible regime). An eligible realization record for $(F; W, \tau)$ consists of:

- (i) a repaired degree-one evaluation record;
- (ii) a degree-one compatible admissible representative and its Appendix B representative record;
- (iii) a realization-compatible realization datum $\mathcal{R}$;
- (iv) the realized object

$$A_{F,W,\tau} = \mathcal{R}(C_{\tau,W}F);$$
- (v) an amplitude certificate

$$A_{F,W,\tau} \in D^{[-1,0]}(k\text{-mod});$$
- (vi) a residual edge extractor Edge_1 ;
- (vii) an edge-realization certificate $\epsilon_{F,W,\tau}$;
- (viii) the field coefficient declaration;
- (ix) the representative-equivalence and realization-equivalence policy;
- (x) applicable claim-status and manifest records;
- (xi) immutable artifacts sufficient to reconstruct every item above.

The Ext clause is eligible only when the required record is complete and consistent. If the clause is invoked but an item is missing or ill-typed, its audit status is `undefined`.

Remark 3.22 (Eligibility is a gate condition). Eligibility is stronger than representative existence. It licenses the use of the categorical edge. Missing eligibility is not categorical vanishing, not a passing clause, and not a mathematical counterexample.

Remark 3.23 (Non-scalar obligations). Representative existence, realization compatibility, amplitude, the edge extractor, the edge-realization isomorphism, claim status, and manifest consistency are non-scalar obligations. They cannot be repaired by increasing a metric budget or threshold clearance.

3.5 Formality over a field and the canonical Ext^1 -edge

Lemma 3.24 (Semisimplicity over a field). *The abelian category $k\text{-mod}$ of finite-dimensional k -vector spaces is semisimple. Equivalently, every short exact sequence in $k\text{-mod}$ splits.*

Proof. Every finite-dimensional vector space over a field is projective and injective. Hence every short exact sequence splits. $\square$

Lemma 3.25 (Derived splitting over a field). *For every $A \in D^b(k\text{-mod})$, there exists an isomorphism*

$$A \cong \bigoplus_{j \in \mathbb{Z}} H^j(A)[-j]$$

in the bounded derived category.

In general, this direct-sum isomorphism depends on choices and is not used to prove naturality of the Ext-edge theorem below.

Proof. Choose a bounded complex representing A . Semisimplicity permits splittings of cycles, boundaries, and cohomology in every degree, producing a quasi-isomorphism to the cohomology complex with zero differential. The chosen splittings need not be functorial. $\square$

Corollary 3.26 (Two-term splitting). *If $A \in D^{[-1,0]}(k\text{-mod})$, then there exists an isomorphism*

$$A \cong H^{-1}(A)[1] \oplus H^0(A).$$

This isomorphism is explanatory and is not the source of the canonical naturality in Theorem 3.28.

Proof. Apply Lemma 3.25 and the amplitude restriction. $\square$

Definition 3.27 (Ext^1 in the derived category). For $A \in D^b(k\text{-mod})$, define

$$\text{Ext}^1(A, k) := \text{Hom}_{D^b(k\text{-mod})}(A, k[1]).$$

Theorem 3.28 (Canonical two-term Ext^1 -edge identification). *Let $A \in D^{[-1,0]}(k\text{-mod})$. Then there is a canonical isomorphism, natural in A ,*

$$\text{Ext}^1(A, k) \xrightarrow{\sim} \text{Hom}_k(H^{-1}(A), k).$$

Proof. The standard t -structure gives the canonical truncation triangle

$$H^{-1}(A)[1] \longrightarrow A \longrightarrow H^0(A) \xrightarrow{+1} H^{-1}(A)[2].$$

Apply the contravariant cohomological functor

$$\text{Hom}_{D^b(k\text{-mod})}(-, k[1]).$$

The resulting long exact sequence contains the natural exact segment

$$\begin{aligned} \text{Hom}_D(H^0(A), k[1]) &\longrightarrow \text{Hom}_D(A, k[1]) \\ &\longrightarrow \text{Hom}_D(H^{-1}(A)[1], k[1]) \\ &\longrightarrow \text{Hom}_D(H^0(A), k[2]), \end{aligned}$$

where $D = D^b(k\text{-mod})$.

By semisimplicity,

$$\text{Hom}_D(H^0(A), k[1]) = \text{Ext}_k^1(H^0(A), k) = 0$$

and

$$\text{Hom}_D(H^0(A), k[2]) = \text{Ext}_k^2(H^0(A), k) = 0.$$

Hence the middle arrow is an isomorphism. Finally,

$$\text{Hom}_D(H^{-1}(A)[1], k[1]) \cong \text{Hom}_k(H^{-1}(A), k).$$

The truncation triangle and the long exact sequence are functorial in A , so the resulting isomorphism is natural. $\square$

Corollary 3.29 (Naturality square). *For every morphism $f : A \rightarrow A'$ in $D^{[-1,0]}(k\text{-mod})$, the square*

$$\begin{array}{ccc} \text{Ext}^1(A', k) & \longrightarrow & \text{Ext}^1(A, k) \\ \sim \downarrow & & \downarrow \sim \\ \text{Hom}_k(H^{-1}(A'), k) & \longrightarrow & \text{Hom}_k(H^{-1}(A), k) \end{array}$$

commutes, where the horizontal arrows are induced contravariantly by f and $H^{-1}(f)$.

Proof. This is the naturality assertion of Theorem 3.28 written as a commutative square. $\square$

Corollary 3.30 (Edge vanishing criterion). *If $A \in D^{[-1,0]}(k\text{-mod})$ and $H^{-1}(A) = 0$, then*

$$\text{Ext}^1(A, k) = 0.$$

Proof. Apply Theorem 3.28. $\square$

Remark 3.31 (Why the field hypothesis matters). Over a general coefficient ring, the same truncation argument contains outer terms

$$\text{Ext}^1(H^0(A), k) \quad \text{and} \quad \text{Ext}^2(H^0(A), k),$$

which need not vanish. Additional projectivity, injectivity, or homological-dimension hypotheses would be required.

Remark 3.32 (Splitting versus naturality). Lemma 3.25 explains the semisimple shape of the answer, but a chosen splitting need not be natural. The canonical truncation triangle is therefore essential for the naturality and representative transport used by the audit architecture.

Remark 3.33 (No reverse information). The canonical edge identifies the categorical group with the dual of H^{-1} . It does not identify H^{-1} with the full persistence profile and does not make the edge extractor faithful.

3.6 The one-way repaired bridge

Proposition 3.34 (Repaired degree-one vanishing implies edge vanishing). *Let $(F; W, \tau)$ have a complete eligible realization record. If*

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

then

$$H^{-1}(A_{F,W,\tau}) = 0.$$

Equivalently, the same conclusion follows from

$$\text{PH}_1(C_{\tau,W}F) = 0$$

when this notation is licensed by the degree-one representative record.

Proof. The edge-realization certificate gives

$$H^{-1}(A_{F,W,\tau}) \cong \text{Edge}_1(\text{Eval}_{1,W,\tau}(F)).$$

If the repaired profile vanishes, zero compatibility gives

$$\text{Edge}_1(\text{Eval}_{1,W,\tau}(F)) = \text{Edge}_1(0) = 0.$$

Hence $H^{-1}(A_{F,W,\tau}) = 0$. □

Remark 3.35 (Where the topological input enters). The only topological input is vanishing of the repaired windowed degree-one profile. The edge extractor and edge-realization certificate then translate that vanishing into the vector-space statement required by the canonical Ext-edge.

Theorem 3.36 (One-way repaired bridge). *Let $(F; W, \tau)$ have a complete eligible realization record. If*

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

then

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Equivalently, when the representative record licenses the notation,

$$\text{PH}_1(C_{\tau,W}F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Proof. Set

$$A_{F,W,\tau} := \mathcal{R}(C_{\tau,W}F).$$

Eligibility gives

$$A_{F,W,\tau} \in D^{[-1,0]}(k\text{-mod}).$$

By Proposition 3.34, repaired degree-one vanishing implies

$$H^{-1}(A_{F,W,\tau}) = 0.$$

Corollary 3.30 gives

$$\text{Ext}^1(A_{F,W,\tau}, k) = 0.$$

Substitute the definition of $A_{F,W,\tau}$. □

Remark 3.37 (Discharge theorem). Theorem 3.36 is a discharge theorem for the categorical clause of B-Gate⁺. It does not reconstruct the repaired profile from categorical vanishing.

Remark 3.38 (Independence from the metric repair proof). The categorical implication is logically independent of the proof of threshold-gap-safe stability. Metric repair controls how repaired vanishing may be transported between nearby inputs; eligibility and the canonical Ext-edge control the categorical discharge once the vanishing input has been established.

3.7 Gate integration and typed clause status

Definition 3.39 (Ext-edge discharge record). An Ext-edge discharge record contains:

- (i) the repaired evaluation artifact establishing $\text{Eval}_{I,W,\tau}(F) = 0$;
- (ii) the complete eligible realization record of Definition 3.21;
- (iii) the theorem identifier for Theorem 3.36;
- (iv) the canonical Ext-edge identification and naturality data;
- (v) the resulting categorical conclusion;
- (vi) immutable artifact and claim-register references sufficient for replay.

If the topological vanishing is transported under perturbation, the record also cites the relevant two-sided threshold-gap certificate and scalarized metric bound.

Definition 3.40 (Typed Ext-edge clause status). The categorical Ext clause has one of the following audit meanings:

- (i) `not_invoked`: the manifest explicitly excludes the Ext clause from scope;
- (ii) `undefined`: the clause is invoked but a required representative, realization, amplitude, edge, artifact, claim-status, or manifest item is missing or ill-typed;
- (iii) `pass`: the clause is well-typed and the Ext group is evidenced to vanish, either by Theorem 3.36 or by a separately verified direct computation;
- (iv) `reject`: the clause is well-typed and the Ext group is evidenced to be nonzero.

The statuses `undefined` and `not_invoked` are distinct and neither is a fabricated zero group.

Proposition 3.41 (Discharge of the Ext^1 -clause of B-Gate^+). *If $(F; W, \tau)$ is eligible and*

$$\text{Eval}_{I,W,\tau}(F) = 0,$$

then the categorical Ext clause passes:

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Proof. This is Theorem 3.36 read as a gate-discharge statement. □

Remark 3.42 (Remaining B-Gate^+ clauses). The categorical discharge is only one atomic clause. A full pass still requires independently:

$$\text{Eval}_{I,W,\tau}(F) = 0, \quad \text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau(\mathfrak{B}),$$

a complete and consistent manifest, admissible claim status, and a Type IV scope status equal to

$$\text{not_invoked} \quad \text{or} \quad \text{certified_zero}.$$

No passing categorical clause cancels another rejected or required undefined clause.

3.8 Representative and realization invariance

Definition 3.43 (Realization-equivalent eligible representatives). Two eligible representatives $C_{\tau,W}F$ and $C'_{\tau,W}F$ are realization-equivalent if there is a declared isomorphism

$$\alpha : \mathcal{R}(C_{\tau,W}F) \xrightarrow{\sim} \mathcal{R}(C'_{\tau,W}F)$$

in $D^b(k\text{-mod})$, and the two edge-realization certificates are compatible with $H^{-1}(\alpha)$.

Proposition 3.44 (Eligibility is invariant under admissible representative equivalence). *Let $C_{\tau,W}F$ and $C'_{\tau,W}F$ be realization-equivalent eligible representatives. Then amplitude, edge realization, and categorical clause typing transport across the declared equivalence.*

Proof. An isomorphism in the derived category preserves cohomology and amplitude. Compatibility of the edge certificates transports the edge-realization isomorphism. Therefore every eligibility component is preserved. $\square$

Proposition 3.45 (One-way bridge is representative-independent). *For realization-equivalent eligible representatives, the Ext groups are canonically identified along the declared realization equivalence, and the truth value of categorical vanishing is independent of the representative choice.*

Proof. The isomorphism α induces an isomorphism under the contravariant functor

$$\text{Hom}_{D^b(k\text{-mod})}(-, k[1]).$$

By Corollary 3.29, this agrees with dualizing the induced map on H^{-1} . Compatibility of the edge certificates transports the discharge record. $\square$

Remark 3.46 (Representative independence is not canonicity). Representative independence does not make $C_{\tau,W}F$ unique or canonical. It says only that the categorical verdict is stable under the specific realization equivalence declared by the proof object.

Remark 3.47 (Profile invariance is weaker). Monitored persistence-profile equivalence from Appendix B does not by itself imply realization equivalence. Ext use requires either a specific fixed representative or an evidenced realization equivalence.

3.9 Interaction with the E_1 detector

Definition 3.48 (Imported residual degree-one bar-mass detector). The symbol

$$E_1(F; W, \tau)$$

denotes the canonical residual degree-one bar-mass detector fixed in Chapter 3, Definition 3.40. Concretely, it is the sum of the lengths of all finite residual degree-one bars in

$$\text{Eval}_{1,W,\tau}(F),$$

counted with multiplicity. If residual infinite degree-one bars are present, the canonical value is $+\infty$, unless a separately declared truncated or finite-window policy applies.

For audit serialization, the finite-bar contribution and the presence or absence of residual infinite bars may be stored as separate fields. This serialization convention does not create a second definition of E_1 : its normative semantics remain those fixed in Chapter 3. The detector is not a complete persistence profile and not a metric or threshold-gap certificate.

Proposition 3.49 (Minimal E_1 -first implication). *If*

$$E_1(F; W, \tau) = 0$$

and the record separately certifies that no residual infinite degree-one bar is present, then

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

Consequently, under eligibility,

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Proof. Every nonempty finite barcode interval has positive length. Hence a zero sum implies that no residual finite bar is present. The separate infinite-bar certificate excludes all residual infinite bars. Therefore the repaired profile is zero, and Theorem 3.36 applies. $\square$

Remark 3.50 (No converse). This appendix uses E_1 only as an operational vanishing detector. It does not claim that E_1 is bottleneck-stable, a threshold-gap certificate, a metric-ledger component, or a complete invariant of the repaired profile.

3.10 Explicit failure of the reverse bridge

Proposition 3.51 (Reverse implication fails under the stated eligibility axioms). *Under Definitions 3.12–3.21, the implication*

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0 \implies \text{Eval}_{1,W,\tau}(F) = 0$$

is false in general.

Proof. Choose $L > \tau > 0$ and a finite filtered complex whose repaired degree-one profile on a suitable window is the nonzero interval module

$$\text{Eval}_{1,W,\tau}(F) \cong k_{[0,L]}.$$

Use a degree-one compatible representative of that profile.

Define the constant-zero realization functor

$$\mathcal{R}_0(-) := 0$$

and the constant-zero edge extractor

$$\text{Edge}_{1,0}(P) := 0$$

for every repaired profile P . Then

$$H^{-1}(\mathcal{R}_0(C_{\tau,W}F)) = 0 \xrightarrow{\sim} \text{Edge}_{1,0}(\text{Eval}_{1,W,\tau}(F)) = 0$$

is an edge-realization certificate, and the amplitude condition holds because $0 \in D^{[-1,0]}(k\text{-mod})$.

However,

$$\text{Ext}^1(0, k) = 0$$

while

$$\text{Eval}_{1,W,\tau}(F) \neq 0.$$

Thus reverse recovery requires an additional faithfulness theorem not present in the Core. $\square$

Remark 3.52 (What a reverse theorem would require). A reverse bridge would require a separately proved condition ensuring that the realization and edge extractor jointly detect the full repaired degree-one profile. Such faithfulness is a new Bridge Theorem with new hypotheses; it cannot be inferred from eligibility or Ext vanishing alone.

3.11 Eligibility manifest extension and obligation handoff

Definition 3.53 (Eligibility manifest extension). A proof object invoking the Ext edge records at least:

- (i) the repaired evaluation and pre-threshold profile identifiers;
- (ii) the representative identifier and construction artifact;
- (iii) the monitored degree, window, and threshold;
- (iv) the realization procedure, version, and artifact identifier;
- (v) the amplitude certificate;
- (vi) the edge extractor and its version;
- (vii) the edge-realization certificate and compatibility data;
- (viii) the coefficient field;
- (ix) the representative and realization equivalence policy;
- (x) the typed Ext-clause status;
- (xi) the theorem or direct-computation identifier supporting a pass;
- (xii) threshold-gap and scalarized metric records when the topological input is transported under perturbation;
- (xiii) claim-register entries, hashes, and provenance links sufficient for replay.

Definition 3.54 (Eligible-realization obligation bundle). The non-scalar obligation bundle generated by Appendix C contains:

- (i) representative availability;
- (ii) degree-one profile compatibility;
- (iii) realization-domain membership;
- (iv) realization compatibility or fixed-representative commitment;
- (v) amplitude certification;
- (vi) edge-extractor declaration and zero compatibility;
- (vii) edge-realization isomorphism;
- (viii) field-coefficient declaration;
- (ix) claim-status admissibility;
- (x) manifest completeness and consistency;
- (xi) immutable artifact identity.

Remark 3.55 (Appendix F/G handoff). Appendix F may integrate these obligations into the general status algebra, dependency closure, failure routing, and non-compensation rules. Appendix G may serialize them in the canonical proof-object schema.

Neither appendix changes the mathematical meaning of eligibility or supplies a missing proof merely by recording a status.

Remark 3.56 (Manifest status and failure routing). An invoked Ext clause with missing eligibility data is undefined. An inconsistent manifest yields a Type V claim/manifest rejection under the canonical Chapter 7 audit architecture. Only a complete and consistent record can support a categorical pass.

3.12 Eligible realization and Ext-edge package

Theorem 3.57 (Appendix C eligible-realization package). *Within the v19 eligible realization regime, the following package is available.*

(i) *The repaired degree-one input is*

$$\text{Eval}_{1,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_1(F)).$$

(ii) *An admissible representative realizes this profile at monitored degree one.*

(iii) *A realization datum produces*

$$A_{F,W,\tau} = \mathcal{R}(C_{\tau,W}F) \in D^{[-1,0]}(k\text{-mod}).$$

(iv) *The residual edge extractor is vector-space-valued and satisfies $\text{Edge}_1(0) = 0$.*

(v) *The edge-realization certificate identifies*

$$H^{-1}(A_{F,W,\tau}) \cong \text{Edge}_1(\text{Eval}_{1,W,\tau}(F)).$$

(vi) *The canonical truncation triangle yields a natural isomorphism*

$$\text{Ext}^1(A, k) \cong \text{Hom}_k(H^{-1}(A), k)$$

for every $A \in D^{[-1,0]}(k\text{-mod})$.

(vii) *Therefore*

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

(viii) *The discharge is invariant under declared realization-equivalent representative choices.*

(ix) *The reverse implication is false without an additional faithfulness theorem.*

(x) *An invoked Ext clause with incomplete eligibility is undefined, not pass and not categorical vanishing.*

Proof. Items (i)–(v) are Definitions 3.7, 3.8, 3.15, 3.16, and 3.18. Item (vi) is Theorem 3.28. Item (vii) is Theorem 3.36. Item (viii) is Proposition 3.45. Item (ix) is Proposition 3.51. Item (x) is Definition 3.40. $\square$

3.13 Explicit exclusions

The following are not asserted in Appendix C.

- (i) No implication

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0 \implies \text{Eval}_{1,W,\tau}(F) = 0$$

is asserted.

- (ii) No implication

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0 \implies \text{PH}_1(C_{\tau,W}F) = 0$$

is asserted.

- (iii) No global equivalence

$$\text{PH}_1 \iff \text{Ext}^1$$

is asserted.

- (iv) No persistence profile is directly identified with a finite-dimensional vector space without a declared edge extractor.

- (v) No faithfulness of Edge_1 is assumed.

- (vi) No profile-level representative equivalence automatically implies realization equivalence.

- (vii) No global strict filtered lift

$$C_{\tau} : \text{FiltCh}(k) \rightarrow \text{FiltCh}(k)$$

is asserted.

- (viii) No identity

$$C_{\tau,W}F = (C_{\tau}F)|_W$$

is asserted.

- (ix) No exactness or Serre localization of positive-threshold deletion is used.

- (x) No realization outside the eligible record may invoke the Core Ext-edge.

- (xi) No non-field coefficient generalization is claimed without separate homological hypotheses.

- (xii) No chosen derived splitting is treated as the source of naturality.

- (xiii) No E_1 detector is treated as a complete profile invariant, bottleneck-stable quantity, or threshold-gap certificate.

- (xiv) No metric budget repairs a missing representative, realization, amplitude certificate, edge extractor, edge isomorphism, claim status, or manifest field.

- (xv) No Type IV conclusion is supplied by the Ext-edge.

- (xvi) No Search-layer report, validity map, spectral indicator, Bridge Program, or AI confidence value discharges the categorical clause.

- (xvii) No problem-specific arithmetic, PDE, geometric, categorical, or cryptographic conclusion is proved.

3.14 Provenance and status

The semisimplicity of finite-dimensional vector spaces, the standard truncation triangle, and the derived Hom long exact sequence are standard. The v19-specific content is the typed assembly of these facts with repaired windowed persistence profiles, admissible representatives, a vector-space-valued residual edge extractor, eligibility records, typed clause status, and the explicit non-converse boundary.

Status labels for Appendix C. *Standard derived-category results:* Lemma 3.24, Lemma 3.25, Corollary 3.26, Theorem 3.28, Corollary 3.29, Corollary 3.30.

Core bridge theorem and corrections: Proposition 3.34, Theorem 3.36, Proposition 3.41, Proposition 3.51, Theorem 3.57.

Core definitions and eligibility policy: Definitions 3.3, 3.4, 3.5, 3.7, 3.8, 3.11, 3.12, 3.13, 3.15, 3.16, 3.18, 3.21, 3.27, 3.39, 3.40, 3.43, 3.48, 3.53, 3.54.

Conditional and operational propositions: Proposition 3.44, Proposition 3.45, Proposition 3.49.

Core doctrine and cautions: Remarks 3.1, 3.2, 3.6, 3.9, 3.10, 3.14, 3.17, 3.19, 3.20, 3.22, 3.23, 3.31, 3.32, 3.33, 3.35, 3.37, 3.38, 3.42, 3.46, 3.47, 3.50, 3.52, 3.55, 3.56.

Explicit exclusions: Subsection 3.13.

4 Appendix D: Tower Diagnostics and Terminal-Generic Type IV Defect Calculus

4.1 Scope and dependence on the repaired Core

This appendix supplies the detailed Foundation layer for the failure taxonomy and terminal-generic Type IV diagnostics of Chapter 5. It consumes:

- (i) the repaired windowed evaluation and threshold-gap discipline of Chapters 2–3 and Appendix A;
- (ii) the representative and actual-morphism discipline of Appendix B;
- (iii) the eligible one-way bridge of Chapter 4 and Appendix C;
- (iv) the typed audit statuses and Minimal Manifest hierarchy of Chapters 1 and 7.

All tower diagnostics are formed after the repaired evaluation

$$\text{Eval}_{i,W,\tau}(F) := T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

The diagnostic is not an invariant of a bare sequence of barcodes. It is an invariant of a declared comparison morphism between a typed tower-side source profile and a repaired apex profile.

The central v19 distinction is

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

under status `certified_zero`. This means absence of monitored *terminal-generic* kernel and cokernel defect relative to the declared artifact. It does not, without a separate no-interior-defect theorem, imply that the comparison morphism is an isomorphism.

Remark 4.1 (Appendix D does not enlarge the Core). This appendix does not restore a global exact collapse functor, manufacture a tower comparison morphism from a barcode matching, identify an undeclared apex with a categorical colimit, or promote zero terminal diagnostics to full comparison-map isomorphism. It gives the window-relative definitions and proof obligations already required by the Chapter 5 contract.

Remark 4.2 (The bounded-window repair). The former shorthand

$$\mathrm{gdim}(M) = \lim_{t \rightarrow +\infty} \dim_k M(t)$$

is valid only on a right-unbounded window. On a bounded window, extension by zero outside the window would force this limit to vanish and would make Type IV vacuous. The canonical v19 invariant is therefore the window-relative terminal-generic dimension tgdim_W .

4.2 Directed towers and repaired profile systems

Definition 4.3 (Directed tower). A *directed tower* in the declared filtered regime is a sequence

$$F_0 \longrightarrow F_1 \longrightarrow F_2 \longrightarrow \cdots$$

of objects of $\mathrm{FiltCh}(k)$ and filtration-preserving morphisms. We write $F_\bullet = \{F_n\}_{n \in \mathbb{N}}$.

Definition 4.4 (Declared apex object). A filtered object F_∞ is a *declared apex object* if it is supplied as the object against which the repaired tower-side source profile is compared. Its provenance may be an actual filtered colimit, a verified stabilization, a completion, or an implementation artifact. The provenance status is part of the tower record and is never inferred from the symbol F_∞ .

Remark 4.5 (Apex is declared, not automatic). The notation F_∞ does not assert existence of an objectwise filtered colimit inside the finite-type constructible regime. If the repaired apex evaluation, provenance, or comparison typing is absent, the apex is not a Core-readable Type IV input.

Definition 4.6 (Monitored degrees and monitored scope). For a fixed window W and threshold τ , let I_{mon} be the finite set of monitored homological degrees. More generally, a run may declare a finite monitored scope

$$\mathcal{M} \subseteq \{(i, W, \tau)\}.$$

Every total diagnostic is summed only over its declared finite scope. An infinite scope requires a separately proved summability or aggregation theorem.

Remark 4.7 (Constructibility caveat). A raw tower or limit may leave the constructible regime. Type IV is defined only when all repaired profiles, transition morphisms, source and apex profiles, comparison morphisms, and defect profiles belong to a declared abelian constructible profile category on the relevant window.

Definition 4.8 (Windowed collapsed finite-stage profile). For every stage F_n , monitored degree i , window W , and threshold $\tau > 0$, define

$$M_{n;i,W,\tau} := \mathcal{P}_{i,W,\tau}(F_n) := \mathrm{Eval}_{i,W,\tau}(F_n).$$

Definition 4.9 (Repaired windowed collapsed tower-profile system). A *repaired tower-profile system* at (i, W, τ) consists of:

- (i) the repaired stage profiles $M_{n;i,W,\tau}$;
- (ii) actual morphisms in the declared profile category

$$\rho_{n,m} : M_{n;i,W,\tau} \longrightarrow M_{m;i,W,\tau} \quad (n \leq m);$$

(iii) the identity and composition laws

$$\rho_{n,n} = \text{id}, \quad \rho_{m,\ell} \rho_{n,m} = \rho_{n,\ell};$$

(iv) provenance and artifact identifiers for the morphisms.

A sequence of barcode multisets, or a family of pairwise bottleneck matchings, is not by itself a repaired tower-profile system.

Definition 4.10 (Collapsed source profile). Given a repaired tower-profile system, a source profile

$$\text{ColimProfile}_{i,W,\tau}(F_\bullet)$$

is supplied with exactly one provenance status:

$$\text{genuine_colimit}, \quad \text{verified_stabilization}, \quad \text{implementation_artifact}.$$

In the first case it is a genuine colimit in the declared profile category. In the second it is a certified stabilized profile. In the third its relation to the tower is itself part of the artifact and is not promoted to a categorical colimit.

Definition 4.11 (Collapsed apex profile). For a declared apex F_∞ , define

$$\text{ApexProfile}_{i,W,\tau}(F_\infty) := \text{Eval}_{i,W,\tau}(F_\infty),$$

provided the repaired evaluation record for F_∞ is declared.

Remark 4.12 (No automatic colimit–collapse interchange). No identity

$$\text{colim}_n \text{Eval}_{i,W,\tau}(F_n) \cong \text{Eval}_{i,W,\tau}(F_\infty)$$

or

$$\text{colim}_n T_\tau^{\text{bar}}(M_n) \cong T_\tau^{\text{bar}}\left(\text{colim}_n M_n\right)$$

is available in the general Core. Any such comparison is precisely what the artifact must construct or justify.

4.3 Tower comparison artifacts

Definition 4.13 (Tower comparison artifact). A *tower comparison artifact* for (i, W, τ) consists of:

- (i) a repaired tower-profile system;
- (ii) a source profile with one of the provenance statuses of Definition 4.10;
- (iii) a repaired apex profile;
- (iv) an actual morphism in the declared constructible profile category,

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty);$$

- (v) a construction or proof that $\phi_{i,W,\tau}$ is well-typed;
- (vi) every repaired-evaluation, threshold-gap, crop-budget, representative, and metric-ledger record used to construct its source, target, or comparison;

- (vii) the terminal mode `bounded_terminal` or `right_unbounded`;
- (viii) terminal-tameness certificates and kernel/cokernel computation artifacts, or a proof that those computations occur in an abelian constructible profile category;
- (ix) immutable artifact and claim identifiers.

Remark 4.14 (Comparison artifact, not automatic map). The map $\phi_{i,W,\tau}$ is not produced by the existence of a filtered tower, a barcode decomposition, or a bottleneck matching. It must be explicitly constructed, or supplied canonically by a separately proved exact-admissible regime.

Remark 4.15 (A bottleneck matching is not a persistence morphism). A bottleneck matching controls metric discrepancy between barcodes. It is not, by itself, a natural transformation of persistence modules and therefore does not define the kernel or cokernel needed by Type IV.

Definition 4.16 (Exact-admissible tower artifact). A tower comparison artifact is *exact-admissible* if its transition morphisms, source construction, comparison map, kernels, and cokernels are supplied by a declared regime in which the retained-long-bar assignment is functorial and preserves every exact sequence used by the diagnostic.

Remark 4.17 (Order of operations). The normative order is

$$F_n \mapsto \mathbf{P}_i(F_n) \mapsto W_W \mathbf{P}_i(F_n) \mapsto T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F_n)),$$

followed by construction of the repaired profile system, source profile, apex profile, and actual comparison morphism. Windowing and threshold deletion are not silently interchanged.

Remark 4.18 (Why repaired collapse precedes diagnostics). The terminal defect is diagnosed on the repaired profile selected by the Core. This does not mean that threshold deletion is exact or globally functorial; the artifact must separately supply the morphisms that make the repaired tower well-typed.

4.4 Terminal-tame profiles and terminal-generic dimension

Definition 4.19 (Terminal-tame profile). Let $W = [u, v)$, where $v \in \mathbb{R} \cup \{+\infty\}$, and let M be a constructible persistence profile on W . We call M *terminal-tame* if:

- (i) its fibers near the terminal end are finite-dimensional;
- (ii) if $v < +\infty$, there exists $s < v$ such that every structure map $M(t) \rightarrow M(t')$ with $s \leq t \leq t' < v$ is an isomorphism;
- (iii) if $v = +\infty$, there exists $s \in \mathbb{R}$ such that every structure map $M(t) \rightarrow M(t')$ with $s \leq t \leq t'$ is an isomorphism.

Definition 4.20 (Generic and terminal-generic dimension). For a terminal-tame profile M on $W = [u, v)$, define

$$\text{tgdim}_W(M) := \begin{cases} \lim_{t \uparrow v} \dim_k M(t), & v < +\infty, \\ \lim_{t \rightarrow +\infty} \dim_k M(t), & v = +\infty. \end{cases}$$

When $v = +\infty$, the same quantity may be written $\text{gdim}(M)$. On a bounded window, gdim after zero extension is not the canonical diagnostic.

Proposition 4.21 (Well-definedness of terminal-generic dimension). *For every terminal-tame profile M on W , the quantity $\text{tgdim}_W(M)$ exists and is a finite nonnegative integer.*

Proof. Terminal tameness supplies a terminal subinterval on which all structure maps are isomorphisms. The finite-dimensional fibers on that subinterval therefore have one constant dimension. This value is exactly the displayed limit. $\square$

Definition 4.22 (Cofinal bar). A bar I in a barcode profile on $W = [u, v)$ is *cofinal in W* if

$$\sup I = v \quad \text{when } v < +\infty,$$

or if it is unbounded above when $v = +\infty$. A cofinal bar on a bounded window is a boundary-reaching bar relative to that window; it is not a global infinite bar.

Proposition 4.23 (Terminal dimension counts cofinal bars). *Let M be terminal-tame and interval-decomposable on W . Then $\text{tgdim}_W(M)$ equals the multiplicity of bars cofinal in W . In particular, on a right-unbounded window it equals the multiplicity of infinite bars.*

Proof. Choose a terminal tail on which every structure map is an isomorphism. An interval summand contributes one dimension to every fiber on that tail exactly when it is cofinal in W . Every noncofinal interval dies before the tail. Summing the surviving summands proves the claim. $\square$

Remark 4.24 (Endpoint conventions). Endpoint bookkeeping at finite endpoints does not alter terminal-generic dimension. What matters is whether the bar reaches the declared terminal end cofinally.

Proposition 4.25 (Terminal tameness passes to defects). *Let $\phi : M \rightarrow N$ be a morphism of constructible profiles on W . If M and N are terminal-tame, then $\ker \phi$ and $\text{coker } \phi$ are terminal-tame.*

Proof. Choose a common terminal tail on which the structure maps of M and N are isomorphisms. Naturality of ϕ gives commuting squares on that tail. The induced maps on pointwise kernels and cokernels are therefore isomorphisms, and finite-dimensionality is inherited pointwise. $\square$

4.5 Defect profiles and canonical diagnostics

Definition 4.26 (Kernel and cokernel defect profiles). For a valid tower comparison artifact, define

$$\text{Defect}_{i,W,\tau}^{\ker} := \ker(\phi_{i,W,\tau}), \quad \text{Defect}_{i,W,\tau}^{\text{coker}} := \text{coker}(\phi_{i,W,\tau}).$$

These are objects of the declared constructible profile category on W .

Remark 4.27 (Kernel and cokernel require a profile regime). The defect profiles are not inferred from a tower of filtered objects or a barcode matching. They require the actual comparison morphism and an abelian profile regime in which pointwise kernels and cokernels are defined.

Remark 4.28 (Interpretation of kernel and cokernel defects). The kernel records repaired tower-side classes killed by the comparison. The cokernel records repaired apex classes not supplied by the tower-side source. Both are artifact-relative, after-window, after-threshold defects.

Definition 4.29 (Full defect profile). The full defect profile is

$$\text{Defect}_{i,W,\tau} := (\text{Defect}_{i,W,\tau}^{\ker}, \text{Defect}_{i,W,\tau}^{\text{coker}}).$$

Definition 4.30 (Degreewise tower diagnostics). Assume that both defect profiles are terminal-tame. Define

$$\mu_{i,W,\tau} := \text{tgdim}_W(\text{Defect}_{i,W,\tau}^{\text{ker}}), \quad \nu_{i,W,\tau} := \text{tgdim}_W(\text{Defect}_{i,W,\tau}^{\text{coker}}).$$

Thus μ measures terminal-generic kernel defect and ν measures terminal-generic cokernel defect.

Definition 4.31 (Total collapse diagnostics). For fixed (W, τ) and finite I_{mon} , define

$$\mu_{\text{Collapse}}(W, \tau) := \sum_{i \in I_{\text{mon}}} \mu_{i,W,\tau}, \quad \nu_{\text{Collapse}}(W, \tau) := \sum_{i \in I_{\text{mon}}} \nu_{i,W,\tau}.$$

For a finite run scope $\mathcal{M}$, define the run totals by summing over $(i, W, \tau) \in \mathcal{M}$. When the scope is clear, the arguments are suppressed. No infinite aggregate is admitted without a separate summability theorem.

The symbol μ_{Collapse} is deprecated. The canonical cokernel notation is ν_{Collapse} .

Remark 4.32 (Meaning of μ). On a bounded window, $\mu_{i,W,\tau}$ counts kernel defect bars cofinal to the right endpoint. On a right-unbounded window it counts infinite kernel bars.

Remark 4.33 (Meaning of ν). On a bounded window, $\nu_{i,W,\tau}$ counts cokernel defect bars cofinal to the right endpoint. On a right-unbounded window it counts infinite cokernel bars.

Proposition 4.34 (Diagnostics count terminal-cofinal defects). *For every valid terminal-tame artifact, $\mu_{i,W,\tau}$ and $\nu_{i,W,\tau}$ equal the multiplicities of cofinal interval summands in the kernel and cokernel defect profiles, respectively. On a right-unbounded window these are precisely infinite-bar defects.*

Proof. Apply Proposition 4.23 to the two defect profiles. □

Definition 4.35 (Type IV diagnostic record). A Type IV diagnostic record contains the declared scope and exactly one status

`not_invoked,      undefined,      certified_zero,      obstructed.`

Its status-dependent fields are:

- (i) for `not_invoked`: an explicit no-tower scope rationale and no numerical diagnostic;
- (ii) for `undefined`: a partial/invalid artifact identifier or a missing-artifact marker, together with every missing or ill-typed obligation, and no asserted numerical Core diagnostic;
- (iii) for `certified_zero` or `obstructed`: a valid artifact identifier, source provenance, terminal mode, terminal-tameness certificates, defect-profile artifacts, all degreewise and total μ, ν values, and a separately typed interior-defect status.

The status is `certified_zero` exactly when all invoked inputs are valid and $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$; it is `obstructed` exactly when all inputs are valid and $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0)$.

4.6 Typed Type IV status

Definition 4.36 (Monitored terminal-generic Type IV obstruction). Relative to a valid tower comparison artifact, a run has a *Type IV obstruction* if

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0).$$

On a bounded window this is a defect cofinal to the declared right endpoint; on a right-unbounded window it is an infinite-bar defect.

Definition 4.37 (Type IV-free in the monitored terminal-generic sense). Relative to a valid terminal-tame artifact, a run is *Type IV-free in the monitored terminal-generic sense* if

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0).$$

Theorem 4.38 (Characterization of certified Type IV-free status). *A tower diagnostic record has status `certified_zero` if and only if:*

- (i) *the tower comparison artifact is valid;*
- (ii) *its kernel and cokernel defect profiles are terminal-tame;*
- (iii) $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$.

It has status `obstructed` when the first two clauses hold and $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0)$. Missing or ill-typed input yields `undefined`, never zero.

Proof. These are exactly the validity, terminal-tameness, and numerical clauses in Definition 4.35. The first two clauses are necessary to ensure that the displayed numbers are defined Core diagnostics rather than missing values. $\square$

Remark 4.39 (No full-isomorphism conclusion). The equality $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ says that no defect bar reaches the terminal end of the declared window. It does not imply

$$\ker \phi_{i,W,\tau} = 0 \quad \text{or} \quad \text{coker } \phi_{i,W,\tau} = 0.$$

Interior defect bars may remain.

Remark 4.40 (Undefined is not zero). Absence of an actual morphism, invalid source provenance, failure of terminal tameness, or unavailable kernel/cokernel artifacts yields `undefined`. Reporting that state as $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ is a Type V manifest failure.

Remark 4.41 (No scalar repair of Type IV typing). No metric budget or threshold clearance can manufacture a missing profile system, comparison morphism, terminal-tameness certificate, or defect artifact. Those are non-scalar proof obligations.

4.7 Interior defects and the full-isomorphism upgrade

Definition 4.42 (Interior or noncofinal defect). An *interior defect* is a noncofinal interval summand of either $\ker \phi_{i,W,\tau}$ or $\text{coker } \phi_{i,W,\tau}$. On a right-unbounded window these are exactly the finite-bar defects. On a bounded window, a globally finite bar reaching the right endpoint is cofinal and is therefore counted by $\text{tgd}_{\text{dim}_W}$, not classified as an interior defect.

Definition 4.43 (No-interior-defect condition). A comparison artifact satisfies the *no-interior-defect condition* if its kernel and cokernel have no noncofinal interval summands. The legacy phrase “no-finite-defect” is retained only as a label alias; the window-relative notion is no-interior-defect.

Proposition 4.44 (Zero diagnostics plus no interior defects imply isomorphism). *Assume the defect profiles are terminal-tame and interval-decomposable. If*

$$(\mu_{i,W,\tau}, \nu_{i,W,\tau}) = (0, 0)$$

and the no-interior-defect condition holds, then $\phi_{i,W,\tau}$ is an isomorphism in the declared abelian constructible profile category.

Proof. Zero terminal diagnostics exclude every cofinal summand of the kernel and cokernel. The no-interior-defect condition excludes every noncofinal summand. Thus both defect profiles have empty interval decompositions and are zero. A morphism with zero kernel and cokernel is an isomorphism in the declared abelian category. $\square$

Example 4.45 (Zero terminal diagnostics with a nonisomorphism). Let $W = [0, 10)$ and consider

$$0 \longrightarrow k_{[2,3]}.$$

Its cokernel is $k_{[2,3]}$, so the map is not an isomorphism. The bar is not cofinal in W , hence

$$\text{tgdim}_W(k_{[2,3]}) = 0.$$

Thus zero terminal diagnostics do not exclude an interior defect.

Remark 4.46 (Why the interior condition is separated). Type IV intentionally monitors terminal-generic failure. Full comparison isomorphism is stronger and requires a separately recorded interior-defect verdict. This prevents a zero terminal count from being overread as complete agreement.

4.8 Pure kernel, pure cokernel, and mixed modes

Definition 4.47 (Pure kernel Type IV). A valid artifact has *pure kernel Type IV* if

$$\mu_{\text{Collapse}} > 0, \quad \nu_{\text{Collapse}} = 0.$$

Definition 4.48 (Pure cokernel Type IV). A valid artifact has *pure cokernel Type IV* if

$$\mu_{\text{Collapse}} = 0, \quad \nu_{\text{Collapse}} > 0.$$

Definition 4.49 (Mixed Type IV). A valid artifact has *mixed Type IV* if

$$\mu_{\text{Collapse}} > 0, \quad \nu_{\text{Collapse}} > 0.$$

Remark 4.50 (Interpretation of the three modes). Let K_W denote one cofinal interval bar: $k_{[u,v]}$ for bounded $W = [u, v)$, and $k_{[u,+\infty)}$ for a right-unbounded window. Then the zero maps

$$K_W \rightarrow 0, \quad 0 \rightarrow K_W, \quad K_W \oplus 0 \rightarrow 0 \oplus K_W$$

have diagnostics $(1, 0)$, $(0, 1)$, and $(1, 1)$, respectively. These are artifact-level kernel loss, apex-only birth, and mixed failure. No external geometric or arithmetic interpretation follows without an interface theorem.

4.9 Finite-stage admissibility does not determine Type IV

Definition 4.51 (Finite-level collapse admissibility). A finite stage F_n is *finite-level collapse-admissible* on (W, τ) if its repaired topological clause passes and, when the Ext clause is invoked, a valid eligible realization record discharges that clause. This stagewise notion contains no undeclared tower-comparison verdict.

Proposition 4.52 (Finite admissibility does not imply monitored tower admissibility). *There exists a valid artifact-level witness in which every finite stage is finite-level collapse-admissible while the declared source-to-apex comparison has pure cokernel Type IV.*

Proof. Fix $W = [u, v)$ and let every repaired finite-stage profile be zero, with zero transition morphisms. Take the source profile to be the genuine colimit zero. Declare an apex profile K_W consisting of one cofinal bar and take the actual comparison morphism

$$0 \longrightarrow K_W.$$

Every finite stage has zero repaired degree-one profile and may be represented by the zero filtered complex. The comparison cokernel is K_W , so

$$(\mu_{i,W,\tau}, \nu_{i,W,\tau}) = (0, 1).$$

The finite profiles and the cofinal apex barcode admit filtered realizations by the finite-barcode realization theorem of Appendix B, Theorem 2.21. This is a declared apex-mismatch witness; it does not claim that an arbitrary externally chosen apex is the categorical colimit of the tower. $\square$

Remark 4.53 (Why Type IV is invisible to finite stages). Finite-stage evaluations do not determine the kernel or cokernel of an undeclared source-to-apex morphism. Type IV becomes meaningful only after the comparison artifact has been supplied.

4.10 Artifact-coherent stable bands

Definition 4.54 (Degreewise artifact-coherent stable band). Fix i , a window W , a tower-profile construction rule, source provenance, and a terminal mode. An interval $J \subset (0, +\infty)$ is a *degreewise stable band* if for every $\tau \in J$:

- (i) the same declared construction rule supplies a valid artifact $\phi_{i,W,\tau}$;
- (ii) source provenance and terminal mode do not change silently;
- (iii) the defect profiles are terminal-tame;
- (iv) every repaired comparison used by the artifact has its required two-sided threshold-gap, crop-budget, ledger, and scalarization records;
- (v) the Type IV diagnostic status is `certified_zero`.

Definition 4.55 (Global artifact-coherent stable band). A threshold interval J is a *global stable band* for a finite monitored degree set if Definition 4.54 holds coherently for every monitored degree and the total record is `certified_zero` for every $\tau \in J$.

Proposition 4.56 (Degreewise stable bands exclude degreewise Type IV). *If J is a degreewise stable band for i , then no terminal-generic Type IV obstruction occurs in degree i for any $\tau \in J$, relative to the declared artifact family.*

Proof. Every threshold in the band has a valid record with status `certified_zero`. Theorem 4.38 applies pointwise. $\square$

Proposition 4.57 (Global stable bands exclude monitored Type IV). *If J is a global stable band, then every $\tau \in J$ is Type IV-free in the monitored terminal-generic sense relative to its declared artifact.*

Proof. Apply Proposition 4.56 in every monitored degree and sum over the finite monitored scope. $\square$

Remark 4.58 (Uniform reserve is a separate claim). If one metric budget is claimed uniformly over J , the manifest must provide a common ledger $\mathfrak{d}_{\text{met},J}$ and prove

$$\text{val}_B(\mathfrak{d}_{\text{met},J}) < \inf_{\tau \in J} \text{gap}_\tau(\mathcal{B}_\tau).$$

Pointwise positive reserves do not imply that this infimum is positive.

Remark 4.59 (Stable band does not mean full-isomorphism band). A stable band controls terminal-generic defects. Interior defects may remain at some or all thresholds.

Definition 4.60 (Full-isomorphism band). A threshold interval J is a *full-isomorphism band* if every artifact $\phi_{i,W,\tau}$, for every monitored degree and every $\tau \in J$, is an isomorphism in the declared constructible profile category.

Proposition 4.61 (Full-isomorphism band implies stable band). *Every artifact-coherent full-isomorphism band is a global stable band.*

Proof. An isomorphism has zero kernel and cokernel, hence zero terminal diagnostics and a `certified_zero` record, provided the remaining artifact fields are present. $\square$

Remark 4.62 (Converse requires interior-defect data). A global stable band becomes a full-isomorphism band only after the no-interior-defect condition is verified throughout the band.

4.11 Integration with admissibility and B-Gate⁺

Definition 4.63 (Tower-clean structural collapse admissibility). A declared run is *tower-clean structurally collapse-admissible* if:

- (i) $\text{Eval}_{1,W,\tau}(F) = 0$;
- (ii) if the Ext clause is invoked, an eligible realization record exists and the clause is discharged;
- (iii) the Type IV status is either `not_invoked` under an explicit no-tower scope decision or `certified_zero`;
- (iv) no Type V claim-status or manifest defect occurs in the structural record.

Proposition 4.64 (Eligible reduction of tower-clean admissibility). *In the eligible regime of Chapter 4 and Appendix C, the categorical clause is discharged by repaired degree-one vanishing. Hence the independent structural conditions reduce to*

$$\text{Eval}_{1,W,\tau}(F) = 0$$

and a correctly typed Type IV status equal to `not_invoked` or `certified_zero`, together with absence of Type V failure.

Proof. The one-way bridge, Theorem 3.36, discharges the Ext clause from the repaired topological clause in the eligible regime. Type IV remains independent because it concerns a source-to-apex comparison morphism. A no-tower claim records `not_invoked`; an invoked tower claim requires a valid `certified_zero` record. $\square$

Remark 4.65 (Relation to B-Gate⁺). A final audited B-Gate⁺ pass additionally requires every applicable two-sided threshold-gap, crop-budget, scalarized metric-ledger, representative, claim-status, and manifest clause. Type IV zero cannot compensate for failure of those independent obligations.

4.12 Diagnostic equivalence and invariance

Definition 4.66 (Diagnostically equivalent tower artifacts). Two valid tower artifacts are *diagnostically equivalent* at (i, W, τ) if their kernel and cokernel defect profiles are isomorphic in the declared profile category by isomorphisms compatible with the terminal mode and terminal-tameness records.

Proposition 4.67 (Invariance of diagnostics). *Diagnostically equivalent artifacts have equal degreewise diagnostics. If the equivalence holds over the same finite monitored scope, their total μ_{Collapse} and ν_{Collapse} values agree.*

Proof. Terminal-generic dimension is invariant under profile isomorphism. Apply this to the kernel and cokernel defect profiles and sum over the common finite scope. $\square$

Remark 4.68 (Representative independence). The result is artifact-relative, not a claim that arbitrary strict filtered representatives are canonically identical. Representative changes must preserve the actual comparison artifact and its defect profiles in the declared sense.

4.13 Manifest extension and obligation handoff

Definition 4.69 (Type IV manifest entry). Every run records a Type IV scope status. When tower semantics are invoked, the manifest additionally records:

- (i) the monitored scope and terminal mode;
- (ii) tower-profile-system and transition-morphism artifacts;
- (iii) source provenance and apex evaluation records;
- (iv) the actual comparison morphism and its typing proof;
- (v) terminal-tameness and defect-profile artifacts;
- (vi) degreewise and total μ, ν values;
- (vii) the separately typed interior-defect status;
- (viii) every threshold-gap, crop-budget, ledger, representative, claim, and immutable provenance reference consumed by the artifact.

An incomplete invoked entry yields undefined; an inconsistent entry or undefined-as-zero declaration is a Type V rejection.

Definition 4.70 (Type IV obligation bundle). Appendix D forwards the following non-scalar obligations to the Appendix F status calculus and Appendix G manifest schema:

tower_system,	actual_comparison_morphism,
source_provenance,	apex_evaluation,
terminal_tameness,	defect_profiles,
diagnostic_scope,	interior_defect_status,
artifact_identity,	claim_status.

No scalar budget purchases any of these obligations.

4.14 Terminal-generic Type IV package

Theorem 4.71 (Appendix D terminal-generic Type IV package). *Within the repaired constructible tower regime, the following package is available.*

- (i) *Type IV is defined only relative to a repaired tower-profile system, typed source provenance, a repaired apex profile, and an actual comparison morphism.*
- (ii) *On a bounded window, tgdim_W counts defect bars cofinal to the right endpoint; on a right-unbounded window it counts infinite bars.*
- (iii) *Terminal tameness makes tgdim_W a well-defined finite integer and passes from terminal-tame source and target profiles to kernel and cokernel defects.*
- (iv) *The canonical diagnostics are*

$$\mu_{i,W,\tau} = \text{tgdim}_W(\ker \phi_{i,W,\tau}), \quad \nu_{i,W,\tau} = \text{tgdim}_W(\text{coker } \phi_{i,W,\tau}).$$
- (v) *Total diagnostics are finite-scope sums; infinite aggregation requires a separate theorem.*
- (vi) *The four statuses are `not_invoked`, `undefined`, `certified_zero`, and `obstructed`.*
- (vii) *An invoked record is `certified_zero` exactly when the artifact is valid, the defects are terminal-tame, and $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$.*
- (viii) *Zero terminal diagnostics are weaker than comparison-map isomorphism; the latter additionally requires absence of interior defects.*
- (ix) *Finite-stage admissibility does not determine Type IV because the diagnostic belongs to the declared source-to-apex comparison.*
- (x) *A stable band is an artifact-coherent family of passing records, not a pointwise collection of unrelated numerical zeros.*
- (xi) *In the eligible regime, repaired degree-one vanishing discharges the Ext clause but does not discharge Type IV or any metric/manifest clause.*
- (xii) *Missing morphisms, terminal-tameness, provenance, or defect artifacts yield `undefined`, never numerical zero.*

Proof. Items (i)–(v) are Definitions 4.9, 4.13, 4.20, 4.30, and 4.31, together with Propositions 4.21–4.34. Items (vi)–(vii) are Definition 4.35 and Theorem 4.38. Item (viii) is Proposition 4.44 and Example 4.45. Item (ix) is Proposition 4.52. Item (x) is the stable-band contract. Item (xi) is Proposition 4.64. Item (xii) is the typed failure routing of Remarks 4.40 and 4.41. $\square$

4.15 Explicit exclusions

The following are not asserted in Appendix D.

- (i) No barcode matching is treated as the tower comparison morphism.
- (ii) No comparison $\text{colim}_n \text{Eval}(F_n) \rightarrow \text{Eval}(F_\infty)$ is automatic.

- (iii) No colimit–collapse interchange is asserted in the general Core.
- (iv) No global exactness, Serre localization, or strict global filtered collapse functor is used.
- (v) No bounded-window diagnostic is computed from the dimension after zero extension to $+\infty$.
- (vi) No globally finite but window-cofinal bar is excluded from tgdim_W .
- (vii) No missing or ill-typed artifact is interpreted as $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$.
- (viii) No $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ implies full comparison-map isomorphism without a no-interior-defect theorem.
- (ix) No pointwise family of unrelated zero diagnostics defines a stable band.
- (x) No finite-stage pass determines an undeclared tower diagnostic.
- (xi) No metric budget repairs a missing morphism, provenance record, terminal-tameness certificate, or manifest field.
- (xii) No local PH–Ext reverse bridge is introduced.
- (xiii) No Search, AI, spectral, or advisory proxy replaces the actual artifact.
- (xiv) No geometric, arithmetic, PDE, cryptographic, or categorical interpretation follows without a separately proved interface theorem.

4.16 Provenance and status

Appendix A fixes the repaired evaluation, actual-morphism requirement, and canonical μ/ν notation contract. Appendix B supplies representative tower and comparison-artifact obligations. Appendix C supplies only the eligible one-way categorical discharge. The present appendix gives the full window-relative terminal-generic calculus required by Chapter 5.

Status labels for Appendix D. *Foundation definitions:* Definitions [4.3](#), [4.4](#), [4.6](#), [4.8](#), [4.9](#), [4.10](#), [4.11](#), [4.13](#), [4.16](#), [4.19](#), [4.20](#), [4.22](#), [4.26](#), [4.29](#), [4.30](#), [4.31](#), [4.35](#), [4.36](#), [4.37](#), [4.42](#), [4.43](#), [4.47](#), [4.48](#), [4.49](#), [4.51](#), [4.54](#), [4.55](#), [4.60](#), [4.63](#), [4.66](#), [4.69](#), [4.70](#).

Foundation propositions and theorems: Propositions [4.21](#), [4.23](#), [4.25](#), [4.34](#), [4.44](#), [4.52](#), [4.56](#), [4.57](#), [4.61](#), [4.64](#), [4.67](#); Theorems [4.38](#), [4.71](#).

Operational cautions: Remarks [4.1](#), [4.2](#), [4.5](#), [4.7](#), [4.12](#), [4.14](#), [4.15](#), [4.17](#), [4.27](#), [4.39](#), [4.40](#), [4.41](#), [4.46](#), [4.53](#), [4.58](#), [4.59](#), [4.62](#), [4.65](#), [4.68](#).

Explicit exclusions: Subsection [4.15](#).

5 Appendix E: Window, Overlap, κ -Restart, and Record-Level Gluing

5.1 Scope and dependence on the repaired Core

This appendix is the Foundation expansion of Chapter 6 in the AK-HPDST v19 repaired Core. It consumes the repaired evaluation, representative, eligible-realization, and typed Type IV disciplines of Chapters 2–5 and Appendices A–D. Its purpose is to specify exactly when local certificate records may be compared, continued, and assembled without silently asserting an unsupported global-object theorem.

The central separations are:

- (i) a right-open partition cover is used for sequential continuation and Restart;
- (ii) an overlap-admitting cover is used for pairwise and higher compatibility;
- (iii) the Overlap Gate compares post-threshold profiles, while threshold safety is certified on pre-threshold profiles;
- (iv) pairwise overlap compatibility is not higher Čech coherence;
- (v) Restart is licensed only by an explicit κ_k -admissible continuation artifact;
- (vi) ordinary unweighted summability is not sufficient under contracting κ_k ;
- (vii) finite and countable gluing conclusions concern certificate records and atlases, not automatically the underlying mathematical objects.

Every windowed persistence profile is evaluated in the fixed order

$$\mathbf{P}_i(F) \mapsto W_W \mathbf{P}_i(F) \mapsto T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)) = \text{Eval}_{i,W,\tau}(F).$$

Remark 5.1 (Appendix E does not enlarge the Core). This appendix does not create a universal Restart constant, an automatic continuation map, an unrestricted local-to-global theorem, or an object-level descent theorem. It develops a typed record architecture whose non-scalar obligations cannot be repaired by enlarging a numerical budget. Search-side maps, AI-generated proposals, and advisory quantities remain non-verdict unless they re-enter through independently verified Core records.

5.2 Right-open windows, partitions, and overlap covers

Definition 5.2 (Right-open window). A *right-open Core window* is an interval

$$W = [u, v), \quad -\infty < u < v \leq +\infty.$$

It is bounded when $v < +\infty$ and right-unbounded when $v = +\infty$.

Definition 5.3 (Declared window range). A *declared window range* is an interval $\Omega \subseteq \mathbb{R}$ together with the cover type, index set, endpoint convention, and audit scope on which local certification is performed.

Remark 5.4 (Why right-open windows). If

$$W_k = [u_k, u_{k+1}), \quad W_{k+1} = [u_{k+1}, u_{k+2}),$$

then $W_k \cap W_{k+1} = \emptyset$. Hence a partition does not double-count boundary parameters. This convention does not itself provide a continuation law across the boundary.

Definition 5.5 (Window restriction). For $M \in \text{Pers}_k^{\text{cons}}$, the window restriction $W_W M$ is the restriction of M to W , extended by zero outside W only when a globally indexed profile is required. At barcode level, each interval is clipped to W and empty intersections are discarded.

Definition 5.6 (Core window-evaluation order). For a filtered object F , monitored degree i , window W , and threshold $\tau > 0$, define

$$B_{i,W}(F) := W_W \mathbf{P}_i(F), \quad \text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(B_{i,W}(F)).$$

The pre-threshold profile $B_{i,W}(F)$ is the object on which clearance is measured.

Remark 5.7 (No unconditional crop–collapse commutation). In general,

$$W_W T_\tau^{\text{bar}} \not\equiv T_\tau^{\text{bar}} W_W.$$

A crop-after-collapse substitution is permitted only under the exact all-retained-bars criterion of Appendix A, namely Definition 1.45 and Proposition 1.46, including globally retained infinite bars clipped by bounded windows.

Definition 5.8 (MECE windowing / right-open partition cover). A family

$$\mathcal{W}_{\text{MECE}} = \{W_k = [u_k, u_{k+1})\}_{k \in K}$$

is a *right-open partition cover* of Ω if the windows are pairwise disjoint, their union is Ω , and the index order is part of the data whenever sequential continuation is invoked. The term *MECE windowing* is retained as an operational alias.

Definition 5.9 (MECE certificate sequence). A *MECE certificate sequence* is an ordered family $\{C_k\}_{k \in K}$ of complete local certificate records indexed by a right-open partition cover. Ordering alone does not certify continuation.

Remark 5.10 (MECE windowing is not overlap gluing). A disjoint partition has no nonempty overlap on which an Overlap Gate can be evaluated. Conversely, an overlap cover does not by itself supply an ordered continuation path. The two cover types have different proof obligations.

Definition 5.11 (Overlap-admitting cover). A family $\mathcal{U} = \{U_\alpha\}_{\alpha \in A}$ of right-open windows is an *overlap-admitting cover* of Ω if

$$\Omega = \bigcup_{\alpha \in A} U_\alpha$$

and every nonempty finite intersection used by the certificate logic is explicitly indexed and recorded.

Definition 5.12 (Overlap stratum). For a finite index set $\sigma = \{\alpha_0, \dots, \alpha_p\}$, define

$$U_\sigma := U_{\alpha_0} \cap \dots \cap U_{\alpha_p}.$$

A stratum participates only when it is nonempty and included in the declared nerve record.

Definition 5.13 (Declared overlap structure). A *declared overlap structure* records the cover, every nonempty nerve simplex used by the run, its face relations, common thresholds, monitored degrees, metric ledgers and scalarizations, representative and realization provenance, and typed Type IV scope information.

Remark 5.14 (Why overlap data must be declared). Without an indexed nerve and immutable overlap artifacts, an auditor cannot determine which local records, thresholds, face restrictions, budgets, or Type IV artifacts were compared. Mere set-theoretic overlap is not a certificate.

5.3 Scalarized reserves and non-scalar obligations

Definition 5.15 (Bottleneck scalarization for Appendix E). Let $(Q, \oplus, \preceq, 0_Q)$ be the declared commutative ledger quantale. A bottleneck scalarization is a monotone subadditive map

$$\text{val}_B : Q \longrightarrow [0, +\infty], \quad \text{val}_B(0_Q) = 0, \quad \text{val}_B(q \oplus q') \leq \text{val}_B(q) + \text{val}_B(q').$$

Only after this scalarization may a ledger value be compared with a real threshold clearance.

Definition 5.16 (Gate-scoped threshold reserve). Let $\mathfrak{B} = \{B_j\}_{j \in J}$ be the finite pre-threshold profile family consumed by one local, overlap, simplex, or Restart gate, and let ε be its certified scalarized bottleneck budget. Set

$$\text{gap}_\tau(\mathfrak{B}) := \min_{j \in J} \text{clear}_\tau(B_j), \quad \text{res}_\tau(\mathfrak{B}; \varepsilon) := \text{gap}_\tau(\mathfrak{B}) - \varepsilon.$$

This reserve belongs only to the declared gate and profile family.

Definition 5.17 (Non-scalar continuation and coherence obligations). Representative existence, actual comparison or continuation morphisms, threshold reconciliation, κ -admissibility, higher-face coherence, terminal tameness, Type IV typing, claim status, manifest completeness, and artifact readability are non-scalar obligations. No increased budget, larger gap, or summability estimate can replace them.

Remark 5.18 (Obligation handoff to Appendices F and G). The statuses and dependency closure of the non-scalar obligations in Definition 5.17 are forwarded to the typed status calculus of Appendix F. Their immutable identifiers, evidence references, scope fields, and failure routes are forwarded to the Manifest schema of Appendix G. This handoff records obligations; it does not convert missing evidence into a pass.

5.4 Certificate records

Definition 5.19 (Window-restricted certificate record). A *window-restricted certificate record* on $(F; W, \tau)$ records at least:

- (i) immutable object, window, threshold, and monitored-degree identifiers;
- (ii) the pre-threshold profiles $B_{i,W}(F)$ and post-threshold profiles $\text{Eval}_{i,W,\tau}(F)$;
- (iii) clearance, gap, metric-ledger provenance, scalarization, and positive-reserve verdicts;
- (iv) the local B-Gate⁺, representative, eligibility, and typed Type IV statuses;
- (v) every invoked representative, realization, tower, comparison, crop, or continuation artifact;
- (vi) a local status pass, reject, or undefined;
- (vii) artifact references sufficient for independent reconstruction.

Definition 5.20 (Globally coherent certificate record). A family of local records forms a *globally coherent certificate record system* only if every required local, pairwise, higher-simplex, Restart, budget, Type IV, and manifest clause is present and typed. The phrase does not denote a single global B-Gate⁺ pass unless an independent global object and global gate record are supplied.

Remark 5.21 (Certificate record versus proof object). A certificate record is the audit-readable representation of declared proof data. Schema completeness does not by itself prove the mathematical clauses stored in the record, and record gluing does not construct a new mathematical object.

5.5 Direct common-window comparison and the sharp Overlap Gate

Definition 5.22 (Overlap budget). For a nonempty recorded simplex σ , let $\mathfrak{d}_\sigma \in \mathcal{Q}$ be the gate-scoped metric ledger and define its scalarized budget

$$\varepsilon_\sigma := \text{val}_B(\mathfrak{d}_\sigma) \in [0, +\infty].$$

For a pair $\sigma = \{\alpha, \beta\}$, this is the overlap budget formerly denoted $\Sigma\delta_{\alpha\beta}$.

Definition 5.23 (Preferred repaired overlap profile). For $U_{\alpha\beta} := U_\alpha \cap U_\beta \neq \emptyset$, define the direct common-window profiles

$$B_{i,\alpha\beta}^\alpha := W_{U_{\alpha\beta}} \mathbf{P}_i(F_\alpha), \quad B_{i,\alpha\beta}^\beta := W_{U_{\alpha\beta}} \mathbf{P}_i(F_\beta).$$

After declaring a common overlap threshold $\tau_{\alpha\beta}$, set

$$O_{\alpha\beta,i}(F_\alpha) := T_{\tau_{\alpha\beta}}^{\text{bar}}(B_{i,\alpha\beta}^\alpha), \quad O_{\beta\alpha,i}(F_\beta) := T_{\tau_{\alpha\beta}}^{\text{bar}}(B_{i,\alpha\beta}^\beta).$$

Definition 5.24 (Overlap comparison data). The overlap data consist of the two direct pre-threshold profiles, the common threshold and threshold-reconciliation record, a pre-threshold bottleneck certificate, two-sided clearance certificates, post-threshold comparison data, representative provenance when invoked, typed Type IV status, and all artifact references.

Definition 5.25 (Threshold reconciliation record). If local records use thresholds τ_α and τ_β , an Overlap Gate is well typed only after a common threshold $\tau_{\alpha\beta}$ is declared and both local objects are directly re-evaluated on $U_{\alpha\beta}$ at that threshold. The data justifying this choice form the threshold-reconciliation record.

Remark 5.26 (Preferred overlap convention). The default comparison is performed directly on $U_{\alpha\beta}$. Cropping profiles already thresholded on U_α or U_β is forbidden unless the exact criterion of Definition 1.45 and Proposition 1.46 is certified.

Definition 5.27 (Two-sided overlap metric certificate). For each monitored degree i , a two-sided overlap metric certificate at budget $\varepsilon_{\alpha\beta,i}$ consists of

$$d_B(B_{i,\alpha\beta}^\alpha, B_{i,\alpha\beta}^\beta) \leq \varepsilon_{\alpha\beta,i},$$

$$\text{clear}_{\tau_{\alpha\beta}}(B_{i,\alpha\beta}^\alpha) > \varepsilon_{\alpha\beta,i}, \quad \text{clear}_{\tau_{\alpha\beta}}(B_{i,\alpha\beta}^\beta) > \varepsilon_{\alpha\beta,i},$$

together with the ledger provenance and direct common-window records.

Proposition 5.28 (Sharp overlap transfer). *Under a two-sided overlap metric certificate,*

$$d_B(O_{\alpha\beta,i}(F_\alpha), O_{\beta\alpha,i}(F_\beta)) \leq \varepsilon_{\alpha\beta,i}.$$

In the declared one-parameter constructible regime, the same bound may be read using interleaving distance.

Proof. The two clearance inequalities exclude finite bars from the closed band $[\tau_{\alpha\beta} - \varepsilon_{\alpha\beta,i}, \tau_{\alpha\beta} + \varepsilon_{\alpha\beta,i}]$ on both sides. Apply Appendix A, Theorem 1.33, to the certified pre-threshold bottleneck bound, then use the bottleneck–interleaving isometry in the declared regime. $\square$

Definition 5.29 (Overlap Gate). The pairwise Overlap Gate on $U_{\alpha\beta}$ passes exactly when, for every monitored degree:

- (i) a two-sided overlap metric certificate exists;
- (ii) the post-threshold discrepancy bound of Proposition 5.28 holds;
- (iii) representative and eligible-realization provenance used on the overlap is compatible;
- (iv) if Type IV is in scope, its status is `certified_zero`;
- (v) `undefined` is never interpreted as zero and `obstructed` is a failure;

(vi) all required crop, threshold-reconciliation, comparison, and artifact records are present.

Remark 5.30 (Overlap Gate is a compatibility test). The discrepancy verdict is post-threshold, but the safety certificate is pre-threshold. Small post-threshold distance alone does not prove stable threshold classification. Passing a pairwise gate also does not create a local pass or higher coherence.

Proposition 5.31 (Overlap Gate preserves pairwise audit readability). *If the pairwise Overlap Gate passes and all of its records are immutable and referenced, then the pairwise compatibility claim is independently reconstructible. No higher-simplex conclusion follows from this proposition.*

Proof. The gate records both pre-threshold inputs, the scalarized metric bound, the two clearance inequalities, the post-threshold verdict, typed statuses, and all required provenance. These data reconstruct precisely the pairwise claim and nothing stronger. $\square$

5.6 Finite Čech depth and simplex coherence

Definition 5.32 (Čech nerve). The Čech nerve $N(\mathcal{U})$ has a p -simplex for every nonempty intersection of $p + 1$ distinct cover windows.

Definition 5.33 (Finite Čech depth). The cover has *Čech depth at most K* if every intersection of $K + 2$ distinct windows is empty. Equivalently, $\dim N(\mathcal{U}) \leq K$. Thus nonempty records occur on at most $(K + 1)$ -fold intersections.

Definition 5.34 (Simplex compatibility record). For a nonempty simplex σ , a simplex compatibility record contains direct pre-threshold profiles on U_σ , a common threshold, all required face comparison data, sharp two-sided gap certificates, face-compatibility statements, typed Type IV statuses, scalarized budgets, and artifact references.

Proposition 5.35 (Finite overlap reduction). *If the cover has Čech depth at most K , no compatibility record is required above simplicial degree K .*

Proof. There are no nonempty intersections indexed by higher-dimensional simplices. This bounds the order of coherence conditions, but not the number of simplices when the cover is infinite. $\square$

Remark 5.36 (Why finite depth is a Core condition). Finite depth bounds overlap order only. It does not imply local finiteness, summability, face compatibility, or finiteness of the record set. Those are separate obligations.

Definition 5.37 (Compatible local certificate family). A family $\{C_\alpha\}$ is *Čech-compatible* if every local record passes, every nonempty nerve simplex carries a complete simplex compatibility record, all face restrictions agree under the declared equivalence policy, every scalarized simplex reserve is positive, and no required status is undefined.

Definition 5.38 (Čech-coherent certificate atlas). A *Čech-coherent certificate atlas* is a Čech-compatible family indexed by a declared nerve, with immutable local and simplex records and mutually compatible face restrictions.

Theorem 5.39 (Finite-cover record-level local-to-global principle). *Let $\mathcal{U}$ be a finite overlap-admitting cover of finite Čech depth. Assume:*

(i) *every window carries a passing local certificate record;*

- (ii) every nonempty nerve simplex carries a simplex compatibility record;
- (iii) every Type IV scope has status `not_invoked` or `certified_zero`, never `undefined`;
- (iv) every scalarized simplex budget has positive sharp threshold reserve;
- (v) all face restrictions and artifact references are audit-readable.

Then the family is a Čech-coherent certificate atlas on the covered range.

Proof. The local records provide the vertices. The simplex records supply every nonempty overlap through the maximal nerve degree, and the face conditions establish coherence under restriction. Finiteness of the cover and finite depth make the required registry finite. Positive reserves and typed Type IV statuses prevent unstable or undefined comparisons from being reported as coherent. $\square$

Remark 5.40 (No unrestricted object-level gluing). Theorem 5.39 glues records into an atlas. It does not glue filtered complexes, persistence modules, derived objects, sheaves, or other mathematical objects.

5.7 κ -admissible Restart

Definition 5.41 (Sequential continuation step). For adjacent partition windows W_k and W_{k+1} , a continuation step is a declared artifact linking the source and target local records. Adjacency itself supplies no inequality and no morphism.

Definition 5.42 (Per-window budget). Let $\mathfrak{B}_k$ be the finite pre-threshold profile family used by the local gate. Choose a finite working gap

$$0 < g_k \leq \text{gap}_{\tau_k}(\mathfrak{B}_k),$$

using a finite audit cap when the exact clearance is $+\infty$. Let $\varepsilon_k < +\infty$ be the gate-scoped scalarized local budget. These quantities must have immutable provenance.

Definition 5.43 (Restart margin). The finite local reserve is

$$r_k := g_k - \varepsilon_k \in \mathbb{R}.$$

The local record is reserve-positive exactly when $r_k > 0$. A finite working gap avoids undefined extended-real arithmetic in the Restart recursion.

Definition 5.44 (Continuation artifact). A continuation artifact from W_k to W_{k+1} records source and target identifiers and thresholds, all endpoint-state, initialization, representative, and comparison data used by the transition, a finite continuation loss $\eta_k \geq 0$ not already counted in the target local budget, a degradation factor $\kappa_k \in (0, 1]$, and independently checkable evidence for the reserve inequality below. Missing non-scalar obligations may not be encoded numerically inside η_k .

Definition 5.45 (κ_k -admissible continuation). The artifact is κ_k -admissible if it certifies

$$r_{k+1} \geq \kappa_k r_k - \eta_k.$$

Neither κ_k nor the inequality is inferred from adjacency, threshold deletion, or a generic “admissible continuation class.”

Definition 5.46 (Restart transition status). Each transition has exactly one status:

`not_invoked`, `undefined`, `certified`, `failed`.

The status is `certified` only when the artifact is complete and the declared κ_k -inequality is verified. A `certified` transition is necessary but not sufficient for a complete Restart pass: the target local record must also pass and have positive reserve.

Lemma 5.47 (Finite-horizon Restart bound). *Assume the transitions $0 \rightarrow 1, \dots, n-1 \rightarrow n$ are respectively $\kappa_0, \dots, \kappa_{n-1}$ -admissible. Then*

$$r_n \geq \left(\prod_{j=0}^{n-1} \kappa_j \right) r_0 - \sum_{m=0}^{n-1} \left(\prod_{j=m+1}^{n-1} \kappa_j \right) \eta_m,$$

where an empty product is 1.

Proof. For $n = 1$ this is the certified continuation inequality. If the formula holds at n , substitute it into $r_{n+1} \geq \kappa_n r_n - \eta_n$, distribute κ_n , and collect the product-weighted loss terms. $\square$

Remark 5.48 (Restart is conditional). There is no universal $\kappa > 0$, no default $\kappa = 1$, and no unconditional Restart lemma. Window restriction, threshold changes, and unrelated target profiles can destroy inherited reserve. A missing continuation artifact gives status `undefined`, not harmless continuation.

Definition 5.49 (Normalized Restart loss). Set

$$K_0 := 1, \quad K_n := \prod_{j=0}^{n-1} \kappa_j \quad (n \geq 1),$$

and

$$L_n^{\text{norm}} := \sum_{m=0}^{n-1} \frac{\eta_m}{K_{m+1}}.$$

Then Lemma 5.47 implies

$$\frac{r_n}{K_n} \geq r_0 - L_n^{\text{norm}}.$$

5.8 Countable Restart and normalized summability

Definition 5.50 (Summability of window budgets). A certified countable Restart run is κ -normalized summable if

$$\sum_{k=0}^{\infty} \frac{\eta_k}{K_{k+1}} < \infty.$$

It is *reserve-safe* if, in addition,

$$\sum_{k=0}^{\infty} \frac{\eta_k}{K_{k+1}} < r_0.$$

Ordinary summability $\sum_k \eta_k < \infty$ is not an equivalent condition when K_k contracts.

Definition 5.51 (Summable restart sequence). A MECE certificate sequence is a *reserve-safe Restart sequence* if the initial reserve is positive, every invoked transition is `certified`, the normalized loss is reserve-safe, every target has a complete passing local record including all invoked representative, eligibility, and Type IV clauses, and no transition or target is `undefined`.

Theorem 5.52 (Conditional countable Restart–Summability principle). *For a reserve-safe Restart sequence, every finite-stage reserve r_n is positive and the ordered family forms an auditable sequential certificate record on the partition cover. More precisely, if*

$$L_\infty^{\text{norm}} := \sum_{k=0}^{\infty} \frac{\eta_k}{K_{k+1}}, \quad c := r_0 - L_\infty^{\text{norm}} > 0,$$

then

$$r_n \geq K_n c > 0$$

for every finite n . A uniform lower bound follows only under the additional condition $\inf_n K_n > 0$.

Proof. For each finite n , Definition 5.49 gives

$$\frac{r_n}{K_n} \geq r_0 - L_n^{\text{norm}} \geq r_0 - L_\infty^{\text{norm}} = c.$$

Since $K_n > 0$, the pointwise bound follows. The continuation artifacts and complete target records make the record chain reconstructible. Taking infima yields a positive uniform reserve only when $\inf_n K_n > 0$. $\square$

Remark 5.53 (Why ordinary summability is not enough). Even when $\sum_k \eta_k < \infty$, the normalized series can diverge if $K_{k+1} \rightarrow 0$. Conversely, pointwise positivity $r_n > 0$ at every finite stage does not imply a uniform positive reserve. The theorem certifies record inheritance, not convergence to a global object.

5.9 Countable overlap atlases

Definition 5.54 (Definable window). A window is *definable* if it is definable in the declared ambient structure. Definability may support reconstructibility but does not replace local finiteness, simplex records, or coherence.

Definition 5.55 (Countable definable cover). A *countable definable overlap cover* is a countable overlap-admitting cover whose windows are definable, whose Čech depth is finite, and whose complete recorded nerve is included in the audit scope. To support a countable atlas theorem it must additionally be audit-locally finite.

Definition 5.56 (Audit-locally finite cover). A countable overlap cover is *audit-locally finite* if every bounded subinterval meets only finitely many windows and only finitely many recorded nerve simplices.

Definition 5.57 (Summable simplex budget). Let $\varepsilon_\sigma \geq 0$ be the scalarized metric budget of each recorded nonempty simplex. Define the unordered nonnegative sum

$$\sum_{\sigma \in N_{\text{rec}}} \varepsilon_\sigma := \sup_{S \subseteq N_{\text{rec}}, S \text{ finite}} \sum_{\sigma \in S} \varepsilon_\sigma.$$

The simplex budget is summable when this quantity is finite. This controls total declared exposure but does not create missing face maps or replace the simplexwise inequalities $\varepsilon_\sigma < \text{gap}_{\tau_\sigma}(\mathfrak{B}_\sigma)$.

Theorem 5.58 (Countable definable-cover gluing). *Let $\{U_n\}_{n \geq 1}$ be a countable, definable, audit-locally finite overlap cover of finite Čech depth. Assume:*

- (i) *every local window has a complete passing certificate record;*
- (ii) *every nonempty recorded nerve simplex has a complete simplex compatibility record;*

- (iii) every simplex has positive sharp threshold reserve;
- (iv) the scalarized simplex budgets are summable;
- (v) all invoked Type IV scopes are defined and correctly typed;
- (vi) all face restrictions and artifact references are audit-readable.

Then the family forms an audit-locally finite Čech-coherent certificate atlas on the covered range and has finite declared scalarized simplex exposure.

Proof. Audit-local finiteness makes every bounded audit query finite. Finite Čech depth bounds overlap order. The simplex records and face conditions establish coherence, positive reserves license each thresholded comparison, and the unordered sum records finite total scalarized exposure. Hence the atlas is locally reconstructible and globally indexed as a coherent record system. $\square$

Remark 5.59 (Definability is not decorative, but is not sufficient). Definability constrains the description language. It does not imply audit-local finiteness, summability, positive reserve, or higher coherence. All of those remain explicit theorem hypotheses.

5.10 Budget aggregation across windows and simplices

Definition 5.60 (Global window budget). For a declared sequence of scalarized local or continuation exposures $\varepsilon_k \geq 0$, define the global window exposure by the unordered or ordered nonnegative sum specified by the run. This quantity is not a quantale element unless the manifest separately records the corresponding $\oplus$ -aggregation.

Definition 5.61 (Global overlap budget). For a finite or countable recorded nerve, the global overlap exposure is the scalarized simplex sum of Definition 5.57. It includes every declared simplex, not merely pairwise overlaps.

Definition 5.62 (Total gluing budget). The *total gluing exposure* is a manifest-declared aggregation of window, continuation, crop, and simplex metric components after preserving their provenance. When represented by a quantale element it is aggregated using $\oplus$; when compared with real clearance it is first scalarized by val_B .

Proposition 5.63 (Budget auditability under summability). *Finite scalarized window exposure and a summable simplex budget make the declared total metric exposure finite and audit-readable. They do not establish any local or global gate pass without the corresponding profile-scoped strict reserve inequalities and non-scalar obligations.*

Proof. The nonnegative sums are finite by hypothesis, while the provenance-preserving aggregation and scalarization make the value reconstructible. Gate validity remains separate because a finite total may still exceed a local clearance or coexist with a missing artifact. $\square$

Remark 5.64 (Relation to gap_τ). There is no single untyped global inequality $\Sigma \delta_{\text{glue}} < \text{gap}_\tau$ unless the run declares a global profile family, threshold, aggregation law, and scalarization. The canonical requirements are the simplexwise and gatewise strict reserve inequalities actually consumed by the proof. A separately claimed uniform reserve requires an independent positive infimum certificate.

5.11 Compatibility with typed Type IV diagnostics

Definition 5.65 (Overlap Type IV cleanliness). A local, overlap, simplex, or continuation-target record is *Type IV-clean* when Type IV is in scope and its status is `certified_zero` relative to the declared comparison artifact and terminal-generic data. When Type IV is explicitly outside scope, `not_invoked` is permitted. Status `undefined` is never clean.

Proposition 5.66 (Overlap cleanliness is required for gluing). *No simplex may participate in a successful certificate atlas with Type IV status `obstructed` or `undefined`. If Type IV is in scope it must be `certified_zero`; otherwise the scope policy must justify `not_invoked`.*

Proof. This is the typed Type IV participation rule of Appendix D, Theorem 4.38, applied to every vertex, simplex, and continuation target. Compatibility cannot repair a rejected or undefined local clause. $\square$

Remark 5.67 (Type IV cleanliness is terminal-generic). On a bounded window, terminal-generic diagnostics count bars cofinal to the right endpoint, even though such bars may be globally finite. Therefore Type IV cleanliness is not accurately described as merely “absence of infinite-bar defect.” It also does not exclude interior noncofinal defects or prove full isomorphism.

5.12 Object-level gluing as a non-Core strengthening

Definition 5.68 (Object-level gluing claim). An *object-level gluing claim* asserts that local filtered complexes, persistence modules, derived objects, sheaves, or categorical objects descend to a global object in a declared category.

Proposition 5.69 (Certificate gluing does not imply object gluing). *A Čech-coherent certificate atlas does not, by itself, imply an object-level gluing claim.*

Proof. The atlas stores verdicts, profiles, budgets, statuses, face records, and artifact references. Object-level descent additionally requires morphisms or equivalences satisfying the descent axioms of a declared category, stack, or higher stack. Those hypotheses are not consequences of record coherence. $\square$

Remark 5.70 (Where object-level gluing belongs). Object-level descent belongs to a separately imported technical extension and must be reproved under the Core guard rails. Even then, descent does not replace the audit manifest.

5.13 Window, overlap, Restart, and atlas package

Theorem 5.71 (Appendix E conditional closure package). *Within the repaired Core, the following package is available.*

- (i) *Overlap profiles are evaluated directly on the common window before threshold deletion.*
- (ii) *Metric ledger data are compared with clearance only after bottleneck scalarization.*
- (iii) *A sharp two-sided overlap certificate transfers a pre-threshold bottleneck bound to the post-threshold evaluations.*
- (iv) *Pairwise Overlap Gates do not imply higher Čech coherence.*
- (v) *A finite-depth finite cover yields a coherent certificate atlas only when every simplex record and face condition is supplied.*

- (vi) *Restart is conditional on an explicit κ_k -admissible continuation artifact.*
- (vii) *Finite-horizon reserves satisfy the product-weighted inequality of Lemma 5.47.*
- (viii) *Countable Restart requires normalized reserve-safety; ordinary summability alone is insufficient.*
- (ix) *Pointwise positive reserves do not imply a uniform positive lower bound without control of $\inf_n K_n$.*
- (x) *A countable atlas additionally requires audit-local finiteness, finite Čech depth, simplex coherence, positive simplex reserves, and summable scalarized exposure.*
- (xi) *Undefined artifacts or statuses are never promoted to successful overlap, Restart, or gluing verdicts.*
- (xii) *Record-level gluing does not construct a global mathematical object or a single global $B\text{-Gate}^+$ pass.*

Proof. Items (i)–(iii) are Definitions 5.23–5.29 and Proposition 5.28. Items (iv)–(v) are Definitions 5.34–5.38 and Theorem 5.39. Items (vi)–(ix) are Definitions 5.44–5.51, Lemma 5.47, and Theorem 5.52. Item (x) is Theorem 5.58. Item (xi) follows from the typed status rules, and item (xii) is Proposition 5.69. $\square$

5.14 Explicit exclusions

The following are not asserted in Appendix E.

- (i) No unconditional Restart lemma or automatically existing $\kappa > 0$ is asserted.
- (ii) No missing continuation artifact is interpreted as $\kappa = 1$, zero loss, or a certified transition.
- (iii) No ordinary unweighted summability is sufficient under arbitrary contracting κ_k .
- (iv) No pointwise reserve statement is promoted to a uniform positive lower bound without an additional product lower bound.
- (v) No pairwise Overlap Gate implies triple or higher coherence.
- (vi) No differently thresholded outputs are compared without threshold reconciliation and direct common-threshold re-evaluation.
- (vii) No crop-after-collapse profile replaces direct common-window evaluation without a commutation certificate.
- (viii) No unscalarized quantale element is compared directly with real threshold clearance.
- (ix) No budget, gap, or summability estimate repairs a missing representative, morphism, coherence record, Type IV certificate, claim status, or manifest field.
- (x) No finite Čech depth alone implies a finite record set or audit-local finiteness.
- (xi) No definability alone implies coherence, summability, or local finiteness.
- (xii) No finite prefix is treated as a complete countable audit certificate.
- (xiii) No summable atlas is automatically a single global $B\text{-Gate}^+$ pass.
- (xiv) No certificate atlas is automatically a global filtered object, persistence module, sheaf, or derived object.

- (xv) No undefined local, overlap, Restart, representative, or Type IV status is treated as zero or pass.
- (xvi) No Search-side map, score, or AI proposal supplies a missing overlap, continuation, or gluing artifact.
- (xvii) No global exactness of positive-threshold deletion or global strict collapse functor is assumed.

5.15 Provenance and status

The sharp overlap-transfer theorem is inherited from Appendix A, Theorem 1.33. Representative and eligible-realization clauses are inherited from Appendices B–C. Typed terminal-generic Type IV participation is inherited from Appendix D, Theorem 4.38. The non-scalar obligation states are forwarded to Appendix F and their Manifest serialization to Appendix G. The distinctive contribution of Appendix E is the record architecture joining these inputs with simplex coherence, conditional κ -Restart, product-weighted reserve bounds, normalized countable summability, and audit-locally finite countable atlases.

Status labels for Appendix E. *Foundation definitions:* Definitions 5.2, 5.3, 5.5, 5.6, 5.8, 5.9, 5.11, 5.12, 5.13, 5.19, 5.20, 5.22, 5.23, 5.24, 5.29, 5.32, 5.33, 5.37, 5.41, 5.42, 5.43, 5.50, 5.51, 5.54, 5.55, 5.60, 5.61, 5.62, 5.65, 5.68.

Foundation lemmas/propositions/theorems: Propositions 5.31, 5.35, Theorem 5.39, Lemma 5.47, Theorems 5.52, 5.58, Propositions 5.63, 5.66, 5.69, and Theorem 5.71.

Operational cautions: Remarks 5.1, 5.4, 5.7, 5.10, 5.14, 5.21, 5.26, 5.40, 5.48, 5.53, 5.59, 5.67, 5.70, and 5.18.

Explicit exclusions: Subsection 5.14.

6 Appendix F: Typed Obligations, Status Algebra, Dependency Closure, and Certificate Readiness

6.1 Scope, dependency position, and design boundary

This appendix supplies the Foundation-level typed obligation calculus supporting Chapter 7. It consumes:

- (i) the representative lifecycle, representative obligation bundle, and manifest handoff of Appendix B, especially Definition 2.62;
- (ii) the eligible-realization, Ext-clause, and obligation bundle of Appendix C, especially Definition 3.54;
- (iii) the Type IV diagnostic record, four typed Type IV statuses, and obligation bundle of Appendix D, especially Definition 4.70;
- (iv) the scalar/non-scalar separation, overlap and simplex obligations, and κ -admissible Restart obligations of Appendix E, especially Definition 5.17.

Its output is a typed status-and-dependency semantics suitable for serialization by Appendix G. The central rule is:

A Core verdict is a dependency-closed conjunction of independently evidenced clauses. No numerical reserve, passing sibling clause, or complete-looking manifest may create missing evidence.

Remark 6.1 (Appendix F does not enlarge the mathematical Core). This appendix proves no new persistence, derived-category, tower, overlap, or Restart theorem. It governs the admissibility and aggregation of evidence already defined elsewhere. A status record, registry entry, or manifest field never replaces the theorem, proof, or verified artifact whose existence it records.

Remark 6.2 (Normative Foundation semantics). Most statements below are definitions, typing contracts, or elementary consequences of those contracts. They are normative for v19 audit objects but are not external-domain mathematical claims.

6.2 Immutable obligation atoms and registries

Definition 6.3 (Typed obligation atom). A *typed obligation atom* is a tuple

$$\mathfrak{o} := (\text{ObligationID}, \text{owner}, \text{scope}, \text{kind}, \\ \text{evidence policy}, \text{dependencies}, \text{failure routes}, \\ \text{manifest fields}).$$

Its fields identify:

- (i) the immutable clause identity;
- (ii) the chapter or appendix owning its mathematical meaning;
- (iii) the object, degree, window, threshold, tower, overlap, transition, or atlas scope;
- (iv) the class of evidence needed to evaluate it;
- (v) the prerequisite obligations;
- (vi) every broad Type I–V route that may apply on failure;
- (vii) the fields required for manifest serialization and replay.

Definition 6.4 (Obligation kinds). Each obligation atom has one primary kind:

- (i) *mathematical verdict*: for example repaired profile vanishing or eligible Ext vanishing;
- (ii) *existence/construction*: for example a representative, comparison morphism, or continuation artifact;
- (iii) *compatibility/coherence*: for example profile compatibility, edge realization, face compatibility, or composition coherence;
- (iv) *metric-bearing safety*: a clause whose evidence is a declared upper bound consumed by a metric gate;
- (v) *operational transition/computation*: for example terminal tameness, scalarization, or a Restart inequality;
- (vi) *governance/manifest*: claim status, repair history, artifact identity, scope justification, schema consistency, or replayability.

The primary kind does not prevent one failed atom from receiving several failure routes.

Definition 6.5 (Declared obligation registry). For an immutable audit run $\mathfrak{R}$, a *declared obligation registry* is a finite or countable family

$$\text{Obl}(\mathfrak{R}) = \{\mathfrak{o}_a\}_{a \in A}$$

with immutable identifiers, together with:

- (i) the required subset $\text{Req}(\mathfrak{R}) \subseteq \text{Obl}(\mathfrak{R})$;
- (ii) an explicit scope-exclusion record for every atom outside $\text{Req}(\mathfrak{R})$;
- (iii) the dependency edges and their provenance;
- (iv) evidence identifiers and evidence-policy versions;
- (v) the registered claim and repair-record versions governing every theorem-level use.

A silent change in any of these data defines a new run or revision.

Definition 6.6 (Evidence satisfaction predicate). Each obligation atom $\mathfrak{o}$ has a declared predicate

$$\text{Sat}_{\mathfrak{o}}(E_{\mathfrak{o}}) \in \{\text{true}, \text{false}\}$$

which is evaluable only when the evidence object $E_{\mathfrak{o}}$ is present, typed, version-compatible, and attached to the same immutable run scope. If those typing conditions fail, the atom is not assigned a mathematical truth value by this predicate.

Remark 6.7 (Registry completeness is not proof). A registry lists what must be checked. It does not prove the listed atoms. A complete registry with empty or invalid evidence remains incomplete as a proof object.

6.3 Scalarizable and non-scalarizable obligations

Definition 6.8 (Scalarizable obligation). An obligation is *scalarizable* only when its evidence policy is a declared metric upper bound that may enter the typed ledger and whose scalarization is proved to dominate the actual bottleneck discrepancy consumed by a specific gate.

Definition 6.9 (Non-scalarizable obligation). The following are non-scalarizable:

- (i) representative existence, profile compatibility, monitored-scope correctness, and equivalence strength;
- (ii) realization compatibility, amplitude, residual edge extraction, and edge-realization certificates;
- (iii) actual profile morphisms, source provenance, terminal tameness, defect profiles, and Type IV typing;
- (iv) threshold reconciliation, higher-face coherence, continuation artifacts, and κ -admissibility;
- (v) local-pass, target-pass, atlas-completeness, and countable-registry coverage obligations;
- (vi) claim status, repair records, artifact identity, manifest completeness, consistency, and replayability.

Proposition 6.10 (No scalar manufacture of non-scalar evidence). *If a required non-scalarizable obligation is absent, ill-typed, inconsistent, or attached to another run, then no ledger value, scalarized tolerance, threshold clearance, or positive reserve makes that obligation pass.*

Proof. A scalar metric record bounds only the declared metric discrepancy named by its evidence policy. It does not construct, type, or verify any of the non-scalar objects in Definition 6.9. $\square$

Remark 6.11 (Non-compensation begins at the typing boundary). A large reserve may absorb a larger certified metric discrepancy. It may not absorb missing semantics. This distinction is prior to any aggregate verdict rule.

6.4 Canonical atomic statuses

Definition 6.12 (Atomic audit status). Every obligation atom in the declared registry has exactly one canonical status

$$\text{AuditStatus} = \{\text{pass}, \text{reject}, \text{undefined}, \text{not_invoked}\}.$$

Their meanings are:

- (i) **pass**: the atom is required, well-typed, fully evidenced, and its satisfaction predicate is true;
- (ii) **reject**: the atom is required, well-typed, fully evidenced, and its satisfaction predicate is false or the declared obstruction is nonzero;
- (iii) **undefined**: the atom is required but incomplete, ill-typed, inconsistent, blocked by an undischarged dependency, or not computable from the declared evidence;
- (iv) **not_invoked**: the immutable scope positively declares that the atom is outside the claim.

Remark 6.13 (Missing is not not-invoked). A missing atom is not automatically outside scope. Without a valid scope-exclusion record it is undefined.

Definition 6.14 (Justified scope exclusion). A scope exclusion is *justified* when it identifies the excluded atom, the governing policy, the reason it is irrelevant to the declared claim, and the immutable run fields on which that exclusion depends. An exclusion inferred only after observing an inconvenient result is invalid.

Definition 6.15 (Effective aggregation normalization). For aggregation only, define

$$\text{Norm}(s, j) := \begin{cases} \text{pass}, & s = \text{not_invoked} \text{ and } j \text{ is a valid scope exclusion,} \\ \text{undefined}, & s = \text{not_invoked} \text{ and no valid exclusion exists,} \\ s, & s \in \{\text{pass}, \text{reject}, \text{undefined}\}. \end{cases}$$

The original **not_invoked** status remains stored; normalization does not erase the exclusion record.

6.5 Specialized status embeddings from Appendices B–E

Definition 6.16 (Representative lifecycle embedding). The representative lifecycle statuses of Definition 2.26 map as follows when a representative is required:

$$\begin{aligned} \text{verified} &\mapsto \text{pass}, & \text{failed} &\mapsto \text{reject}, \\ \text{constructed, missing} &\mapsto \text{undefined}. \end{aligned}$$

The status **not_required** maps to **not_invoked** only under a valid scope exclusion.

Definition 6.17 (Ext-clause embedding). The typed Ext-clause statuses of Definition 3.40 map identically to the canonical statuses with the same names.

Definition 6.18 (Type IV embedding). The Type IV statuses of Definition 4.35 map by

$$\text{certified_zero} \mapsto \text{pass}, \quad \text{obstructed} \mapsto \text{reject}, \quad \text{undefined} \mapsto \text{undefined}.$$

Type IV **not_invoked** remains canonical **not_invoked** only under an explicit no-tower scope decision.

Definition 6.19 (Restart transition embedding). The Restart transition statuses of Definition 5.46 embed at the level of the *continuation-law subclause*:

$$\text{certified} \mapsto \text{pass}, \quad \text{failed} \mapsto \text{reject}, \quad \text{undefined} \mapsto \text{undefined}.$$

This map does not assign the complete Restart bundle a passing status.

Definition 6.20 (Complete Restart bundle). A complete Restart step contains the following required atoms:

- (i) a certified continuation-law subclause;
- (ii) a defined positive target reserve;
- (iii) a passing target-local certificate record;
- (iv) every required endpoint, representative, threshold-reconciliation, scalarization, and comparison artifact;
- (v) no required undefined non-reserve obligation.

Its canonical status is the aggregate status of this bundle.

Remark 6.21 (Certified transition is necessary but not sufficient). A certified transition proves only

$$r_{k+1} \geq \kappa_k r_k - \eta_k.$$

The complete Restart step may still reject because the target reserve is nonpositive or the target-local record rejects, and it may remain undefined because another required artifact is missing.

Definition 6.22 (Manifest status embedding). Manifest statuses embed by

$$\text{complete} \mapsto \text{pass}, \quad \text{incomplete} \mapsto \text{undefined}, \quad \text{inconsistent} \mapsto \text{reject}.$$

A complete manifest supports, but does not itself prove, the mathematical atoms it records.

6.6 Claim-use preflight

Definition 6.23 (Registered claim-use status). Each invoked claim has exactly one registered status

$$\text{ClaimStatus} = \{\text{proved}, \text{conditional}, \text{operational}, \text{spec}, \text{deprecated}, \text{rejected}, \text{unknown}\}.$$

Their meanings are the canonical Chapter 7 meanings: theorem-level permission, conditional permission, procedural use only, target/specification, provenance-only legacy status, explicit mathematical rejection, or absence of authoritative registration.

Definition 6.24 (Claim-use atom status). For one intended use of a claim:

- (i) proved with matching hypotheses maps to pass;
- (ii) conditional maps to pass only when every registered condition is discharged, otherwise to undefined;
- (iii) operational maps to pass only for procedural use and to reject if used as theorem evidence;
- (iv) spec, deprecated, or rejected used as theorem evidence maps to reject;

- (v) unknown maps to undefined;
- (vi) a registered status attached to the wrong version, theorem location, or repair state maps to undefined or reject according to whether the mismatch is unresolved or evidenced inconsistent.

Definition 6.25 (Audit preflight bundle). The preflight bundle contains:

- (i) complete run identity and immutable scope;
- (ii) admissible claim-use atoms;
- (iii) every relevant v18-to-v19 repair record;
- (iv) required artifacts with matching types, versions, and hashes;
- (v) absence of inconsistent artifact resolution;
- (vi) the manifest status atom.

Its aggregate status is denoted $\text{Preflight}(\mathfrak{R})$.

Proposition 6.26 (Preflight failure blocks Core readiness). *If the preflight bundle is not pass, the complete Foundation readiness bundle cannot be pass.*

Proof. The preflight atoms are required dependencies of every theorem-level and manifest-level final verdict. Apply the dependency-closure and aggregate rules below. $\square$

6.7 Well-founded dependency registries

Definition 6.27 (Immediate dependency relation). For obligation atoms $\mathfrak{p}, \mathfrak{o}$, write

$$\mathfrak{p} \prec \mathfrak{o}$$

when $\mathfrak{p}$ is an immediate prerequisite of $\mathfrak{o}$. The transitive dependency closure is denoted

$$\text{Cl}(\mathfrak{o}) = \{\mathfrak{p} : \mathfrak{p} \prec^* \mathfrak{o}\}.$$

Only dependencies active under the immutable run scope enter the active closure.

Definition 6.28 (Well-founded dependency registry). A registry is *well-founded* when either:

- (i) it is finite and its dependency graph is acyclic; or
- (ii) it is countable and carries a rank function

$$\text{rk} : \text{Obl}(\mathfrak{R}) \longrightarrow \mathbb{N}$$

such that $\mathfrak{p} \prec \mathfrak{o}$ implies $\text{rk}(\mathfrak{p}) < \text{rk}(\mathfrak{o})$.

Self-dependence and cyclic mutual certification are forbidden.

Definition 6.29 (Dependency-complete pass). A required atom $\mathfrak{o}$ has a *dependency-complete pass* when:

- (i) its own evidence is present, typed, run-consistent, and satisfies its evidence predicate;
- (ii) every active required atom in $\text{Cl}(\mathfrak{o})$ has status pass;

- (iii) every inactive atom in the declared dependency template has a valid scope exclusion;
- (iv) all claim, artifact, and repair-record dependencies agree with the immutable run identity.

Proposition 6.30 (Pass is downward closed). *If $\mathfrak{o}$ has a dependency-complete pass, then every active required atom in $\text{Cl}(\mathfrak{o})$ has status *pass*.*

Proof. This is clause (ii) of Definition 6.29. □

Definition 6.31 (Blocked theorem-application atom). Suppose an atom records the application of a theorem or construction whose active prerequisite is not *pass*. The application atom is *blocked* and has canonical status *undefined*, even when the failed prerequisite has status *reject*. The rejected or undefined prerequisite remains separately recorded.

Remark 6.32 (Why blocked application is not automatically reject). Failure of a theorem hypothesis blocks that theorem application; it does not necessarily prove the intended conclusion false. The surrounding conjunction or bundle still rejects whenever the rejected prerequisite is itself a required atom.

Proposition 6.33 (Dependency blocker propagation). *If an active prerequisite is *undefined* or *reject*, then a dependent theorem-application atom cannot pass. It is *undefined* as an application atom, while the aggregate bundle preserves any rejected prerequisite.*

Proof. The dependency-complete pass criterion fails. Definition 6.31 types the application atom as *undefined*. Aggregate rejection, when applicable, follows from the rejected prerequisite itself. □

6.8 The status aggregation algebra

Definition 6.34 (Effective precedence order). After applying the normalization of Definition 6.15, use the total order

$$\text{pass} < \text{undefined} < \text{reject}.$$

For effective statuses s, t , define

$$s \otimes t := \max\{s, t\}$$

in this order.

Proposition 6.35 (Status-algebra laws). *The operation $\otimes$ is commutative, associative, and idempotent, with neutral element *pass*.*

Proof. These are the corresponding properties of the maximum operation on a total order. □

Definition 6.36 (Aggregate audit verdict). Write

$$A_{\text{req}}(\mathfrak{R}) := \{a \in A \mid \mathfrak{o}_a \in \text{Req}(\mathfrak{R})\}, \quad A_{\text{exc}}(\mathfrak{R}) := A \setminus A_{\text{req}}(\mathfrak{R}).$$

Let j_a be the scope-exclusion record, when any, for atom $\mathfrak{o}_a$. The aggregate verdict is

$$\text{Agg}(\mathfrak{R}) := \left(\bigotimes_{a \in A_{\text{req}}(\mathfrak{R})} \text{Norm}(s(\mathfrak{o}_a), j_a) \right) \otimes \left(\bigotimes_{a \in A_{\text{exc}}(\mathfrak{R})} \text{Norm}(s(\mathfrak{o}_a), j_a) \right),$$

provided the finite or countable registry satisfies the completeness conditions of this appendix. The two factors make the required/scope-excluded partition explicit; they do not permit an excluded atom to bypass the justified-scope condition of Definition 6.14. Equivalently:

- (i) reject if at least one atom in $\text{Req}(\mathfrak{R})$ rejects;
- (ii) undefined if none rejects but at least one atom in $\text{Req}(\mathfrak{R})$ is undefined, or one purported `not_invoked` atom outside $\text{Req}(\mathfrak{R})$ lacks valid scope justification;
- (iii) pass if every atom in $\text{Req}(\mathfrak{R})$ passes and every atom outside $\text{Req}(\mathfrak{R})$ has justified `not_invoked` status.

Theorem 6.37 (Non-compensation). *No number of passing atoms, zero diagnostics, positive reserves, or favorable scalar values cancels a rejected or required undefined atom in the aggregate verdict.*

Proof. The aggregate is the maximum in the effective precedence order. Passing atoms are neutral, while undefined and reject remain respectively above pass. Proposition 6.10 separately prevents numerical evidence from changing the type of a non-scalar atom. $\square$

Proposition 6.38 (Partition invariance of hierarchical aggregation). *Partition a complete registry into disjoint complete subregistries. Aggregate each subregistry and then aggregate the resulting statuses. The headline verdict equals the direct aggregate of all atoms, provided all rejected, undefined, and justified `not_invoked` identifiers remain attached to the intermediate records.*

Proof. Associativity and commutativity of $\otimes$ give equality of headline statuses. Retention of atom identifiers preserves the complete diagnostic detail. $\square$

Remark 6.39 (Headline compression does not erase detail). If one atom rejects and another is undefined, the headline is reject, but both atomic facts, dependency blockers, and failure routes remain in the record.

6.9 Failure routing and nonexclusive diagnostics

Definition 6.40 (Failure-routing set). Every non-pass run records all applicable routes

$$\text{Routes}(\mathfrak{R}) \subseteq \{\text{I, II, III, IV, V}\}.$$

Their meanings are:

- (i) Type I: nonzero repaired degree-one topological profile;
- (ii) Type II: nonzero categorical obstruction in a complete eligible realization regime;
- (iii) Type III: failed or undefined metric, crop, representative, realization, overlap, continuation, Restart, terminal-tameness, or other operational condition;
- (iv) Type IV: nonzero terminal-generic tower defect relative to a valid comparison artifact;
- (v) Type V: claim-status, scope, manifest, repair-record, artifact-identity, version, replayability, or proposer/verifier governance failure.

The routes are nonexclusive and are not a severity ranking.

Definition 6.41 (Route assignment discipline). Route assignment obeys the following distinctions:

- (i) a nonzero valid Type IV diagnostic is Type IV, not merely missing evidence;
- (ii) an absent Type IV artifact makes the Type IV atom undefined and is routed operationally as Type III and, when omitted or falsely serialized, also as Type V;

- (iii) missing eligibility is not Type II; it is Type III and possibly Type V;
- (iv) an inconsistent manifest is Type V and canonical `reject`;
- (v) an incomplete manifest is canonical `undefined`, unless another evidenced atom already rejects;
- (vi) reporting `undefined` as zero or pass creates an additional Type V route.

Proposition 6.42 (Atomic status and failure route are different types). *Two atoms may share a failure route while having different canonical statuses. In particular, a Type III operational clause may be evidenced `false` and `reject`, or `incomplete` and `undefined`.*

Proof. A route identifies the domain of failure. The atomic status identifies whether the clause was evidenced false or could not be validly evaluated. $\square$

6.10 Bundle readiness without a fifth verdict

Definition 6.43 (Bundle readiness). For a complete declared obligation bundle $\mathcal{B}$, its *certificate-readiness status* is its aggregate canonical status:

$$\text{Ready}(\mathcal{B}) := \text{Agg}(\mathcal{B}).$$

The words *ready*, *blocked*, and *rejected* may describe the cases `pass`, `undefined`, and `reject`, but they do not form a competing fifth status system.

Definition 6.44 (Representative readiness bundle). The representative readiness bundle is the obligation family of Definition 2.62 with the lifecycle embedding of Definition 6.16 and every active dependency required for the intended use.

Definition 6.45 (Eligible Ext readiness bundle). When the Ext clause is invoked, the Ext readiness bundle is the obligation family of Definition 3.54, including the repaired topological input, representative, realization, amplitude, residual-edge, edge-realization, field, theorem-use, and manifest atoms.

Definition 6.46 (Type IV readiness bundle). When tower semantics are invoked, the Type IV readiness bundle is the obligation family of Definition 4.70, including actual morphism, source provenance, terminal mode, terminal tameness, defect profiles, interior-defect typing, and the canonical Type IV status.

Definition 6.47 (Overlap and Restart readiness bundle). When overlap, atlas, or sequential inheritance is invoked, the corresponding readiness bundle contains the non-scalar obligations of Definition 5.17, all metric reserve atoms, all required simplex records, and every complete Restart bundle of Definition 6.20.

Definition 6.48 (Foundation readiness bundle). The complete Foundation readiness bundle contains, as applicable:

- (i) the audit identity, immutable scope, claim-use, repair, and manifest-preflight atoms;
- (ii) the gate-scoped metric ledger, scalarization, actual upper-bound theorem, gap, and reserve atoms;
- (iii) the representative readiness bundle;
- (iv) the eligible Ext readiness bundle;
- (v) the Type IV readiness bundle;

- (vi) all local, overlap, simplex, atlas, continuation, and complete Restart bundles;
- (vii) manifest completeness, consistency, replayability, aggregate verdict, and failure-routing atoms.

Theorem 6.49 (Certificate-readiness criterion). *A Foundation bundle has readiness status `pass` exactly when:*

- (i) *its registry is complete and well-founded;*
- (ii) *preflight passes;*
- (iii) *every required atom has a dependency-complete pass;*
- (iv) *every excluded atom has justified `not_invoked` status;*
- (v) *the manifest fragment is complete, consistent, and run-compatible;*
- (vi) *the aggregate verdict is `pass`.*

Proof. The forward direction follows from the meanings of `pass`, dependency-complete pass, and manifest completeness. The reverse direction follows from Definition 6.36 after all listed conditions are satisfied. $\square$

Remark 6.50 (Readiness is scope-relative). A passing readiness bundle certifies only the declared Core audit scope. It does not create a stronger claim, a global object, an object-level gluing theorem, or an external arithmetic, geometric, PDE, cryptographic, or computational conclusion.

6.11 Complete Restart and atlas composition rules

Proposition 6.51 (Complete Restart verdict). *For an invoked Restart step:*

- (i) *the complete bundle is `pass` exactly when the transition-law subclause passes, the target reserve is defined and positive, the target-local record passes, and all required non-reserve artifacts pass;*
- (ii) *it is `reject` if the certified inequality is false, the defined target reserve is nonpositive, or the target-local record rejects;*
- (iii) *it is `undefined` if no required item rejects but a required continuation, reserve, target, representative, comparison, or manifest atom is undefined;*
- (iv) *it is `not_invoked` only under an explicit scope exclusion of sequential inheritance.*

Proof. Apply the aggregate verdict to the complete Restart bundle of Definition 6.20. $\square$

Proposition 6.52 (Atlas composition rule). *A finite or countable certificate atlas has readiness status `pass` only when:*

- (i) *every required local vertex record passes;*
- (ii) *every required pairwise and higher-simplex record passes;*
- (iii) *the declared Čech-depth, audit-local-finiteness, threshold reconciliation, and simplex-budget atoms pass;*
- (iv) *every invoked Type IV and Restart bundle has passing status;*
- (v) *the countable registry, when any, has a complete coverage certificate;*

(vi) *the manifest is complete and consistent.*

A rejected required atom rejects the atlas. If none rejects but one required atom is undefined, the atlas is undefined.

Proof. Apply the aggregate and dependency-closure rules to the atlas obligation registry of Appendix E. $\square$

Remark 6.53 (Atlas readiness is not global-object readiness). A passing atlas bundle certifies record-level coherence. It does not construct a single global filtered object, persistence module, derived object, or global B-Gate⁺ record.

6.12 Countable registries and hidden-tail prevention

Definition 6.54 (Audit-readable countable registry). A countable obligation registry is *audit-readable* when it has:

- (i) a declared countable index set and immutable enumeration or generator;
- (ii) a coverage certificate proving that every obligation in scope occurs in the registry;
- (iii) a canonical status, evidence record, and scope decision for every index;
- (iv) a replayable dependency generator and well-founded rank;
- (v) a universal status certificate sufficient to verify that all required atoms pass or to identify all non-pass atoms;
- (vi) no hidden, unindexed, or implementation-dependent tail.

Proposition 6.55 (Finite-prefix checks do not certify countable readiness). *A finite prefix of a countable registry cannot support a pass verdict for the full registry unless a separate coverage proof establishes that the omitted tail contains no required atom or only atoms with valid scope exclusions.*

Proof. Without such a proof, an unchecked tail may contain a rejected, undefined, or unjustifiably excluded atom. The full aggregate verdict is therefore not determined by the finite prefix. $\square$

Theorem 6.56 (Countable aggregate soundness). *For an audit-readable countable registry, the aggregate verdict of Definition 6.36 is sound under the universal status certificate. In particular, a pass verdict requires universal dependency-complete pass over all required indices.*

Proof. The coverage certificate identifies the full registry, the rank proves dependency well-foundedness, and the universal status certificate supplies the required universal quantification. The aggregate cases then follow from Definition 6.36. $\square$

6.13 Repair calculus and immutable verdict history

Definition 6.57 (Repair record). A repair record contains:

- (i) the predecessor RunID or revision identifier;
- (ii) every repaired rejected or undefined atom;
- (iii) the changed evidence, dependency, scope, theorem, claim-status, or manifest fields;

- (iv) the old and new artifact hashes and schema versions;
- (v) the reason the repair is admissible;
- (vi) the resulting new atomic statuses, failure routes, and aggregate verdict.

Proposition 6.58 (No retroactive verdict mutation). *Resolving an undefined atom or correcting a rejected atom creates a new run or revision record. It does not rewrite the historical verdict of the immutable earlier run.*

Proof. The obligation registry, evidence identities, and repair state belong to the immutable audit scope. Changing them changes the audited object or its revision identity. $\square$

Remark 6.59 (Repairs need not be monotone in headline status). New evidence may resolve undefined to pass or reject; correction of an invalid rejection may also change the result. Immutability concerns historical records, not monotonicity of future revisions.

6.14 Appendix G serialization handoff

Definition 6.60 (Status-record serialization contract). For every obligation atom, the Appendix G schema must be able to serialize at least:

- (i) ManifestID, RunID, obligation ID, owner, and immutable scope;
- (ii) requiredness, scope-exclusion status, and scope-exclusion evidence;
- (iii) obligation kind and evidence-policy version;
- (iv) immediate dependencies, closure references, and rank data;
- (v) specialized source status and canonical atomic status;
- (vi) evidence, theorem, ClaimID, artifact, version, and hash references;
- (vii) scalar ledger, scalarization, actual upper-bound, gap, and reserve fields when applicable;
- (viii) every Type I–V failure route and every blocker identifier;
- (ix) predecessor and repair-record identifiers;
- (x) aggregate bundle verdict, registry-coverage status, consistency, and replayability fields.

Proposition 6.61 (Schema conformance preserves semantics). *A chapter-local or appendix-local record is admissible for the canonical manifest only if its serialized fields preserve the obligation identity, status meaning, dependency relation, failure routes, and immutable artifact references fixed by this appendix.*

Proof. Changing one of those meanings would serialize a different audit atom or run under the same identifier, contradicting immutable registry semantics. $\square$

Remark 6.62 (Semantics here, schema there). Appendix F fixes status meanings, dependency closure, aggregation, readiness, and failure routing. Appendix G fixes canonical field names, serialization constraints, conformance, and replayability. Schema completeness alone is not mathematical proof, and a mathematically true unrecorded clause does not produce an auditable pass until its required evidence is serialized consistently.

6.15 Illustrative audit patterns

Example 6.63 (Constructed but unverified representative). A representative candidate exists, but its profile isomorphisms are not verified. Its lifecycle status is `constructed`; its canonical status is `undefined`. A small bottleneck budget and a large threshold gap do not change that status.

Example 6.64 (Undefined Type IV is not zero). An actual comparison morphism or terminal-tameness certificate is absent. The Type IV atom is `undefined`, and no numerical μ, ν pair is asserted. Recording

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

would add a Type V manifest failure.

Example 6.65 (Certified transition with rejected target). A continuation artifact verifies the κ -inequality, so the transition-law subclause passes. The target-local record has a nonzero repaired topological profile and rejects. The complete Restart bundle therefore rejects.

Example 6.66 (Rejected prerequisite and blocked theorem application). Suppose a theorem application requires an eligible realization, but the amplitude certificate is well-typed and false. The amplitude atom rejects. The theorem-application atom is blocked and `undefined`; the full bundle rejects because it still contains the rejected amplitude atom.

Example 6.67 (Pairwise overlap without higher coherence). Every pairwise Overlap Gate passes, but a required triple-overlap simplex record is missing. The pairwise bundle passes, while the atlas bundle is `undefined`. No numerical overlap reserve manufactures the missing higher-face record.

Example 6.68 (Complete-looking but inconsistent manifest). All expected fields are populated, but two artifact hashes are declared for the same immutable identifier. The manifest is inconsistent and maps to `reject`; the run has a Type V route even if every numerical computation can be replayed separately.

6.16 Appendix F Foundation package

Theorem 6.69 (Appendix F typed-obligation and readiness package). *Within the declared v19 audit scope:*

- (i) *every obligation has an immutable typed identity and evidence policy;*
- (ii) *scalarizable and non-scalarizable obligations are separated;*
- (iii) *every atom has exactly one canonical audit status;*
- (iv) *specialized statuses of Appendices B–E embed without changing their mathematical meaning;*
- (v) *theorem-level use is controlled by claim-use preflight;*
- (vi) *dependency registries are well-founded and a pass is downward closed over active dependencies;*
- (vii) *blocked theorem applications remain distinct from rejected prerequisites;*
- (viii) *status aggregation is commutative, associative, idempotent, and non-compensatory;*
- (ix) *hierarchical aggregation preserves the headline verdict and all diagnostic detail;*
- (x) *failure routes preserve nonexclusive Type I–V classifications without collapsing `undefined` into `reject`;*

- (xi) *readiness is the aggregate canonical status of a complete dependency-closed bundle;*
- (xii) *complete Restart and atlas verdicts are conjunctions of their required subclauses;*
- (xiii) *countable readiness requires a coverage certificate and universal dependency-complete status;*
- (xiv) *repairs create new immutable verdict records;*
- (xv) *Appendix G serializes but does not alter these semantics.*

Proof. Items (i)–(ii) follow from Definitions 6.3, 6.8, and 6.9. Item (iii) is Definition 6.12. Item (iv) is Definitions 6.16–6.19. Item (v) follows from Definitions 6.23–6.25. Items (vi)–(vii) follow from Definitions 6.28, 6.29, and Proposition 6.33. Items (viii)–(ix) follow from Propositions 6.35 and 6.38, together with Theorem 6.37. Item (x) follows from Definition 6.40 and Proposition 6.42. Item (xi) is Theorem 6.49. Item (xii) follows from Propositions 6.51 and 6.52. Item (xiii) is Theorem 6.56. Item (xiv) is Proposition 6.58. Item (xv) follows from Definition 6.60 and Proposition 6.61. $\square$

6.17 Explicit exclusions

Appendix F does not assert:

- (i) that an obligation registry proves the atoms it lists;
- (ii) that a complete manifest proves the mathematical claims it records;
- (iii) that numerical slack repairs missing non-scalar evidence;
- (iv) that `undefined` means zero, false, harmless, or not invoked;
- (v) that `not_invoked` is valid without an explicit immutable scope decision;
- (vi) that one passing atom cancels a rejected or required undefined atom;
- (vii) that a rejected hypothesis proves every dependent conclusion false;
- (viii) that a certified Restart transition implies a complete Restart pass;
- (ix) that pairwise overlap passes imply higher Čech coherence;
- (x) that zero terminal-generic Type IV diagnostics imply full comparison-map isomorphism;
- (xi) that a finite prefix certifies a countable registry;
- (xii) that a repaired run retroactively changes an earlier immutable verdict;
- (xiii) that schema conformance replaces proof;
- (xiv) that Search, AI, ranking, confidence, or proposer output has verifier authority;
- (xv) that certificate readiness creates a global object or an external-domain theorem.

6.18 Provenance and status classification

This appendix integrates the representative obligations of Appendix B, the eligible Ext obligations of Appendix C, the Type IV obligations of Appendix D, and the overlap/Restart obligations of Appendix E under the canonical audit semantics of Chapter 7. It supplies the semantic input for the Appendix G manifest schema.

Foundation definitions. Definitions [6.3–6.6](#), [6.8](#), [6.9](#), [6.12](#), [6.14](#), [6.15](#), [6.16–6.22](#), [6.23–6.25](#), [6.27–6.31](#), [6.34](#), [6.36](#), [6.40](#), [6.41](#), [6.43–6.48](#), [6.54](#), [6.57](#), [6.60](#).

Foundation propositions and theorems. Propositions [6.10](#), [6.26](#), [6.30](#), [6.33](#), [6.35](#), [6.38](#), [6.42](#), [6.51](#), [6.52](#), [6.55](#), [6.58](#), [6.61](#), Theorems [6.37](#), [6.49](#), [6.56](#), [6.69](#).

Operational cautions. Remarks [6.1](#), [6.2](#), [6.7](#), [6.11](#), [6.13](#), [6.21](#), [6.32](#), [6.39](#), [6.50](#), [6.53](#), [6.59](#), [6.62](#).

Explicit exclusions. Subsection [6.17](#).

7 Appendix G: Canonical Manifest Schema, Proof Objects, and Replayability

7.1 Scope, dependency position, and canonical boundary

This appendix is the canonical manifest layer of the v19.0.0 Foundation Appendices. It consumes the mathematical and record-theoretic disciplines of Appendices A–E and the typed obligation semantics of Appendix F. Its purpose is not to invent a fifth verdict system, but to serialize without semantic loss the obligation atoms, dependency relations, evidence, statuses, failure routes, repair history, and aggregate readiness fixed by Appendix F.

Chapter 1 supplies the foundational Minimal Manifest requirement. Chapter 7 supplies the canonical typed audit semantics and Core sovereignty contract. Appendix F fixes the status meanings and dependency calculus. The present appendix fixes the canonical logical field schema, conformance rules, identity, immutability, and replayability requirements.

Appendix CR-A supplies claim-status governance, Appendix CM supplies Core micro-theorem references, and Appendix TB supplies toy-bridge references. Whenever such material is invoked, the manifest records

ClaimRef, CMRef, TBRef,

together with the exact permitted use and every discharged hypothesis.

Remark 7.1 (Appendix G is logically machine-readable, not format-prescriptive). This appendix fixes canonical logical fields, field types, conditional requiredness, identity constraints, and replayability conditions. It does not mandate one concrete syntax such as JSON, YAML, CSV, SQL, or a particular proof-store layout. A concrete encoding is admissible only if it preserves the logical schema bijectively enough for independent reconstruction.

Remark 7.2 (Appendix G does not enlarge the mathematical Core). This appendix creates no new persistence, derived, tower, Restart, overlap, or external-domain theorem. It governs admissible records. A complete schema is not a proof, and serialization cannot strengthen the status of a claim, micro-theorem, toy bridge, or proposed external interface.

Remark 7.3 (Relation to Appendix CR-A). A registered claim is usable only through the claim-use pre-flight semantics of Appendix F. Registration is not proof. The manifest records the claim identifier, snapshot version, registered status, permitted use, discharged conditions, and non-claim boundary.

Remark 7.4 (Relation to Appendix CM). A Core micro-theorem reference records its identifier, statement version, hypotheses, hypothesis evidence, and exact invocation site. Citation does not discharge hypotheses and does not turn an internal Core lemma into an external-domain theorem.

Remark 7.5 (Relation to Appendix TB). A toy-bridge reference records the toy source category, realization route, repaired profiles, actual comparison artifact, diagnostic conclusion, and explicit toy-scope boundary. No toy theorem is promoted to a problem-specific bridge theorem without a separately proved transport theorem.

7.2 Semantic authority: Appendix F first, Appendix G second

Definition 7.6 (Semantic-preserving serialization). A serialization of an Appendix F obligation registry is *semantic-preserving* if it preserves:

- (i) every obligation identity and immutable scope;
- (ii) requiredness and justified scope exclusion;
- (iii) the dependency relation, closure references, and well-founded rank data;
- (iv) specialized source status and canonical atomic status;
- (v) evidence-policy version, artifact identity, and theorem/claim references;
- (vi) scalar ledger data only for scalarizable obligations;
- (vii) all Type I–V failure routes and blocker identifiers;
- (viii) predecessor, revision, and repair-record links;
- (ix) registry coverage, consistency, replayability, and aggregate verdict.

Proposition 7.7 (Appendix F semantics are authoritative). *No Appendix G field, default, omission rule, encoding convention, or implementation shortcut may alter the status meanings, dependency closure, non-compensation rule, readiness criterion, or failure routing fixed by Appendix F, in particular Definition 6.60 and Proposition 6.61.*

Proof. An encoding that changes one of those meanings represents a different obligation registry or verdict while reusing the same identifiers. That violates immutable run semantics and schema conformance. $\square$

Remark 7.8 (Schema completeness is not proof). A schema may be complete and replayable while recording a rejected run. Conversely, a mathematically true but unserialized clause does not yield an auditable pass. Appendix F fixes the semantics; Appendix G fixes how those semantics are exposed for audit.

7.3 Audit identity, schema identity, and immutable scope

Definition 7.9 (Audit run identity). An audit run identity is the tuple

$$\mathfrak{J} = (\text{RunID}, \text{ManifestID}, \text{SchemaID}, \text{Version}_{\text{theory}}, \text{Version}_{\text{claim}}, \text{Version}_{\text{artifact}}, \text{RevisionID}).$$

It identifies the exact theory release, canonical schema release, claim-register snapshot, artifact namespace, and revision under which all fields are interpreted.

Definition 7.10 (Immutable audit scope). The immutable scope of a run records at minimum:

- (i) the object or object family under audit;
- (ii) monitored degrees, windows, thresholds, and terminal modes;

- (iii) invoked clauses and justified non-invoked clauses;
- (iv) collapse policy and evaluation order;
- (v) claim, theorem, micro-theorem, and toy-bridge identifiers used;
- (vi) immutable identifiers of all source, profile, representative, realization, comparison, diagnostic, continuation, and atlas artifacts.

Changing any item creates a new run or revision; it does not silently mutate the old verdict.

Definition 7.11 (Schema version boundary). A schema version boundary is a declared change in field names, field types, requiredness rules, normalization, or conformance semantics. A record from another schema version is admissible only through an explicit, auditable migration map that preserves Appendix F semantics.

Proposition 7.12 (Identity collision is manifest inconsistency). *If two records use the same RunID or ManifestID but assign different immutable scope, theory version, schema version, artifact identity, or revision semantics, then the manifest is inconsistent and routes to Type V rejection.*

Proof. One immutable identifier cannot denote two distinct runs under the canonical single-run semantics. The conflict is therefore an internal governance contradiction rather than mere missing data. $\square$

7.4 Proof objects and certificate records

Definition 7.13 (Proof object). A *proof object* is an abstract mathematical-and-audit entity intended to support a declared claim. It contains or references the mathematical evidence, typed obligation registry, dependencies, ledger records, diagnostics, artifacts, claim-use preflight, and verdict data required by the applicable Core clauses.

Definition 7.14 (Certified proof object). A proof object is *certified* for a declared scope only if:

- (i) its Appendix F foundation readiness bundle has aggregate status `pass`;
- (ii) its manifest is complete, consistent, conformant, and replayable;
- (iii) every required obligation in the dependency closure has canonical status `pass`;
- (iv) every justified `not_invoked` clause has a valid scope-exclusion record;
- (v) no hidden registry tail or unresolved blocker remains.

Manifest completeness alone does not certify a proof object.

Definition 7.15 (Certificate record). A *certificate record* is the canonical manifest-level representation of one proof object or one scope-relative proof-object fragment. It serializes evidence and verdict semantics but is not identical to the underlying mathematical object.

Remark 7.16 (Proof object vs. certificate record). The proof object is the evidenced entity; the certificate record is its audit-readable representation. A record may be complete and record a rejection. A record may also be incomplete even when an unrecorded mathematical argument happens to be true.

Definition 7.17 (Local certificate record). A *local certificate record* is attached to one declared window or overlap stratum and contains the local obligation subregistry, repaired evaluation data, local ledger, typed diagnostics, local verdicts, dependencies, and artifact references.

Definition 7.18 (Global certificate record). A *global certificate record* is a record assembled from local records through a declared record-level Čech atlas, Restart sequence, finite union, or other proved certificate-record composition rule. It retains all local identities, simplex records, transition records, budgets, routes, and dependencies used in the assembly.

Remark 7.19 (Global record is not object-level gluing). A globally coherent certificate atlas does not construct a global filtered complex, persistence module, derived object, sheaf, or external-domain solution. Object-level descent requires separate mathematical hypotheses.

7.5 Canonical manifest object and field cardinalities

Definition 7.20 (Canonical manifest object). A canonical manifest is the typed record

$$\text{Manifest} = (\text{Identity}, \text{Scope}, \text{Objects}, \text{Windows}, \text{Policies}, \text{Registry}, \text{Evidence}, \text{Artifacts}, \\ \text{Ledger}, \text{DomainRecords}, \text{References}, \text{Verdicts}, \text{Routes}, \text{Repairs}, \text{Replay}).$$

Each component is itself typed and versioned. Conditional components are absent only through a justified scope-exclusion record; silent omission is not permitted.

Definition 7.21 (Canonical cardinality markers). For schema statements, the markers

$$[1], \quad [0..1], \quad [0..*], \quad [1..*]$$

mean respectively exactly one, optional singleton, finite or countable family possibly empty, and nonempty finite or countable family. Countable families require a coverage certificate and replayable generator.

Definition 7.22 (Conditional requiredness). A field is *conditionally required* when an invocation predicate is true. The record stores both the predicate and its evidence. If the predicate is false, the field is absent only together with a justified `not_invoked` scope-exclusion record. If the predicate is true and the field is missing, the relevant obligation is undefined.

Proposition 7.23 (No silent default theorem). *No omitted representative, comparison morphism, eligibility certificate, continuation artifact, claim status, ledger upper-bound theorem, or coherence record may be reconstructed from a default value.*

Proof. Each listed item is a non-scalarizable proof obligation. A default would manufacture evidence and violate Appendix F non-compensation. $\square$

7.6 Minimal Manifest Axiom

Axiom 7.1 (Minimal Manifest Axiom). *A proof object may support an auditable Core verdict only if its manifest contains, at minimum:*

- (i) *audit, manifest, schema, theory, claim-register, artifact-root, and revision identities;*
- (ii) *immutable scope, declared objects, windows, thresholds, policies, and monitored degrees;*
- (iii) *the full required obligation registry, dependency relation, well-founded rank, and coverage certificate;*
- (iv) *every specialized source status and canonical atomic status;*
- (v) *evidence-policy versions and immutable evidence/artifact references;*

- (vi) *scalar ledger, scalarization, actual metric upper-bound, gap, and reserve records where applicable;*
- (vii) *representative, realization, eligibility, Type IV, overlap, Restart, and atlas records when invoked;*
- (viii) *claim-use, micro-theorem, and toy-bridge preflight records when invoked;*
- (ix) *every blocker and every applicable Type I–V failure route;*
- (x) *aggregate bundle verdict, manifest status, readiness status, consistency, and replayability fields;*
- (xi) *predecessor and repair-record identifiers for revised runs.*

Remark 7.24 (Meaning of minimality). Minimality means necessary for the declared Core scope, not small in byte size. Concrete systems may add runtime logs, timestamps, proof-assistant traces, signatures, or redundant indexes, but may not omit a conditionally required field.

Proposition 7.25 (Necessity of manifest data). *If a required field of Axiom 7.1 is absent or not reconstructible, then the manifest is incomplete and the corresponding audit clause is undefined, unless another evidenced clause already forces aggregate rejection.*

Proof. The missing field prevents reconstruction of at least one required obligation or its evidence. Appendix F maps missing required evidence to undefined; the aggregate precedence rule may still report reject if a separate clause is evidenced false, while retaining the undefined detail. $\square$

7.7 Obligation registry serialization and dependency closure

Definition 7.26 (Obligation atom record). For every obligation atom, the manifest stores exactly one primary atom record containing:

- (i) ObligationID, owner, immutable scope, and obligation kind;
- (ii) requiredness and scope-exclusion state;
- (iii) evidence-policy identifier and version;
- (iv) immediate dependency identifiers and closure reference;
- (v) well-founded rank or finite acyclic-graph certificate;
- (vi) specialized source status and canonical atomic status;
- (vii) evidence references, blockers, routes, predecessor, and repair links.

Definition 7.27 (Dependency record). A dependency record is a directed edge

$$\text{ObligationID}(\mathfrak{p}) \longrightarrow \text{ObligationID}(\mathfrak{o})$$

meaning that $\mathfrak{p}$ is an immediate prerequisite of $\mathfrak{o}$. The manifest stores the edge source, target, dependency kind, evidence for the dependency, and rank check.

Definition 7.28 (Registry coverage certificate). A registry coverage certificate proves that the serialized atom family is exactly the declared finite registry, or for a countable registry, that an immutable generator enumerates every required atom without hidden tail. It includes the generator version, index type, rank rule, and universal status certificate.

Definition 7.29 (Closure reference). A closure reference identifies a reproducible computation or certificate for the transitive dependency closure of an atom or bundle. It does not replace the immediate dependency edges.

Proposition 7.30 (Finite prefix cannot certify a countable registry). *A manifest containing only a finite prefix of a countable obligation registry cannot support a pass for the full registry without a valid coverage certificate and universal status certificate.*

Proof. An unchecked tail may contain a required undefined or rejected atom. Finite-prefix success therefore does not determine the countable aggregate. $\square$

7.8 Canonical statuses, classifications, and aggregate verdicts

Definition 7.31 (Canonical status field). Every obligation atom has exactly one canonical atomic status

$$\text{Status}(o) \in \{\text{pass}, \text{reject}, \text{undefined}, \text{not_invoked}\},$$

with the meaning fixed by Appendix F. The field also stores the specialized source status, normalization rule, evidence reference, timestamp or sequence index if used, and responsible verifier version.

Definition 7.32 (Claim/use classification field). The claim/use classification is a separate field whose values may include

proved, conditional, operational, spec, deprecated, rejected, unknown,

or declared layer tags such as Core, Interface, Bridge Candidate, Bridge Program, Toy Bridge, Search Artifact, Companion Template, and Non-Claim. These are not atomic audit statuses.

Definition 7.33 (Aggregate verdict record). An aggregate verdict record stores:

- (i) the required atom set and justified scope-excluded set;
- (ii) the normalized status of every included atom;
- (iii) the aggregation rule and Appendix F version;
- (iv) the headline aggregate $\text{Agg}(\mathfrak{R})$;
- (v) all retained lower-level statuses, blockers, and routes.

The headline does not erase an undefined atom merely because another atom rejects.

Definition 7.34 (Manifest status field). The manifest itself has one of the source statuses

complete, incomplete, inconsistent.

Under the Appendix F embedding, these map respectively to

pass, undefined, reject.

This status is one obligation atom inside the full readiness bundle, not a replacement for the bundle verdict.

Proposition 7.35 (Status/classification separation). *A strong claim classification cannot compensate for an undefined or rejected audit status, and an audit pass cannot promote a claim beyond its registered classification.*

Proof. The fields have different types and govern different questions. Appendix F non-compensation and claim-use preflight prohibit cross-type substitution. $\square$

7.9 Declared objects and windows

Definition 7.36 (Declared object data). Declared object data include object identity, ambient type, source provenance, monitored degrees, coefficient field, constructibility or finiteness conditions, and immutable source artifacts. For an interface run, both the external object and its projected Core object are recorded together with the interface version.

Definition 7.37 (Declared window data). Declared window data include window identifiers, endpoints, end-point convention, finite or right-unbounded terminal mode, MECE or overlap-cover role, cover/partition identifier, and all nonempty recorded overlap strata.

Definition 7.38 (Window identifier). A window identifier is immutable within one run and links all evaluations, representatives, diagnostics, ledgers, statuses, simplex records, continuation records, and artifacts for that window.

Remark 7.39 (Window declarations and Appendix E). The MECE/overlap distinction, simplex coherence, Restart, and record-level gluing semantics are inherited from Appendix E. The historical label name of this remark is retained, but the governing main-body chapter is Chapter 6 in v19.

Proposition 7.40 (Window data are audit-critical). *Without declared window and terminal-mode data, the repaired evaluation, threshold clearance, terminal-generic Type IV diagnostic, overlap comparison, and Restart scope are not determined.*

Proof. Each listed construction is window-relative. Omitting the window therefore leaves required mathematical inputs undefined. $\square$

7.10 Threshold, collapse, and repaired evaluation records

Definition 7.41 (Threshold policy). A threshold policy records the threshold value or indexed family, fixed-threshold or sweep mode, common-threshold reconciliation rules, stable-band claims, window dependence, and the threshold-policy version.

Definition 7.42 (Collapse policy). A collapse policy records the permitted level of use, including

`barcode_level,      representative_verified,      tau_split,      exact_admissible,`

together with every artifact and hypothesis required by the chosen level. The default is barcode-level hard deletion after window restriction.

Definition 7.43 (Fixed-threshold certificate). A fixed-threshold record evaluates all declared clauses at one immutable $\tau > 0$, with gate-scoped profile families and clearance records.

Definition 7.44 (Threshold-sweep certificate). A threshold-sweep record stores the threshold index set or band, artifact-coherent construction policy, pointwise records, bandwise quantifiers, and any claimed uniform reserve proof. Pointwise positivity is not serialized as a uniform positive infimum without separate evidence.

Remark 7.45 (Threshold policy and repaired collapse). The threshold policy fixes barcode-level deletion semantics. It does not silently provide global exactness, Serre localization, or a strict filtered collapse functor.

Proposition 7.46 (Threshold and collapse data are audit-critical). *Without threshold, policy version, evaluation order, and collapse status, the repaired profile and the set of licensed downstream operations are not reconstructible.*

Proof. Different thresholds and collapse statuses produce different profiles and different admissible theorem uses. $\square$

Definition 7.47 (Evaluation order record). An evaluation order record declares

$$\mathbf{P}_i(F) \mapsto W_W \mathbf{P}_i(F) \mapsto T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)) = \text{Eval}_{i,W,\tau}(F).$$

It stores the operator versions and every conditional commutation certificate if another route is used.

Definition 7.48 (Repaired evaluation record). For each monitored triple (i, W, τ) , a repaired evaluation record stores the pre-threshold profile, barcode or reconstructive profile artifact, thresholded profile, evaluation status, clearance data, and immutable artifact references.

Definition 7.49 (Overlap evaluation record). An overlap evaluation record is computed directly on the common overlap stratum at the reconciled threshold. It includes both local-source identities, the common-window pre-threshold profiles, discrepancy certificate, clearance certificates, and post-threshold comparison.

Remark 7.50 (No hidden crop-collapse commutation). A restriction of an already-collapsed larger-window profile cannot replace direct common-window evaluation without the window-adapted condition of Definition 1.45 and the conditional criterion of Proposition 1.46, or another explicitly proved exact-admissible route.

Proposition 7.51 (Evaluation records are audit-critical). *A record without reconstructible repaired evaluations cannot determine the topological gate atom.*

Proof. The atom is the typed assertion $\text{Eval}_{1,W,\tau}(F) = 0$ or its evidenced negation. Without the profile and artifacts, neither outcome is auditable. $\square$

7.11 Representative, realization, and eligibility records

Definition 7.52 (Representative policy). A representative policy records whether a filtered representative is invoked, its lifecycle status, canonical atomic status, monitored degrees, equivalence policy, representative theorem/artifact, and realization compatibility when required.

Definition 7.53 (Representative record). A representative record serializes $C_{\tau,W}F$, the relation

$$\mathbf{P}_i(C_{\tau,W}F) \cong \text{Eval}_{i,W,\tau}(F),$$

its proof artifact, construction provenance, equivalence class, status, blockers, and repair links.

Definition 7.54 (Eligibility record). When the Ext^1 -clause is invoked, the eligibility record stores the representative, realization functor or procedure, realized object, amplitude certificate, typed residual-edge extractor, edge-realization isomorphism, zero-compatibility, and the canonical clause status.

Remark 7.55 (No representative, no derived clause). A missing representative or eligibility artifact makes the derived theorem-application atom undefined. It is not repaired by topological vanishing, numerical slack, or a claimed Ext value.

Proposition 7.56 (Representative and eligibility records are audit-critical when invoked). *If a run invokes filtered or derived clauses, omission of the applicable representative or eligibility records prevents a dependency-complete pass.*

Proof. Those records are immediate prerequisites in the obligation registry. Appendix F dependency completeness therefore blocks the downstream application. $\square$

7.12 Typed ledger and non-scalar obligations

Definition 7.57 (Ledger policy). A ledger policy declares an ordered commutative monoid or, when fully justified, commutative quantale; each metric-bearing component; aggregation; bottleneck scalarization; actual upper-bound theorem; gate-scoped pre-threshold profile family; clearance; scalarized budget; and reserve.

Definition 7.58 (Budget component). A budget component is a scalarizable metric discrepancy such as algorithmic, discretization, measurement, representative-choice, realization, comparison, crop, continuation, overlap, or Restart discrepancy. Each component stores semantic type, scope, provenance, value, and upper-bound evidence.

Definition 7.59 (Non-scalar obligation record). A non-scalar obligation record covers representative existence, eligibility, actual morphisms, terminal tameness, claim status, threshold reconciliation, simplex coherence, continuation and κ -admissibility, target-local pass, manifest identity, and comparable proof obligations. Such a record has a typed status and evidence; it is not a numerical budget component.

Definition 7.60 (Total budget record). A total budget record stores the typed metric ledger element, component decomposition, scalarization map, scalarized value, and theorem proving that the value dominates the actual gate discrepancy.

Definition 7.61 (Safety margin record). A safety margin record stores the gate-scoped pre-threshold family, each clearance, the minimum gap, scalarized metric budget, reserve, and strict comparison result.

Remark 7.62 (Strict inequality). The boundary equality between scalarized discrepancy and gap is not a pass. The manifest stores the exact comparison operator and does not round equality into positivity.

Proposition 7.63 (Ledger data are audit-critical). *A numerical gate cannot pass from a ledger value alone; it also requires typed component provenance, scalarization, an actual metric upper-bound theorem, and a positive gate-scoped reserve.*

Proof. A number without a theorem connecting it to the consumed discrepancy is not evidence. The reserve must be computed from the relevant pre-threshold profiles, not from an unrelated global scalar. $\square$

7.13 Type IV diagnostics and comparison artifacts

Definition 7.64 (Diagnostic policy). A diagnostic policy records monitored degrees, windows, thresholds, terminal modes, source-profile provenance, actual profile-system morphisms, source-to-apex comparison morphisms, terminal-tameness certificates, kernel and cokernel defect profiles, and the canonical μ/ν diagnostics.

Definition 7.65 (Tower comparison artifact record). A tower comparison artifact record serializes

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty),$$

including actual source and target objects, morphism identity, construction provenance, terminal-tameness, defect artifacts, and immutable hashes or equivalent integrity identifiers.

Definition 7.66 ($(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ record). A $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ record is permitted only when the Type IV specialized status is `certified_zero` and the canonical atomic status is `pass`. The numeric pair alone is insufficient without a valid comparison morphism and terminal-tameness evidence.

Definition 7.67 ($(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0)$ record). A $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0)$ record stores specialized status `obstructed`, atomic status `reject`, the nonzero degreewise diagnostics, defect artifacts, and Type IV failure route.

Definition 7.68 (Undefined Type IV record). If Type IV is invoked but the comparison morphism, source provenance, terminal-tameness, or defect artifacts are missing, the record stores specialized status `undefined` and atomic status `undefined`; it must not serialize $(0, 0)$ as a substitute.

Remark 7.69 (Terminal-generic Type IV meaning). The diagnostic semantics follow Appendix D. On bounded windows, cofinal bars reaching the right endpoint are counted even though they are globally finite. Therefore “infinite-bar only” is not the canonical bounded-window meaning. Zero terminal diagnostics do not imply full comparison isomorphism without the no-interior-defect condition.

Proposition 7.70 (Diagnostic and comparison-artifact data are audit-critical). *A Type IV atom is not well formed without the actual comparison artifact and the terminal-generic diagnostic prerequisites of Appendix D.*

Proof. Kernel, cokernel, and terminal-generic dimensions are defined relative to that typed morphism and regime. Without them, the clause is undefined rather than zero. $\square$

7.14 Overlap, Čech, Restart, and atlas records

Definition 7.71 (Simplex compatibility record). For every recorded nonempty Čech simplex, the manifest stores the simplex identifier, common overlap window, face identifiers, common-threshold reconciliation, pre-threshold profiles, discrepancy and sharp clearance certificates, face compatibility, typed Type IV status, non-scalar coherence obligations, and artifact links.

Definition 7.72 (Restart transition record). A Restart transition record stores source and target windows, source reserve, κ_k , η_k , the certified recurrence inequality, continuation artifact, transition substatus, target reserve, target-local status, and complete Restart verdict.

Definition 7.73 (Countable Restart record). A countable Restart record additionally stores K_n , normalized losses, the reserve-safe series certificate, pointwise reserve conclusions, and any separately proved uniform lower bound. Ordinary unnormalized summability is not recorded as sufficient evidence.

Definition 7.74 (Atlas record). An atlas record stores the cover identity, finite Čech depth, audit-local-finiteness certificate when countable, every required simplex record, face maps, coherence status, ledger exposure, coverage certificate, and the scope-limited conclusion “certificate atlas”.

Proposition 7.75 (Pairwise records do not imply higher coherence). *A manifest containing all pairwise overlap passes but missing a required higher-simplex record cannot support an atlas pass.*

Proof. Higher coherence is a distinct non-scalar obligation in Appendix E. It cannot be inferred from pairwise data or numerical slack. $\square$

7.15 Gate and bundle verdict records

Definition 7.76 (Gate verdict). A gate verdict is a typed bundle record, not merely a Boolean. It stores the gate scope, required atom set, justified non-invoked set, atomic statuses, aggregate status, blockers, reserves, failure routes, and dependency closure used to compute the headline verdict.

Definition 7.77 (B-Gate⁺ verdict record). A B-Gate⁺ verdict record stores distinct atoms for repaired topological vanishing, eligible Ext use when invoked, Type IV status, metric reserve, claim-use preflight, and manifest conformance. The aggregate is computed by Appendix F; no passing atom cancels an undefined or rejected atom.

Definition 7.78 (Overlap Gate verdict record). An Overlap Gate verdict record stores the common-window evaluation and sharp two-sided transfer atoms, threshold reconciliation, metric reserve, Type IV status, actual artifacts, and simplex/face obligations.

Definition 7.79 (Foundation readiness record). A foundation readiness record serializes the Appendix F readiness bundle and its aggregate status. Readiness is not a fifth atomic status; it is the aggregate status of a declared required bundle.

Remark 7.80 (Verdicts are not scores). A confidence, ranking, probability, heuristic score, or AI assessment may appear only as proposer provenance. It is never a canonical atomic status or gate verdict.

Proposition 7.81 (Gate verdicts are audit-critical). *A record that stores only a headline verdict without the atomic statuses, dependency closure, blockers, and failure routes is not replayable and cannot support a canonical Core pass.*

Proof. The headline is a compression of typed detail. Without the detail, the verifier cannot reconstruct the aggregation or distinguish rejection from undefinedness. □

7.16 Claim, micro-theorem, and toy-bridge references

Definition 7.82 (Claim-register reference). A claim-register reference stores ClaimID, claim-register snapshot, stable statement pointer, registered status, layer, permitted use, discharged conditions, forbidden use, exact invocation site, and artifacts.

Definition 7.83 (Micro-theorem reference). A micro-theorem reference stores CMID, statement version, hypotheses, evidence for each hypothesis, exact invocation site, conclusion consumed, and non-claim boundary.

Definition 7.84 (Toy-bridge reference). A toy-bridge reference stores TBID, toy source category, source object/tower, realization route, repaired profiles, actual comparison artifact, diagnostics, ledger assumptions, exact invocation site, and toy-scope boundary.

Definition 7.85 (Reference consistency check). Reference consistency is the claim-use preflight that verifies version identity, registered classification, permitted strength, discharged conditions, and invocation-site compatibility for every reference.

Proposition 7.86 (Reference data are audit-critical when invoked). *An invoked reference without complete preflight data makes the corresponding theorem-application atom **undefined**; a forbidden or status-inconsistent use makes the claim-use atom **reject**.*

Proof. This is the canonical Appendix F claim-use embedding serialized at the invocation site. □

Remark 7.87 (Reference is not proof). A reference identifies a source and permitted use. It neither proves the source nor discharges its hypotheses.

7.17 Artifacts, integrity, and evidence roles

Definition 7.88 (Artifact). An artifact is an immutable or versioned evidential object supporting one or more manifest fields, including source data, barcodes, profiles, filtered representatives, realization records, comparison morphisms, defect profiles, ledgers, simplex records, formal proofs, computation traces, or governance snapshots.

Definition 7.89 (Artifact reference). An artifact reference stores ArtifactID, URI or storage locator, media/type identifier, version, integrity algorithm, digest or equivalent immutable identity, producer, creation provenance, and supported field roles.

Definition 7.90 (Artifact reference policy). The artifact reference policy declares which artifacts are required for verification, recomputation, explanatory context, or external transport; which fields they support; and how identity and integrity are checked.

Definition 7.91 (Evidence role). Every evidence reference has a declared role such as theorem proof, hypothesis discharge, metric upper bound, artifact existence, identity attestation, deterministic computation, provenance, or explanatory context. Explanatory context cannot satisfy a proof role.

Remark 7.92 (Artifact references are not themselves theorems). Integrity and provenance establish which artifact was used; they do not establish that the artifact proves the claimed mathematics.

Proposition 7.93 (Artifact references are audit-critical). *A required evidence claim lacking an immutable artifact reference or an equivalent self-contained proof object cannot be independently reconstructed.*

Proof. The verifier cannot identify or inspect the alleged evidence. The relevant obligation remains undefined. $\square$

7.18 Repair records and immutable verdict history

Definition 7.94 (Repair record). A repair record stores predecessor run and manifest identifiers, changed fields, changed evidence, reason, responsible schema/theory versions, affected obligations, re-evaluated dependency closure, and new verdict.

Definition 7.95 (Revision relation). The revision relation is a directed acyclic predecessor relation between immutable run records. It is not an in-place overwrite operation.

Proposition 7.96 (No retroactive verdict mutation). *A repair may change the verdict of a new revision but cannot alter the stored status or evidence semantics of its predecessor.*

Proof. The predecessor is identified by immutable run, manifest, schema, and artifact identities. Changing its contents would destroy audit history and violate Appendix F repair semantics. $\square$

7.19 Completeness, consistency, replayability, and audit readability

Definition 7.97 (Audit-readable certificate record). A certificate record is *audit-readable* if an independent verifier can identify the run, reconstruct the required registry and dependency closure, inspect all required evidence, recompute deterministic records, reproduce atomic statuses and the aggregate verdict, and verify every failure route and repair link.

Definition 7.98 (Audit reconstruction). Audit reconstruction is the replay procedure that, under the registered theory and schema versions, recomputes or checks all deterministic profile, ledger, diagnostic, status-normalization, aggregation, and conformance steps used by the run.

Definition 7.99 (Manifest completeness). A manifest is complete if every conditionally required field and required atom is present, typed, versioned, and linked to reconstructible evidence, and every omitted conditional field has a valid scope-exclusion record.

Definition 7.100 (Manifest consistency). A manifest is consistent if shared identifiers have one meaning, duplicate field assignments agree, dependency ranks are well founded, status normalizations agree with source records, artifact identities match, and no field contradicts immutable scope or version boundaries.

Definition 7.101 (Replayability). A manifest is replayable if an independent verifier using only declared immutable artifacts, generators, and registered versions can reconstruct every deterministic verdict computation and identify every non-deterministic or theorem-dependent step with its evidence.

Proposition 7.102 (Minimal Manifest implies audit readability). *If a manifest satisfies the Minimal Manifest Axiom and is complete, consistent, conformant, and replayable, then it is audit-readable. This conclusion does not imply that its aggregate verdict is pass.*

Proof. The hypotheses provide all typed fields, identities, dependencies, evidence, and replay instructions required for reconstruction. The reconstructed statuses may still contain rejection. $\square$

Proposition 7.103 (Audit readability implies verifier independence). *An audit-readable record can be evaluated by a verifier that did not participate in proposing or producing the original candidate, provided the verifier implements the registered theory and schema semantics.*

Proof. The verdict depends on immutable Core inputs and typed evidence, not on proposer identity. $\square$

Remark 7.104 (Relation to Core sovereignty). Replayability supports but does not replace mathematical validity. Search-side confidence, proposal history, or AI ranking is not a verdict field.

7.20 Proposer–verifier separation

Definition 7.105 (Proposer-side data). Proposer-side data include candidate objects, windows, thresholds, maps, rankings, terrain models, validity scores, bridge programs, generated proof sketches, and AI recommendations.

Definition 7.106 (Verifier-side data). Verifier-side data are the typed Core objects, obligation atoms, evidence, artifacts, policies, ledgers, diagnostics, references, statuses, dependencies, and repair records required by the registered audit semantics.

Proposition 7.107 (Only verifier-side data determine Core verdicts). *Proposer-side data may suggest or locate evidence, but cannot alter a Core verdict until normalized into the required Core types, registered, evidenced, and re-evaluated by the verifier.*

Proof. This is Core sovereignty: proposal authority and verdict authority are distinct. $\square$

Remark 7.108 (Recording proposer data). Optional proposer provenance must be tagged as non-verdict data and may not fill a missing verifier field.

7.21 Abstract schema and concrete execution encodings

Definition 7.109 (Abstract certificate schema). The canonical logical schema is

$C = (\text{Identity}, \text{Scope}, \text{Object}, \text{Window}, \text{Threshold}, \text{Collapse}, \text{Evaluation},$
Registry, Dependency, Evidence, Representative, Eligibility, Diagnostic, Comparison,
OverlapRestartAtlas, Ledger, Reference, Artifact, AtomicStatus,
AggregateVerdict, FailureRoutes, Repair, Replay).

Every component is typed, versioned, and conditionally required according to the declared scope.

Remark 7.110 (Schema is abstract but canonical). The tuple fixes logical content and field roles, not punctuation or storage syntax. A JSON object, relational schema, proof-assistant structure, or other encoding may refine it only through a declared conformance map.

Definition 7.111 (Execution schema). An execution schema is a concrete encoding with a total decoding/conformance procedure into the canonical logical schema, plus validation rules for requiredness, types, identities, versions, and integrity references.

Remark 7.112 (Appendix G vs. execution companion). Appendix G fixes what a conformant record means. The Execution/Reproducibility Companion may fix concrete field names, file layouts, APIs, signatures, or replay commands, but may not weaken the logical requirements.

Proposition 7.113 (Concrete execution schemas must refine the abstract schema). *A concrete schema is Core-admissible only if decoding preserves identities, types, statuses, dependencies, evidence roles, routes, repairs, and aggregate semantics.*

Proof. Otherwise two non-equivalent audit records could decode to the same canonical record or one canonical field could be lost, violating semantic-preserving serialization. $\square$

7.22 Schema conformance and validation profiles

Definition 7.114 (Schema conformance). A record is schema-conformant if it passes structural type validation, conditional-requiredness validation, identifier uniqueness, version compatibility, dependency well-foundedness, status normalization, artifact integrity, and cross-field consistency checks.

Definition 7.115 (Validation profile). A validation profile is a named set of required field families for a declared scope, such as `topological_only`, `eligible_ext`, `typeIV`, `overlap_atlas`, `restart`, or `full_foundation`. Profiles control conditional requiredness only; they do not change field meanings.

Proposition 7.116 (Canonical manifest conformance). *A chapter-local or appendix-local fragment is admissible only if it embeds into one canonical manifest without changing shared field meanings, identifiers, statuses, version boundaries, or artifact identities.*

Proof. The canonical manifest represents one immutable run. Incompatible fragments produce manifest inconsistency. $\square$

Definition 7.117 (Status field, legacy label retained). The historical phrase “final status field” is interpreted in v19 as the pair

(AtomicStatusRegistry, AggregateVerdict)

together with a separate claim/use classification field. No mixed enumeration of pass/reject with operational, interface, toy, or spec layers is canonical.

Proposition 7.118 (Schema sufficiency for Core audit). *A conformant, complete, consistent, and replayable record is sufficient to reconstruct a Core audit verdict, but not necessarily sufficient to obtain a pass.*

Proof. The record exposes all inputs and rules required for reconstruction. Reconstruction may yield reject or undefined. $\square$

7.23 Incomplete, inconsistent, and failed records

Definition 7.119 (Incomplete certificate record). A record is incomplete when at least one required field, atom, dependency, evidence reference, coverage certificate, or justified scope exclusion is missing or not reconstructible.

Definition 7.120 (Failed certificate record). A record is failed when enough data are present to evidence at least one required atom as false, giving canonical status `reject`. A failed record may simultaneously contain other undefined atoms.

Definition 7.121 (Inconsistent certificate record). A record is inconsistent when shared identifiers, immutable scope, versions, artifact identities, dependency ranks, source statuses, or duplicate field assignments contradict one another.

Remark 7.122 (Incomplete vs. failed). Incomplete means audit undefinedness for the affected clause; failed means evidenced rejection. Inconsistency is a Type V rejection. The aggregate headline follows Appendix F precedence while retaining all details.

Proposition 7.123 (Failure is preferable to incompleteness). *For diagnosis and repair, an audit-readable failed record is more informative than an incomplete record because it exposes a reconstructible counterevidence path. This does not make rejection normatively preferable to pass.*

Proof. A failed record identifies the atom, evidence, and route. An incomplete record lacks at least one required reconstructive component. $\square$

7.24 Appendix G manifest package

Theorem 7.124 (Appendix G manifest package). *Within the v19 Core audit regime:*

- (i) *Appendix F fixes verdict semantics and Appendix G serializes them without mutation;*
- (ii) *immutable run, manifest, schema, theory, artifact, and revision identities delimit one audit meaning;*
- (iii) *the Minimal Manifest includes the full required registry, dependency closure, evidence, artifacts, statuses, routes, repairs, and replay data;*
- (iv) *canonical atomic statuses are exactly `pass`, `reject`, `undefined`, and justified `not_invoked`;*
- (v) *claim/use classification is separate from atomic status;*
- (vi) *scalar ledgers contain only metric-bearing discrepancies and cannot encode missing non-scalar evidence;*
- (vii) *Type IV zero requires a valid actual comparison artifact and terminal-generic certificates;*
- (viii) *overlap, higher coherence, Restart, target-local pass, and atlas records remain distinct obligations;*

- (ix) *a complete manifest may record rejection, and manifest completeness never implies theorem truth;*
- (x) *countable registries require coverage and universal-status certificates;*
- (xi) *repairs create new immutable revisions rather than mutating historical verdicts;*
- (xii) *conformance, completeness, consistency, and replayability yield audit readability but not automatic pass;*
- (xiii) *proposer data cannot alter verdicts without Core re-entry;*
- (xiv) *concrete execution schemas must refine the canonical logical schema.*

Proof. The items collect the definitions and propositions of this appendix together with Appendix F’s authoritative status, dependency, aggregation, readiness, and repair semantics. □

7.25 Explicit exclusions

The following are not asserted in Appendix G.

- No concrete JSON, YAML, CSV, database, API, signature, or file layout is mandated.
- No schema field creates mathematical evidence.
- No complete manifest is automatically a passing proof.
- No passing mathematical atom repairs an incomplete or inconsistent manifest atom.
- No numerical budget contains claim-status, representative-existence, morphism-existence, eligibility, coherence, or manifest obligations.
- No omitted field receives a theorem-producing default.
- No undefined Type IV record is serialized as (0, 0).
- No `not_invoked` value is accepted without immutable scope-exclusion evidence.
- No claim/use classification is conflated with atomic audit status.
- No headline verdict erases lower-level undefined, reject, blocker, or failure-route detail.
- No finite prefix certifies a countable registry.
- No repair mutates a historical run in place.
- No pairwise overlap family automatically supplies higher Čech coherence.
- No certified transition alone supplies a complete Restart pass.
- No certificate atlas implies object-level descent or a global mathematical object.
- No Search, Platform, Companion, or AI output updates a verdict without Core re-entry.
- No Claim Register entry is proof merely because it is registered.
- No micro-theorem reference discharges its own hypotheses.
- No toy bridge is promoted to an external bridge theorem by serialization.
- No global exactness of positive-threshold collapse or strict filtered lift is assumed.
- No tower comparison artifact is automatic.

7.26 Provenance and status

This appendix consumes:

- (i) Chapter 1 for the foundational Minimal Manifest requirement;
- (ii) Chapters 2–3 and Appendices A–B for evaluation, threshold, and representative records;
- (iii) Chapter 4 and Appendix C for eligible Ext records;
- (iv) Chapter 5 and Appendix D for terminal-generic Type IV records;
- (v) Chapter 6 and Appendix E for overlap, Čech, Restart, and atlas records;
- (vi) Chapter 7 and Appendix F for canonical typed audit semantics, dependency closure, aggregation, readiness, non-compensation, and repair history;
- (vii) Appendix CR-A, Appendix CM, and Appendix TB for governed external references.

Status labels for Appendix G. *Normative schema definitions and axiom:* Definitions [7.6](#), [7.9](#), [7.10](#), [7.13](#), [7.14](#), [7.20](#), [7.26](#), [7.31](#), [7.34](#), [7.109](#), and Axiom [7.1](#).

Contract consequences: Propositions [7.7](#), [7.12](#), [7.23](#), [7.30](#), [7.35](#), [7.75](#), [7.96](#), [7.102](#), [7.116](#), [7.118](#), and Theorem [7.124](#).

Operational cautions and exclusions: Remarks [7.1](#), [7.2](#), [7.8](#), [7.16](#), [7.19](#), [7.55](#), [7.69](#), [7.80](#), [7.87](#), [7.92](#), [7.104](#), and Subsection [7.25](#).